

Complete Study Guide

Generated from the Markdown study system. Labs with executable code remain in the `labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

CCNA Automation 200-901 v1.1 — Complete Study Guide

This is the primary textbook-style study document for **Automating Networks Using Cisco Platforms (200-901 CCNAAUTO) v1.1**. Passing that exam earns the **CCNA Automation** certification.

It is organized by the official Cisco exam blueprint in `200-901-CCNAAUTO_v.1.1.pdf`. Objective numbers, domain names, weights, and command verbs are copied from that document. Do not study a homemade curriculum in place of those objectives.

The Learning Matrix (`Automation-v2.0-Learning-Matrix.xlsx`) was used only as a **secondary** map of books, Cisco U. training, Cisco Live sessions, and online references. Where the matrix still uses older verbs (*Identify*) or older product names (*DNA Center*, *Umbrella*, *Webex Teams*), this guide follows the **PDF**.

This material does **not** guarantee that you will pass. Cisco can include related topics beyond the printed bullets. The goal is exam-aligned understanding plus enough hands-on practice that the blueprint verbs (especially *construct*, *interpret*, *troubleshoot*, and *utilize*) are realistic.

How to use this guide

1. Build the lab in `CCNAAUTO_LAB_SETUP.md`.
2. Follow `CCNAAUTO_STUDY_PLAN.md` (about eight weeks, adjustable).
3. Study each objective in this file. Every objective has five parts:
 - **What Cisco expects me to know** — depth implied by the verb
 - **Detailed explanation** — how the technology works
 - **Syntax and examples** — code, APIs, configs, commands
 - **Exam-style understanding** — original practice ideas, not real exam items
 - **Hands-on exercise** — free labs first
4. Track status in `CCNAAUTO_Study_Tracker.xlsx` (a copy of the matrix; the original file was not modified).
5. Drill `CCNAAUTO_PRACTICE_QUESTIONS.md` with the answer section folded away.
6. In the final days, use `CCNAAUTO_FINAL_REVIEW.md`.

The same chapters also live as smaller files under `study-guide/` if you prefer to open one domain pair at a time.

Exam snapshot

Item	Official fact
Exam	200-901 CCNAAUTO v1.1
Title	Automating Networks Using Cisco Platforms
Certification	CCNA Automation
Duration	120 minutes
Typical item types	Multiple choice, drag-and-drop, performance-based
Languages	English, Japanese
Associated course	Developing Applications and Automating Workflows using Cisco Core Platforms (now titled Automating Networks Using Cisco Platforms)

This exam is the rebranded **DevNet Associate (200-901 DEVASC)** content with updated product names. Treat older DEVASC books as useful, then overlay v1.1 names: Catalyst Center, Catalyst SD-WAN, Secure Connect, Terraform (not Puppet/Chef), CML (not VIRL).

Official topics: <https://learningnetwork.cisco.com/s/ccnaauto-exam-topics>

Domain weights

Domain	Weight	Why it matters
1.0 Software Development and Design	15%	Data formats, Python structure, TDD, Git
2.0 Understanding and Using APIs	20%	Heaviest skill domain with Domain 5
3.0 Cisco Platforms and Development	15%	What each Cisco API is <i>for</i> , plus construct-from-docs
4.0 Application Deployment and Security	15%	Docker, CI/CD, secrets, OWASP, Bash, DevOps
5.0 Infrastructure and Automation	20%	YANG/NETCONF/RESTCONF, Ansible, Terraform, IaC
6.0 Network Fundamentals	15%	Enough networking to automate and troubleshoot apps

Prioritize Domains **2** and **5**, but do not skip 6. Connectivity diagnosis and port numbers are easy points if you know them and easy misses if you assume “this is not CCNA.”

Command verbs (study depth)

Verb	What you must be able to do
Describe	Explain purpose, attributes, and typical use
Compare	Differences, trade-offs, and when to use each option
Explain	Cause/effect and why a design exists
Interpret	Read code, output, a playbook, a diff, a YANG model, or a diagram
Identify	Name the workflow or resource from a given artifact
Construct	Build a request or script from documentation and requirements
Utilize	Perform the operation (Git, Docker, Bash, auth, <code>requests</code>)
Troubleshoot / Diagnose	Given symptoms, find the fault and a fix
Apply	Use the concept in a Cisco setting (especially YANG/NETCONF/RESTCONF)
Recognize	Recall facts such as port numbers

Product names used in v1.1

Blueprint name	You may still see	Exam-relevant idea
Cisco Catalyst Center	DNA Center, DNAC	Intent APIs under <code>/dna/</code>
Cisco Catalyst SD-WAN	vManage, Viptela	Overlay WAN manager API
Cisco NSO	Tail-f NSO	Service orchestration
Webex	Webex Teams	Rooms, messages, memberships
Secure Connect	Umbrella / Secure Access	Cloud security / SASE APIs
Secure Endpoint	AMP for Endpoints	Endpoint event APIs
Secure Malware Analytics	Threat Grid	File/URL detonation API
Cisco Modeling Labs	VIRL	Network simulation
Terraform	Puppet/Chef (removed)	Declarative IaC

Lab map

Full install steps: `CCNAAUTO_LAB_SETUP.md` .

Folder	Primary objectives
labs/01_python_basics	1.5, 1.3, 4.5
labs/02_data_formats	1.1, 1.2
labs/03_rest_api	2.1, 2.4–2.9
labs/04_git	1.7, 1.8, 5.12
labs/05_cisco_apis	3.1–3.5, 3.9
labs/06_yang_netconf_restconf	3.6, 3.8, 5.1, 5.10, 5.11
labs/07_docker	4.6, 4.7
labs/08_ansible	5.6, 5.8
labs/09_terraform	5.5, 5.6
labs/10_network_troubleshooting	6.8, 6.9, 4.9

Free primary resources

Resource	URL	Why
Official exam topics	https://learningnetwork.cisco.com/s/ccnaauto-exam-topics	Blueprint on Cisco Learning Network
Cisco DevNet	https://developer.cisco.com/	API docs, SDKs, Learning Labs
DevNet Sandbox	https://devnetsandbox.cisco.com/	Free IOS XE, NSO, and other labs
Code Exchange	https://developer.cisco.com/codeexchange/	Sample automation projects
Meraki Dashboard API	https://developer.cisco.com/meraki/api-v1/getting-started/	Org/network/device/client calls
Catalyst Center API	https://developer.cisco.com/docs/catalyst-center/getting-started/	Token + inventory
ACI programmability	https://developer.cisco.com/docs/aci/	APIC object model
NSO docs	https://developer.cisco.com/docs/nso/	Service orchestration
Webex APIs	https://developer.webex.com/docs/getting-started	Spaces, people, messages
IOS XE RESTCONF	https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1717/b_1717_programmability_cg/restconf-protocol.html	Official RESTCONF behavior
IOS XE NETCONF	https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/26x/26x-programmability-cg/netconf_protocol.html	Port 830, datastores
NETCONF RFC 6241	https://datatracker.ietf.org/doc/html/rfc6241	Protocol standard
RESTCONF RFC 8040	https://datatracker.ietf.org/doc/html/rfc8040	HTTP mapping of YANG
YANG RFC 7950	https://datatracker.ietf.org/doc/html/rfc7950	Modeling language
Python <code>json</code>	https://docs.python.org/3/library/json.html	Parsing JSON
Python <code>unittest</code>	https://docs.python.org/3/library/unittest.html	Unit tests (4.5)
<code>requests</code>	https://requests.readthedocs.io/	Exam Python HTTP library

Resource	URL	Why
Git documentation	https://git-scm.com/docs	Clone through diff
MDN HTTP status	https://developer.mozilla.org/en-US/docs/Web/HTTP/Status	Response codes
Dockerfile reference	https://docs.docker.com/reference/dockerfile/	Interpret images
Ansible docs	https://docs.ansible.com/	Playbook modules
Terraform docs	https://developer.hashicorp.com/terraform/docs	Plan/apply/state
OWASP Top 10	https://owasp.org/www-project-top-ten/	XSS, SQLi, CSRF
pyATS	https://developer.cisco.com/pyats/	Network testing (5.3)
CML	https://developer.cisco.com/modeling-labs/	Simulation (5.3)
CiscoDevNet netprog_basics	https://github.com/CiscoDevNet/netprog_basics	RESTCONF/NETCONF samples

Paid-but-official (optional, not required to start): *Cisco Certified DevNet Associate DEVASC 200-901 Official Cert Guide* and the Cisco U. CCNAUTO / CCNAUTO course. The Learning Matrix lists these against almost every objective.

Source-material gaps and how this guide filled them

The local PDF lists **what** is tested but not **how** the technologies work. The Learning Matrix points mostly at the Official Cert Guide and Cisco U., which are not in this folder. For those gaps, this guide teaches from Cisco DevNet, Cisco IOS XE programmability guides, IETF RFCs, and vendor docs for Python, Git, Docker, Ansible, and Terraform.

Notable matrix vs blueprint mismatches you should ignore in the spreadsheet:

- 1.6 matrix *Identify* vs blueprint **Explain**
- 2.3 / 2.6 / 3.7 / 4.x several *Identify* vs blueprint **Describe** / **Interpret**
- 3.2 matrix *DNA Center* / *Cisco SD-WAN* vs **Catalyst Center** / **Catalyst SD-WAN**
- 3.5 matrix *Umbrella* vs **Secure Connect**
- 5.6 Puppet/Chef in older DEVASC notes vs **Ansible, Terraform, and Cisco NSO**

Table of contents

- [1.0 Software Development and Design — 15%](#)

- [2.0 Understanding and Using APIs — 20%](#)
- [3.0 Cisco Platforms and Development — 15%](#)
- [4.0 Application Deployment and Security — 15%](#)
- [5.0 Infrastructure and Automation — 20%](#)
- [6.0 Network Fundamentals — 15%](#)

A one-page objective-to-section map is in `CCNAAUTO_OBJECTIVE_CHECKLIST.md`.

1.0 Software Development and Design — 15%

Software Development and Design is the foundation of CCNA Automation (200-901 CCNAAUTO v1.1). The exam is 120 minutes. This domain is weighted at **15%**, which is smaller than APIs (20%) or Cisco platforms, but it is the language the rest of the exam speaks: data formats, Python structure, testing, process models, and Git.

You are not being tested as a full-time software engineer. You are being tested as a network engineer who can read, write, and reason about the artifacts that automation produces. A RESTCONF payload is JSON or XML. An Ansible playbook is YAML. A webhook is an Observer-style callback. A failed merge is a Git conflict. If you cannot compare JSON to YAML, parse either into a Python `dict`, or explain why a function is better than a 200-line script, later domains become guesswork.

This chapter teaches Domain 1 completely, then Domain 2 (Understanding and Using APIs, 20%). Domain 2 is one of the two heaviest domains on the exam; treat it as core, not as an appendix. Official Cisco developer material lives at <https://developer.cisco.com/>. Python's JSON library is documented at <https://docs.python.org/3/library/json.html>. Git's reference is <https://git-scm.com/docs>.

Nothing in this file is a claimed Cisco exam question. Practice items are original scenarios written to match the **blueprint verbs**: Compare, Describe, Explain, Utilize, Construct, Interpret, Troubleshoot.

1.1 Compare data formats (XML, JSON, and YAML)

1. What Cisco expects me to know

Compare means differences, advantages, disadvantages, and use cases — not a one-sentence definition of each format.

You should be able to look at a snippet and name the format; map each format to where Cisco automation actually uses it; and choose a format for a given job (REST body, NETCONF RPC, Ansible inventory, human-edited config).

Format	Typical automation home	What you must notice on sight
JSON	REST / RESTCONF bodies, most modern Cisco APIs	<code>{ }</code> , <code>[]</code> , <code>"keys"</code> , <code>true / false / null</code> , no comments
XML	NETCONF, SOAP, some YANG encodings, device configs	<code><element attr=" ... "></code> , namespaces (<code>xmlns</code>), verbose tags
YAML	Ansible playbooks/inventory, CI files, Kubernetes-style manifests	Indentation, <code>-</code> lists, <code>#</code> comments, unquoted keys

2. Detailed explanation

All three formats serialize the same kind of information: nested maps (key/value), lists, strings, numbers, and booleans. They differ in **syntax**, **strictness**, **human editability**, and **ecosystem**.

JSON (JavaScript Object Notation) is a data interchange format, not a programming language. A JSON document is one value: an object, an array, a string, a number, `true`, `false`, or `null`. Objects are unordered collections of string keys mapped to values. Arrays are ordered lists. Strings use double quotes only. Booleans are lowercase `true` and `false`. Absence of a value is `null`, not `None` or `nil`. JSON has **no comments**. If you see `//` or `#` inside a payload, it is not legal JSON (some parsers tolerate it; the standard does not). JSON is the default for REST APIs because it is compact, maps cleanly onto Python `dict` / `list`, and is easy to generate from code.

XML (eXtensible Markup Language) is a tagged tree. Every piece of data lives in an **element** (`<name>GigabitEthernet1</name>`) or an **attribute** (`<interface name="GigabitEthernet1">`). Elements can nest. Attributes cannot nest — they are string pairs on a start tag. **Namespaces** (`xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"`) disambiguate vocabularies so two standards can both use an `<interface>` element without colliding. XML is **verbose**: every value is wrapped in a start tag and an end tag. That verbosity is a feature for protocols that need explicit structure, schemas (XSD), and mixed content. In Cisco automation you meet XML primarily through **NETCONF** (RPCs wrapped in XML) and older **SOAP** APIs. RESTCONF can return XML, but JSON is more common in study material.

YAML (YAML Ain't Markup Language) is a human-oriented serialization format. Structure is defined by **indentation** (spaces, never tabs mixed with spaces). Lists use `-`. Mappings use `key: value`. Comments start with `#`. YAML is a **superset of JSON in practice**: a valid JSON document is typically valid YAML, so an Ansible playbook can contain a JSON blob and still parse. The reverse is not true — YAML comments, unquoted strings, and `& / *` anchors are not JSON. Ansible, GitHub Actions, Docker Compose, and many CI systems standardized on YAML because operators edit it by hand.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    subgraph same ["Same logical data"]
        I["interfaces: name, ip, enabled"]
    end
    I --> JSON["JSON: REST / RESTCONF"]
    I --> XML["XML: NETCONF / SOAP"]
    I --> YAML["YAML: Ansible / CI"]
```

Advantages and disadvantages

	JSON	XML	YAML
Compact	High	Low	Medium
Comments	None	<!-- -->	#
Namespaces	No native model	First-class	No native model
Attributes vs elements	N/A (only keys)	Must choose	N/A
Human editing	Easy for small docs	Painful	Designed for it
Strictness	Rigid syntax	Rigid + schemas	Indentation-sensitive
Typical failure	Trailing comma, comments	Namespace miss, unclosed tag	Wrong indent, tab/space mix
Cisco-shaped use	Meraki, Catalyst Center, RESTCONF JSON	NETCONF <rpc>, YANG XML	Ansible playbooks

Use-case rule of thumb: talking to a REST API → JSON. Talking to NETCONF → XML. Writing an Ansible playbook or GitHub workflow → YAML. Do not “convert in your head” by changing quotes; convert by parsing into a data structure, then serializing (objective 1.2).

3. Syntax and examples

The same two interfaces appear in `labs/02_data_formats/` as JSON, YAML, and XML. Study the three files side by side.

JSON object and array

```
{
  "interfaces": [
    {
      "name": "GigabitEthernet1",
      "enabled": true,
      "ipv4": { "address": "10.10.20.48", "prefix": 24 }
    }
  ]
}
```

- `{ }` starts an **object** (Python `dict`).
- `[ ]` starts an **array** (Python `list`).
- Keys are always double-quoted strings.
- `true` is a boolean, not the string `"true"`. `24` is a number, not `"24"`.
- No trailing comma after the last property. No comments.

YAML equivalent (from `labs/02_data_formats/interfaces.yaml`)

```
# YAML allows comments. JSON does not.
interfaces:
  - name: GigabitEthernet1
    enabled: true
    ipv4:
      address: 10.10.20.48
      prefix: 24
```

- The `#` line is a comment.
- `interfaces:` is a mapping key whose value is a list.
- `-` starts a list item. Nested keys under that item are indented **further** (typically 2 spaces).
- `true` and `24` are still boolean and integer after parsing — YAML infers types unless you quote.

XML equivalent (from `labs/02_data_formats/interfaces.xml`)

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  <interface>
    <name>GigabitEthernet1</name>
    <enabled>true</enabled>
    <ipv4>
      <address>10.10.20.48</address>
      <prefix>24</prefix>
    </ipv4>
  </interface>
</interfaces>
```

- `xmlns` declares the default **namespace**. Child elements inherit it.
- `<enabled>true</enabled>` is **text**, not a native boolean. Your parser must interpret `"true"` as boolean if you want Python `True`.
- Attributes would look like `<interface name="GigabitEthernet1">`. This sample uses child elements instead — both are legal XML; NETCONF/YANG models usually prefer elements.

Whitespace and quoting traps

Snippet	Format	Verdict
<code>{"enabled": True}</code>	JSON	Invalid. JSON requires <code>true</code> .
<code>{"enabled": true,}</code>	JSON	Invalid. Trailing comma.
<code>enabled: yes</code>	YAML	Often becomes boolean <code>True</code> (YAML 1.1). Quote if you need the string.
<code><enabled>true</enabled></code>	XML	Text node <code>"true"</code> .

4. Exam-style understanding

You should be able to:

1. Identify a blob as JSON, XML, or YAML in one glance (braces vs tags vs indentation).

2. State **one advantage and one disadvantage** of each format relative to the other two.
3. Match a Cisco technology to a format: RESTCONF JSON, NETCONF XML, Ansible YAML.
4. Spot illegal JSON: comments, single quotes, `True / None` , trailing commas.
5. Explain why YAML is preferred for Ansible: comments and indentation are easier for humans than JSON braces.

Original practice scenario. An engineer pastes this into a REST client as a JSON body:

```
{
  // management interface
  'name': 'GigabitEthernet1',
  "enabled": True,
}
```

Three independent JSON errors: a `//` comment, single-quoted strings, Python `True` , and a trailing comma. A correct JSON body is `{"name": "GigabitEthernet1", "enabled": true}` . If the same data were an Ansible var file, YAML with a `#` comment would be appropriate.

5. Hands-on exercise

Free tools: a text editor and Python 3. No Cisco gear.

1. Open `labs/02_data_formats/interfaces.json` , `interfaces.yaml` , and `interfaces.xml` .
2. On paper, list every difference you can see: comments, quotes, namespaces, booleans as text vs native, indentation vs tags.
3. Break JSON on purpose: add a comment, change `true` to `True` , add a trailing comma. Save as `interfaces.bad.json` . Confirm that `python -c "import json; json.load(open('interfaces.bad.json'))"` fails, and read the error.
4. In YAML, change indentation of `prefix` by one space and run `python parse_formats.py` from `labs/02_data_formats/` . Observe that YAML fails on structure, not on a missing brace.
5. Write a four-row comparison table in your notes: compactness, comments, Cisco use, typical parse error.

1.2 Describe parsing of common data format (XML, JSON, and YAML) to Python data structures

1. What Cisco expects me to know

Describe means explain the concept: what “parse” does, which Python types you get, and which library functions you use. You are not required to memorize every XML XPath feature, but you must know the standard library path for JSON, a safe YAML load, and ElementTree for XML.

Target mapping:

On the wire	After a correct parse in Python
JSON object / YAML mapping	<code>dict</code>
JSON array / YAML list	<code>list</code>
JSON/YAML string	<code>str</code>
JSON/YAML number	<code>int</code> or <code>float</code>
JSON <code>true</code> / <code>false</code>	<code>True</code> / <code>False</code> (<code>bool</code>)
JSON <code>null</code> / YAML <code>null</code>	<code>None</code>
XML element tree	<code>Element</code> objects; you extract text yourself

Official JSON docs: <https://docs.python.org/3/library/json.html>.

2. Detailed explanation

Parsing converts a string (or file bytes) that follows a format's grammar into in-memory objects your program can index, loop, and branch on. **Serializing** (dumping) is the reverse: Python objects become a string you can send in an HTTP body or write to disk.

Until you parse, `"{\\"enabled\\": true}"` is just a string. After `json.loads`, it is `{"enabled": True}` and `data["enabled"]` is `True` works.

JSON in the standard library (`json`)

- `json.loads(s)` — parse a **string** (`s` = string).
- `json.load(fp)` — parse an open **file** object.
- `json.dumps(obj)` — serialize a Python object to a **string**.
- `json.dump(obj, fp)` — serialize to a **file**.

Remember the `s`: `loads` / `dumps` work on strings. `load` / `dump` work on files. Mixing them is a common bug (`json.load` on a string raises).

JSON objects become `dict`. Arrays become `list`. Nested structures stay nested. `true` / `false` / `null` become `True` / `False` / `None`. JSON object keys are always strings, so `data[0]` is wrong if the root is an object; `data["interfaces"]` is right.

YAML (`yaml.safe_load`)

PyYAML is not in the standard library; this repo installs it via `labs/requirements.txt`. Use `yaml.safe_load`, not `yaml.load` without a `Loader`. `safe_load` refuses to construct arbitrary Python objects from YAML tags, which would be a code-execution risk. After a safe load, YAML mappings and sequences look like JSON's `dict` and `list`. That is why `parse_formats.py` can compare the three parsed results with `==`.

XML (`xml.etree.ElementTree`)

XML does **not** become a `dict` automatically. `ElementTree` gives you a tree of `Element` nodes. You walk with `.find()`, `.findall()`, `.findtext()`, and you must handle **namespaces**. Text inside an element is a string. There is no automatic `true` $\rightarrow$ `True`.

Namespaces are the exam-relevant trap. If the document has `xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"`, then `root.findall("interface")` often returns **nothing**. You search with a Clark notation `{uri}interface` or a prefix map, as the lab does with `NS = {"if": "urn:ietf:params:xml:ns:yang:ietf-interfaces"}` and `"if:interface"`.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    A["Raw string or file"] --> B["Format"]
    B --> C["JSON | json.loads / json.load"]
    B --> D["YAML | yaml.safe_load"]
    B --> E["XML | ET.parse / ET.fromstring"]
    C --> F["dict / list / str / int / bool / None"]
    D --> F
    E --> G["Element tree → extract text → your own dict"]
```

Why this matters for APIs. `response.json()` on a `requests` `Response` is `json.loads` on the body. If you print `response.text` you still have a string. Automation logic (`if iface["enabled"]:`) only works after parsing.

3. Syntax and examples

The teaching implementation is `labs/02_data_formats/parse_formats.py`.

JSON: string vs file

```
import json
from pathlib import Path

text = Path("interfaces.json").read_text(encoding="utf-8")
data = json.loads(text)          # str → dict
# equivalently:
with open("interfaces.json", encoding="utf-8") as f:
    data = json.load(f)          # file → dict

first_name = data["interfaces"][0]["name"] # str
enabled = data["interfaces"][0]["enabled"] # bool
prefix = data["interfaces"][0]["ipv4"]["prefix"] # int

out = json.dumps(data, indent=2) # dict → str (pretty)
```

`data["interfaces"]` is a `list`. Index `0` is a `dict`. Nested `ipv4` is another `dict`. Wrong: `data.interfaces` (that is JavaScript, not Python).

YAML

```
import yaml

with open("interfaces.yaml", encoding="utf-8") as f:
    data = yaml.safe_load(f)
# data["interfaces"][0]["name"] works the same as JSON
```

`safe_load` returns native Python types. Do not use `yaml.load(f)` in automation code.

XML with a namespace

```
from xml.etree import ElementTree as ET

NS = {"if": "urn:ietf:params:xml:ns:yang:ietf-interfaces"}
root = ET.parse("interfaces.xml").getroot()
for iface in root.findall("if:interface", NS):
    name = iface.findtext("if:name", default="", namespaces=NS)
    enabled_text = iface.findtext("if:enabled", default="", namespaces=NS)
    enabled = enabled_text == "true" # convert text → bool
```

`findtext` returns a string or `None`. The lab converts enabled with `== "true"` because XML has no boolean type.

Round-trip check. After parsing all three files, the lab prints `All three agree: True` when the extracted Python lists match. That is the point of parsing: format is a serialization choice; the data structure is what your code owns.

4. Exam-style understanding

You should be able to:

1. Given a JSON string, say which Python type `json.loads` produces at the root (`dict` vs `list`).
2. Choose `loads` vs `load` vs `dumps` vs `dump` for a described task.
3. Explain why `yaml.safe_load` is preferred over `yaml.load` .
4. Explain why an `ElementTree` `findall("interface")` can return `[]` on a namespaced document.
5. Predict `type(value)` after parse: JSON `null` → `None` ; JSON `true` → `bool` ; XML `<enabled>true</enabled>` → `str` until you convert.

Original practice scenario. Code does `print(payload["hostname"])` immediately after `r = requests.get(url)` . If `payload = r.text` , that is a string and `payload["hostname"]` is indexing characters, not a key. The fix is `payload = r.json()` (or `json.loads(r.text)`). Same idea as objective 1.2 applied to HTTP (Domain 2).

5. Hands-on exercise

From `labs/02_data_formats/` :

```
python parse_formats.py
```

You should see JSON, YAML, and XML all yield `GigabitEthernet1` and `All three agree: True`.

Then, in a Python REPL:

1. `json.loads('{"a": true, "b": null}')` — confirm types with `type( ... )`.
2. `json.dumps({"enabled": True, "desc": None})` — confirm `true` and `null` on the wire.
3. Remove the `xmlns` handling in a **copy** of `parse_xml` and watch `findall` return an empty list.
4. Call `yaml.safe_load` on the contents of `interfaces.json` (JSON as YAML). It should parse. This is the “YAML is a superset of JSON in practice” fact.

1.3 Describe the concepts of test-driven development

1. What Cisco expects me to know

Describe the TDD loop and why it exists. Know **red–green–refactor**, that tests are written **before** production code, and that Python’s common tools are `unittest` (standard library) and `pytest` (popular third-party). Domain 4 also asks you to construct a unit test; this objective is the **concept**.

2. Detailed explanation

Test-driven development (TDD) is a design method, not a test-after checklist. You specify behavior with an automated test that **fails**, then write the smallest implementation that makes it **pass**, then **refactor** while the tests stay green.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    R["RED: write a failing test"] --> G["GREEN: write just enough code"]
    G --> F["REFACTOR: clean up, tests stay green"]
    F --> R
```

Red. You write a test for a behavior you want, for example “a `/24` has 254 usable hosts.” You run the test runner. It fails because the function does not exist or returns the wrong value. Failure is required: a test that never failed might not be testing anything.

Green. You implement `prefix_to_hosts` until that test passes. You do not add features the tests do not demand.

Refactor. You rename, extract functions, or simplify, **without changing behavior**. The tests are the safety net. If a refactor breaks a test, you broke behavior.

Why network automation cares. Device-facing scripts fail in production in expensive ways (wrong ACL, wrong VLAN). TDD forces you to state expected outputs (`254` hosts, `/32` → `1`) before you talk to a router. You still need integration tests against APIs later; TDD’s unit tests cover pure logic (subnet math, payload shaping, status-code mapping).

unittest vs pytest

	unittest	pytest
Ships with Python	Yes	No (<code>pip install pytest</code>)
Test discovery	Classes subclassing <code>TestCase</code> , methods <code>test_*</code>	Files <code>test_*.py</code> , functions <code>test_*</code>
Assertions	<code>self.assertEqual</code> , <code>self.assertRaises</code>	Plain <code>assert</code>
Exam-adjacent	Matches “construct a unit test” with <code>stdlib</code>	Same idea, less boilerplate

TDD is independent of the runner. The concept is the cycle. The lab uses `unittest` so you have zero extra packages.

What TDD is not. It is not “write tests someday.” It is not 100% coverage theater. It is not a substitute for reading API documentation. It also does not require you to test Cisco’s OS — you test **your** functions.

3. Syntax and examples

The lab `labs/01_python_basics/test_subnet.py` is a TDD artifact. The tests were the specification; `prefix_to_hosts` in `functions_classes_modules.py` is the implementation.

```
import unittest
from functions_classes_modules import prefix_to_hosts

class TestPrefixToHosts(unittest.TestCase):
    def test_slash_24(self):
        self.assertEqual(prefix_to_hosts("192.168.1.0/24"), 254)

    def test_invalid_raises(self):
        with self.assertRaises(ValueError):
            prefix_to_hosts("not-an-ip")
```

- `unittest.TestCase` gives you assertion methods.
- Method names **must** start with `test` or the runner skips them.
- `assertEqual(actual, expected)` fails with a diff if they differ.
- `assertRaises(ValueError)` passes only if that exception is raised.

Run:

```
cd labs\01_python_basics
python -m unittest test_subnet.py
```

A passing run prints `OK` . If you change `254` to `255` , you get `FAIL` and a traceback — that is **red**.

TDD sequence for this lab (do it once on purpose):

1. Rename `prefix_to_hosts` temporarily so import fails → collection error / fail (**red**).
2. Restore the function with `return 0` → tests fail on values (**red**).

3. Implement real math until **green**.
4. Refactor comments or helper names; re-run; still **green**.

4. Exam-style understanding

You should be able to:

1. Name the three phases in order: red, green, refactor.
2. State that tests are written **first**.
3. Recognize `unittest` code: `TestCase`, `test_*`, `assertEqual`.
4. Explain one benefit: specification before implementation; safer refactor; regression detection.
5. Distinguish TDD (process) from a unit test (artifact). You can have unit tests without TDD; TDD always produces tests first.

Original practice scenario. A colleague writes a 400-line script that pushes VLANs, then asks “how do we TDD this?” The honest answer: extract pure functions (validate VLAN ID 1–4094, build the JSON body), TDD those, and keep the HTTP call thin. You do not TDD the network itself with `unittest` alone.

5. Hands-on exercise

1. Run `python -m unittest test_subnet.py` from `labs/01_python_basics/`. Confirm `OK`.
2. Change `test_slash_24` expected value to `255`, run again, read the failure (red).
3. Revert the test. In `prefix_to_hosts`, temporarily `return 254` for every prefix. See which tests fail (`/30`, `/32`) — this shows why **multiple tests** exist.
4. Add a new test `test_slash_31` for `10.0.0.0/31` **before** you look at the implementation. Run (red or green depending on the lab’s `/31` rule). Align your understanding with the docstring: `/31` and `/32` return `num_addresses` with no `- 2`.
5. Optional: `pip install pytest` and run `pytest test_subnet.py` — `pytest` can run `unittest` tests. Same cycle, different runner.

1.4 Compare software development methods (agile, lean, and waterfall)

1. What Cisco expects me to know

Compare agile, lean, and waterfall: how work is sequenced, how requirements change is treated, where documentation sits, and which situations fit each method. Cisco automation programs in enterprises often **say** Agile while still having waterfall-like change windows. You need the textbook comparison plus that operational reality.

2. Detailed explanation

A **software development method** (or lifecycle model) is a policy for **when** you gather requirements, design, code, test, and release. The exam names three: waterfall, agile, and lean.

Waterfall is sequential. You finish requirements, then design, then implementation, then testing, then operations. You do not (in the pure model) go back. Documentation is heavy because the next phase inherits a frozen spec. Change is expensive: a missed VLAN requirement discovered in testing means revisiting design. Waterfall fits **well-understood, highly regulated, hard-to-reverse** work (a datacenter cutover with a fixed maintenance window and a signed design). It fails when the customer cannot know the full API surface on day one.

Agile is iterative and incremental. Work is split into short cycles (often called sprints). Each cycle aims to produce **working software** (a script that actually creates a Webex room, not a 40-page design that nobody ran). Requirements are expected to change. Collaboration with the “customer” (the network team consuming the automation) is continuous. Agile is a family (Scrum, Kanban, XP); the exam-level idea is iteration, feedback, and adapting scope. Ceremony (stand-ups, boards) is a means, not the definition.

Lean comes from manufacturing (Toyota) and focuses on **eliminating waste**: unused features, waiting, handoffs, defects, extra inventory (including “inventory” of unfinished Git branches). Lean prefers small batches, fast flow, and measuring value. Kanban boards, limiting work-in-progress, and “stop the line” on defects are lean-flavored. Agile and lean overlap; lean is the waste/flow lens, agile is the iterative-delivery lens.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph water ["Waterfall"]
        W1[Requirements] --> W2[Design] --> W3[Implement] --> W4[Test] --> W5[Operate]
    end
    subgraph ag ["Agile"]
        A1["Sprint N: plan"] --> A2[Build a slice] --> A3["Demo / feedback"] --> A1
    end
    subgraph ln ["Lean"]
        L1[See the value stream] --> L2[Cut waste] --> L3[Smaller batch] --> L1
    end
```

Dimension	Waterfall	Agile	Lean
Sequence	Linear phases	Repeated short cycles	Continuous flow
Requirements	Frozen early	Expected to change	Pull only what creates value
Documentation	Heavy, phase-gated	“Working software over comprehensive docs” (not zero docs)	Docs that do not add value are waste
Feedback	Late (after test phase)	Every iteration	Continuous; defects stop the line
Change cost	High late	Absorbed in backlog	Reduce by smaller batches
Risk	Late integration surprises	Scope may churn	Starvation if WIP is unmanaged
Fits	Fixed contract, compliance, known design	Evolving APIs, automation platforms	Pipeline efficiency, ops + dev flow
Pitfall	Months of design, untested assumptions	“Agile” as no planning	Cutting “waste” that was actually compliance

Advantages / disadvantages (say them out loud)

- Waterfall advantage: predictable milestones and audit trail. Disadvantage: late discovery of wrong assumptions.
- Agile advantage: early working automation and course correction. Disadvantage: can thrash if the product owner never says no; needs discipline.
- Lean advantage: shorter lead time, less unfinished work. Disadvantage: easy to misread “waste” and delete necessary review or testing.

In network automation, pushing a routing change is often **waterfall-like** (CAB, change window), while **building** the Python/Ansible that will do it can be agile (weekly stories: parse inventory, then dry-run, then apply). Lean shows up in CI: do not pile 40 unreviewed PRs.

3. Syntax and examples

These methods are not code syntax. The “syntax” is how you would **plan** the same automation job three ways.

Job: “Add a REST script that sets hostname on 50 IOS XE boxes via RESTCONF.”

Waterfall plan

1. Requirements document: auth method, YANG path, rollback, success criteria.
2. Design: sequence diagram, error matrix, all 50 hostnames.
3. Implement the entire script.
4. Test in a lab, then a maintenance window.
5. Operate / hand off.

You would not demo a half-script to stakeholders mid-phase.

Agile plan

- Sprint 1: one Always-On DevNet box, GET hostname, show JSON. Demo.
- Sprint 2: POST/PATCH hostname on one box with dry-run logging. Demo.
- Sprint 3: loop inventory YAML, add unittest for payload builder, add 401/404 handling.
- Backlog can insert “use token instead of basic auth” when security asks.

Lean plan

- Map the value stream: request → Git PR → CI → sandbox → production.
- Waste: waiting two weeks for a shared sandbox; re-typing hostnames; merging only once a month.
- Change: smaller PRs, automated tests (`test_subnet.py` style for payload functions), pull from a Kanban column with WIP limit 2.

A GitHub Actions file (see `labs/04_git/workflow.yaml` as a checklist of Git operations, not a full CI engine) is a **lean/agile artifact**: automate checks so waiting on a human to “remember to run unittest” is not in the path.

4. Exam-style understanding

You should be able to:

1. Identify a process description as waterfall (phases, freeze, test at end), agile (sprints, working increment, changing requirements), or lean (waste, small batch, flow).
2. Give one advantage and one disadvantage for each.
3. Pick a method for a scenario: nuclear change window vs evolving internal API vs reducing ticket cycle time.
4. Avoid the trap “Agile has no documentation” — Agile de-emphasizes **comprehensive up-front** docs, not runbooks.
5. Avoid the trap “Lean is just Agile” — related, different emphasis.

Original practice scenario. A team spends six months writing a “fully complete” NSO service, never loads it on a device, then discovers the YANG model in production differs. That is waterfall risk (late feedback). An agile team would have applied a thin service to a sandbox in week two. A lean reading: the inventory of untested design was waste.

5. Hands-on exercise

Free: a whiteboard or markdown file. No paid ALM tool required. A GitHub project board on a free account is enough if you want a board.

1. Write the same three-bullet plan for **your** next lab (for example Domain 2 `rest_client.py`) in waterfall, agile, and lean form.
2. Time-box an agile slice: 45 minutes to get `GET https://httpbin.org/get` working and committed, then stop. That is an increment.

3. Lean audit: list waste in your study setup (reinstalling packages globally instead of a venv, committing `.env`, never running tests). Remove one waste item.
 4. Explain to a peer (or a voice note) why a firmware upgrade on a core pair is often waterfall while a Meraki dashboard script is often agile.
-

1.5 Explain the benefits of organizing code into methods / functions, classes, and modules

1. What Cisco expects me to know

Explain the benefits — not “define function.” Why split a script? What does a class buy you? What is a module? Cisco’s wording uses **methods / functions, classes, and modules**. In Python, a **function** is `def`; a **method** is a function on a class (`self`); a **module** is a `.py` file you import.

2. Detailed explanation

A 300-line `automate.py` that mixes HTTP, JSON parsing, VLAN math, and print statements will work once. It will not work for the exam’s mental model or for a team.

Functions (and methods) name a unit of work with inputs and outputs. Benefits:

- **Reuse:** `prefix_to_hosts("10.0.0.0/30")` from tests, CLI, and a REST wrapper without copy-paste.
- **Testability:** TDD (1.3) needs a callable. You cannot unit-test “lines 80–140 of a script” cleanly.
- **Readability:** `build_inventory(raw)` states intent; a nested loop of dict unpacking does not.
- **Scope:** Local variables stay local. You stop colliding on `data =`.
- **Single responsibility:** One function parses; another posts; another logs. Failures have a smaller blast radius.

A **method** is a function bound to an object. `iface.shutdown()` mutates that interface’s `enabled` flag. The benefit over a loose function `shutdown(iface)` is encapsulation: the object carries its data and the operations that are legal on it.

Classes group **state + behavior**. Benefits:

- Model domain objects (`Interface` , `Device` , `Prefix`) the way the network already thinks.
- Invariants live in one place (`__init__` validates name).
- Multiple instances: 50 interfaces without 50 global variables.
- Foundation for patterns in 1.6 (Model in MVC is typically a class).

You do not need inheritance trees for this exam. A small class with `__init__` and two methods is enough.

Modules are files (and packages are directories of files). Benefits:

- **Namespace:** `from functions_classes_modules import prefix_to_hosts` vs pasting the function into every script.

- **Reuse across programs:** `parse_formats.py` functions could be imported by a RESTCONF tool later.
- **Team parallel work:** one module per concern (http, inventory, tests).
- `if __name__ == "__main__":` the file is both a library and a runnable demo. Tests import it without executing the demo prints.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TB
    M["module: functions_classes_modules.py"]
    M --> F["function: prefix_to_hosts"]
    M --> C["class: Interface"]
    C --> MD["methods: shutdown, summary"]
    M --> F2["function: build_inventory"]
    T["module: test_subnet.py"] --> F
  
```

Disadvantage of not organizing: copy-paste drift (one file subtracts 2 hosts on `/32`, another does not), untestable scripts, import cycles if you split badly, and “god classes.” The benefit of organization is not extra files for their own sake; it is **boundaries**.

3. Syntax and examples

Lab: `labs/01_python_basics/functions_classes_modules.py`.

Function

```

from ipaddress import ip_network

def prefix_to_hosts(prefix: str) → int:
    net = ip_network(prefix, strict=False)
    if net.prefixlen ≥ 31:
        return net.num_addresses
    return net.num_addresses - 2
  
```

- `def` names the function. Parameters are inputs. `return` is the output.
- Call site: `prefix_to_hosts("192.168.10.0/24") → 254`.
- Benefit on display: `/31` and `/32` special cases live in **one** place.

Class and methods

```
class Interface:
    def __init__(self, name: str, ip: str, enabled: bool = True):
        self.name = name
        self.ip = ip
        self.enabled = enabled

    def shutdown(self) → None:
        self.enabled = False

    def summary(self) → dict:
        return {"name": self.name, "ip": self.ip, "enabled": self.enabled}
```

- `__init__` is the constructor. `self` is the instance.
- `shutdown` is a **method**: it uses instance state.
- `summary` returns a `dict` ready to become JSON (1.1 / 2.9).

Module import

```
from functions_classes_modules import prefix_to_hosts
```

`test_subnet.py` imports the function — that is the module benefit. The demo under `if __name__ == "__main__":` does not run during `unittest`.

Factory function using the class

```
def build_inventory(raw: list[dict]) → list[Interface]:
    return [Interface(**item) for item in raw]
```

`**item` unpacks dict keys into constructor arguments. One function converts API-like JSON lists into objects.

4. Exam-style understanding

You should be able to:

1. List benefits: reuse, testing, readability, encapsulation, namespace.
2. Distinguish function vs method vs module in a snippet.
3. Explain why tests import a module instead of exec'ing a script.
4. Recognize that a class is appropriate when you have **multiple instances with the same operations**.
5. Reject “modules are only for huge programs” — even two files (`functions_classes_modules.py` + `test_subnet.py`) justify a module.

Original practice scenario. Two scripts both contain a copied `prefix_to_hosts`. A bugfix lands in one. Production uses the other. Organizing into a module with tests would have made the fix universal and verified.

5. Hands-on exercise

```
cd labs\01_python_basics
python functions_classes_modules.py
python -m unittest test_subnet.py
```

Then:

1. Add a method `no_shutdown` that sets `enabled = True`. Call it from the `__main__` block.
2. Move `prefix_to_hosts` into a new file `subnetmath.py` and fix the import in the test. You just created a second module.
3. Note what broke if you forgot to update the import — that is the module boundary teaching you its job.

1.6 Explain the advantages of common design patterns (MVC and Observer)

1. What Cisco expects me to know

Explain the advantages of MVC and Observer. You do not need the Gang of Four book. You need: what problem each pattern solves, how the parts collaborate, and why automation/API systems use them. **Webhooks and event notifications are Observer in network clothing** — Domain 2.2 will reuse this.

2. Detailed explanation

A **design pattern** is a reusable arrangement of types and message flow. Two patterns appear on the blueprint.

MVC — Model–View–Controller

- **Model:** data and rules. In a web GUI for switches, the Model is the interface inventory, VLAN membership, “enabled” flags — possibly classes like `Interface`. The Model does not know about HTML or REST URL paths.
- **View:** presentation. A table of interfaces in a browser, a CLI table, or a JSON pretty-print. The View should not contain subnet math.
- **Controller:** input handling. HTTP request arrives; Controller parses parameters, calls the Model, picks a View. In a REST API, the “controller” is the route handler that maps `GET /interfaces` to `inventory.list()`.

Advantage of MVC: you can change the View (Webex Adaptive Card vs HTML vs JSON API) without rewriting business rules. You can test the Model with `unittest` without a browser. Multiple Controllers (REST, CLI, chatbot) can share one Model. The cost of mixing all three is the classic “script that prints while it mutates while it parses.”

In Cisco’s world, a dashboard (Catalyst Center UI) is a View; the device inventory is the Model; REST endpoints are Controllers for programmatic Views (`application/json`).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    U[User or API client] --> C[Controller]
    C --> M[Model]
    M --> C
    C --> V[View]
    V --> U
```

Observer

A **subject** (publisher) maintains a list of **observers** (subscribers). When the subject's state changes, it **notifies** observers. Observers do not poll in a tight loop. They react.

Advantage of Observer: loose coupling. The switch does not know what will happen when an interface goes down — a syslog collector, a webhook to Webex, a ticket bot. You add observers without editing the subject's core logic. That is how **webhooks** work: you subscribe a URL; the platform POSTs when an event occurs (2.2). MQTT, syslog, and “event-driven Ansible” are the same idea.

Advantage vs polling: less delay, less wasted GET traffic, but you must handle retries, authentication of the callback, and failure of observers (the subject should not crash if one observer is down).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    S[Subject: interface state] -->|notify| O1[Observer: logging module]
    S -->|notify| O2[Observer: webhook to https://hooks.example/ccna]
    S -->|notify| O3[Observer: metrics]
```

MVC vs Observer. MVC structures **layers** of an application. Observer structures **notifications**. A Controller might **observe** a Model and refresh a View — the patterns compose.

3. Syntax and examples

MVC sketched in the same objects as the lab

```
# Model
class Interface:
    def __init__(self, name, enabled=True):
        self.name = name
        self.enabled = enabled

# Controller (would be a Flask route or CLI command)
def shutdown_controller(inventory, name):
    iface = next(i for i in inventory if i.name == name)
    iface.shutdown()
    return iface.summary() # data for the View

# View
def print_view(summary: dict) → None:
    print(f"{summary['name']}: enabled={summary['enabled']}")
```

Advantage on display: `print_view` can be replaced by `json.dumps` for an API without touching `Interface.shutdown`.

Observer sketched

```
class InterfaceSubject:
    def __init__(self, name):
        self.name = name
        self.enabled = True
        self._observers = []

    def attach(self, fn):
        self._observers.append(fn)

    def shutdown(self):
        self.enabled = False
        for fn in self._observers:
            fn(self.name, "down")

def webhook_observer(name, state):
    # later: requests.post(webhook_url, json={ ... })
    print(f"POST event {name}={state}")

gil = InterfaceSubject("GigabitEthernet1")
gil.attach(webhook_observer)
gil.shutdown()
```

The subject does not import Webex or ServiceNow. Observers register. That is the advantage.

`labs/01_python_basics/functions_classes_modules.py` comments `Interface` as a “minimal model” — keep that mapping in your head when you reach webhooks.

4. Exam-style understanding

You should be able to:

1. Name MVC’s three parts and one responsibility each.

2. Give MVC's advantage: separation of data, presentation, and input; easier testing and multiple UIs.
3. Name Observer's parts: subject, observers, notify on change.
4. Give Observer's advantage: loose coupling, event-driven updates, add subscribers without changing the core.
5. Connect Observer → webhooks (HTTP callback on event) and MVC → web/API apps.

Original practice scenario. A script every 30 seconds GETs `/alarms` (polling). An Observer design would register a webhook so the controller POSTs when an alarm opens. Advantage: faster notice, fewer GETs. New constraint: you must expose a URL and validate the sender (Domain 2).

5. Hands-on exercise

1. Read `Interface` in `labs/01_python_basics/functions_classes_modules.py`. Label Model vs what a View/Controller would be.
2. Add a list of callbacks to `Interface.shutdown` that print a line (mini-Observer). Do not touch `summary`.
3. Draw the mermaid MVC and Observer diagrams from memory.
4. When you study 2.2, return here and rewrite `webhook_observer` using `requests.post` to <https://httpbin.org/post>.

1.7 Explain the advantages of version control

1. What Cisco expects me to know

Explain the advantages of version control as a practice. Git is the tool (1.8). This objective is **why** you use it: history, collaboration, branching, rollback, audit, review. "We have a shared USB stick called FINAL_v3" is the anti-pattern.

2. Detailed explanation

Version control records snapshots of files over time, with authorship, messages, and the ability to branch parallel lines of work. Git is a **distributed** version control system: every clone has the full history (1.8.a).

Advantages

Advantage	What it means in automation
History	Who changed the ACL template and when
Rollback	Last week's playbook still exists; <code>git revert</code> / checkout an old commit
Branching	Try RESTCONF auth changes without breaking <code>main</code>
Collaboration	Two engineers, one repo, merge (1.8.f) instead of emailing zips
Review	Pull requests: a second person sees a <code>diff</code> (1.8.g) before production
Audit / compliance	Evidence of what was deployed; tags for releases
Backup	The remote (GitHub) is not the only copy, but it is a durable one
Bisect / debug	Binary search history to find the commit that broke parsing
Automation of automation	CI runs tests on each commit (TDD artifacts stay green)

What version control is not. It is not a backup substitute if nobody pushes. It is not a secrets manager — **do not commit API keys** (see 2.7). It is not a substitute for testing.

Centralized vs distributed (context). Older systems (central SVN) required the server for many operations. Git lets you commit on an airplane, then push. The exam cares that you understand **local vs remote** (1.8), which exists because Git is distributed.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

graph LR
    WT[Working tree] --> ST[Staging index]
    ST --> LR[Local repository]
    LR -->|push| RM[Remote repository]
    RM -->|pull / fetch| LR

```

Advantages accrue only if the team actually **commits small, meaningful snapshots** and uses branches for isolation. A single commit named “stuff” with 40 unrelated files throws away history’s value.

3. Syntax and examples

Version control’s “syntax” is the **commit**, the **message**, and the **log**.

```

git log --oneline
# a1b2c3d Add RESTCONF hostname helper
# 9f8e7d6 Fix YAML indent in inventory

```

Each hash is a complete snapshot pointer. Advantage: you can describe `git show a1b2c3d` in a change ticket.

`.gitignore` is part of using version control well:

```
.venv/  
labs/.env  
__pycache__/
```

Advantage: history stays about **source**, not secrets and byte-compiled junk.

`labs/04_git/workflow.yaml` lists the operations you will **utilize** in 1.8. The advantage list in 1.7 is why that workflow exists.

Bad vs good commit (advantage of discipline)

Message	History advantage
update	Near zero
Fix prefix_to_hosts /31 usable-host count	Future you can find the bugfix

4. Exam-style understanding

You should be able to:

1. List several advantages: history, collaboration, branch/merge, rollback, review, audit.
2. Contrast version control with “copy the folder to dated names.”
3. State that Git being distributed means commits happen **locally** even before push.
4. State that secrets in Git are a disadvantage of **misuse**, not of version control itself.
5. Connect advantages to operations: rollback ↔ commit history; collaboration ↔ push/pull; review ↔ diff.

Original practice scenario. Production Ansible used `enabled: true` for a shutdown window because an intern edited live on the server. With version control, the change would have been a branch, a diff, and a review; rollback would be a revert, not “does anyone have last Tuesday’s file?”

5. Hands-on exercise

Free: Git and a **private** GitHub repo (<https://github.com/>). Official Git docs: <https://git-scm.com/docs>.

1. Read `labs/04_git/workflow.yaml` end to end.
 2. Initialize or clone a throwaway private repo (do **not** publish `labs/.env`).
 3. Make three tiny commits with messages that a stranger could understand. Run `git log`.
 4. Write five advantages in your own words without looking at the table above.
 5. Add a `.gitignore` that excludes `.env` and `.venv`. Confirm `git status` does not list them.
-

1.8 Utilize common version control operations with Git

1. What Cisco expects me to know

Utilize means **perform** the operations, not only define them. The sub-objectives are clone, add/remove, commit, push/pull, branch, merge and handling conflicts, and diff. This parent section is the mental model those commands sit on. Official command reference: <https://git-scm.com/docs>.

2. Detailed explanation

Git has **four places** people confuse:

Place	What it is	Typical command that touches it
Working tree	Files you edit	editing in VS Code
Staging area (index)	The next commit's proposed snapshot	<code>git add</code> , <code>git rm</code>
Local repository	Commits on your machine (<code>.git</code>)	<code>git commit</code>
Remote repository	Copy on GitHub/GitLab	<code>git push</code> , <code>git pull</code> , <code>git clone</code>

Clone copies a remote (including history) to a new local repo. **Add** copies working-tree changes into the index. **Commit** records the index as a snapshot with a message. **Push** sends local commits to a remote. **Pull** fetches remote commits and integrates them (merge or rebase; this course uses merge). **Branch** is a movable pointer to a commit — cheap parallel lines of development. **Merge** combines histories; **conflicts** happen when both sides changed the same region. **Diff** shows line-level changes.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph local ["Your machine"]
        WT[Working tree]
        IX[Staging index]
        REPO[(Local commits)]
        WT -->|git add| IX
        IX -->|git commit| REPO
        REPO -->|git checkout / switch| WT
    end
    REMOTE[(origin on GitHub)]
    REMOTE -->|git clone / git pull| REPO
    REPO -->|git push| REMOTE
```

`git status` is the orientation command: it tells you which of those places has uncommitted or unpushed work.

3. Syntax and examples

Identity (once per workstation; you already have this in `CCNAUTO_LAB_SETUP.md`):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Orientation:

```
git status
git log --oneline --decorate --graph --all
```

The detailed syntax for each operation is in 1.8.a–1.8.g. A full loop looks like:

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git switch -c feature/hostname
# edit files
git add hostname.py
git commit -m "Parameterize RESTCONF base URL"
git push -u origin feature/hostname
# later, on main:
git pull
git merge feature/hostname
git diff HEAD~1
```

4. Exam-style understanding

You should be able to:

1. Place `add`, `commit`, `push` on the working tree → index → local → remote path.
2. Know `clone` creates the local repo from a remote; it is not the same as `commit`.
3. Know `pull` is not automatic just because files changed on GitHub — you must pull (or fetch).
4. Know a conflict is not a Git crash; it is a merge that needs a human.
5. Read a unified diff (1.8.g, lab file `labs/04_git/example.diff`).

Practice item: “I edited `notes.txt` but `git log` does not show it.” Cause: no add/commit. “I committed but GitHub does not show it.” Cause: no push.

5. Hands-on exercise

Use a **private** GitHub repository and the checklist in `labs/04_git/workflow.yaml`. Follow the clone-through-diff sequence in `CCNAAUTO_LAB_SETUP.md` section “Git lab (objectives 1.8.a–g and 5.12)”. Study `labs/04_git/example.diff` until you can explain every line. The subsections below are the graded skills; do each hands-on block, not only this parent one.

1.8.a Clone

1. What Cisco expects me to know

Utilize clone: create a local copy of a repository from a remote URL, including history and a link to `origin`.

2. Detailed explanation

`git clone <url>` is how you **start** from an existing project (Cisco samples on GitHub, your `ccnaauto-labs` repo, a teammate's library). Clone:

1. Creates a new directory (by default named after the repo).
2. Copies all reachable commits (full history in a normal clone).
3. Checks out the default branch (`main` or `master`) into the working tree.
4. Sets `remote.origin.url` so later `push` / `pull` know where to go.

Clone is **not** download-zip. A zip has files but no `.git` history, no branches, no remotes. Clone is **not** `git init` in a random folder — `init` creates an empty repo; clone fills it from elsewhere.

HTTPS vs SSH URLs both work. DevNet Code Exchange examples are often HTTPS:

```
https://github.com/CiscoDevNet/ ...
```

3. Syntax and examples

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git remote -v
git branch -vv
```

- First argument is the URL.
- Optional second argument is the directory name: `git clone <url> lab-copy`.
- `git remote -v` should show `origin` fetch and push URLs.
- Shallow clone `git clone --depth 1 <url>` is a real Git feature for speed; you still cloned, but history is truncated. Prefer a full clone for study.

4. Exam-style understanding

You should be able to:

1. State the purpose: copy a remote repo locally, including Git metadata.
2. Recognize that after clone you already have a local `main` and a remote `origin`.
3. Distinguish clone (new directory from remote) vs pull (update an existing clone).
4. Know that clone of a private repo requires authentication (GitHub login/token).

Original practice: “Clone the repo then immediately `git commit`.” That commit needs a change; clone itself does not create a new commit.

5. Hands-on exercise

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git status
git log --oneline -5
```

If you do not have a GitHub repo yet, create a private empty one in the browser, clone it, and proceed with 1.8.b. Do not clone secrets-filled repos.

1.8.b Add/remove

1. What Cisco expects me to know

Utilize add and remove: stage new/modified files (`git add`) and stage a file’s deletion (`git rm`). The index is the next commit.

2. Detailed explanation

Editing a file only changes the **working tree**. Git still thinks the last commit is the truth until you **stage** and **commit**.

- `git add <path>` stages a new file or a modification.
- `git add -A` or `git add .` stages all changes in the repo or current directory (use with care).
- `git rm <path>` deletes the file from the working tree **and** stages the deletion.
- Deleting in Explorer/VS Code without `git rm` leaves a “deleted” working-tree change; you still `git add` that deletion (or `git add -u`) to stage it.
- `git restore --staged <path>` (or older `git reset HEAD <path>`) **unstages** without destroying the file.

Remove vs ignore. `git rm` is for files that **were** tracked. `.gitignore` prevents **untracked** files from being added. If a secret was already committed, gitignore will not un-publish it; you must stop using the secret.

3. Syntax and examples

```
echo "lab" > notes.txt
git add notes.txt
git status
# Changes to be committed: new file notes.txt

git rm notes.txt
git status
# Changes to be committed: deleted notes.txt
```

Partial add:

```
git add labs/02_data_formats/parse_formats.py
# does not stage unrelated dirty files
```

Windows PowerShell: `git add` paths can use `/` or `\`; Git typically stores paths with `/`.

4. Exam-style understanding

You should be able to:

1. Explain that `add` copies content into the **staging area**.
2. Choose `git rm` when the intent is “this file should disappear in the next commit.”
3. Read `git status` sections: untracked vs changes not staged vs changes to be committed.
4. Avoid thinking `git add` sends files to GitHub (that is push).

Original practice: `git add .` from the repo root after creating `labs/.env` — disaster if `.gitignore` is missing. Add/remove skill includes **what not to add**.

5. Hands-on exercise

In your clone:

1. Create `notes.txt`, `git add notes.txt`, `git status`.
2. Edit `notes.txt`, `git status` (modified, not staged), `git add notes.txt` again.
3. `git rm notes.txt` **after** it has been committed at least once (commit first in 1.8.c if needed), or practice `git rm --cached` on a dummy file to untrack without deleting locally — then restore cleanly.
4. Confirm `labs/.env` would be ignored if you copy the course `.gitignore`.

1.8.c Commit

1. What Cisco expects me to know

Utilize commit: record a staged snapshot in the **local** repository with a message. Commit does not update GitHub by itself.

2. Detailed explanation

A commit object stores: tree (file snapshot), parent commit(s), author, timestamp, message. The current branch pointer moves to the new commit.

Rules that preserve the advantages from 1.7:

- Stage only related changes.

- Write a message that says **why**, not “files.”
- Commits are local until push. You can have many local commits.

`git commit -m "message"` uses the message flag. `git commit` without `-m` opens an editor. `git commit -a` stages tracked modifications and commits (still will not add **new** untracked files). Prefer explicit `git add` while learning.

Amending (`git commit --amend`) rewrites the last commit. Do not amend commits already pushed to a shared branch unless you know the consequences; this study guide does not require amend.

3. Syntax and examples

```
git add notes.txt
git commit -m "Add lab notes"
git log -1
```

Good messages for this course:

```
Add prefix_to_hosts unit tests
Parameterize RESTCONF hostname URL
```

Empty commit attempt: if nothing is staged, Git refuses (`nothing to commit`). That is a feature.

4. Exam-style understanding

You should be able to:

1. State that commit records the **index** to the **local** repo.
2. Know a message is required (policy) and `-m` supplies it.
3. Sequence: edit → add → commit → (later) push.
4. Recognize `nothing to commit, working tree clean`.

Original practice: “I committed, why is the GitHub file unchanged?” Because commit is local.

5. Hands-on exercise

```
echo "lab" > notes.txt
git add notes.txt
git commit -m "Add lab notes"
git status
git log --oneline -3
```

Make a second commit that only changes one line. Confirm two hashes in `git log`.

1.8.d Push / pull

1. What Cisco expects me to know

Utilize push and pull: publish local commits to a remote; bring remote commits into your local branch.

2. Detailed explanation

Push (`git push`) uploads your commits to `origin` (usually). The first push of a branch often needs `-u origin <branch>` to set upstream so later `git push` has a default.

Pull (`git pull`) is `git fetch` (download remote commits) plus an integrate step. Default integrate is **merge** if you set `pull.rebase false` as in the lab setup. Pull is how you sync before starting work and how you receive a teammate's commits.

If the remote has commits you lack, push is rejected until you pull (or otherwise reconcile). That protection prevents silently overwriting published history.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant L as Local main
    participant O as origin/main
    L->>O: git push
    O->>L: git pull
```

Fetch vs pull. `git fetch` updates remote-tracking branches (`origin/main`) without merging. `git pull` fetches and merges into your current branch. For the exam, pull = get remote changes into my branch.

3. Syntax and examples

```
git push -u origin main
git push
git pull
```

New branch:

```
git switch -c feature/hostname
git push -u origin feature/hostname
```

If push fails with “non-fast-forward”:

```
git pull
# resolve conflicts if any (1.8.f)
git push
```

Authentication: GitHub HTTPS may require a personal access token instead of a password. SSH uses keys. The exam tests the **operations**, not GitHub's current login UI.

4. Exam-style understanding

You should be able to:

1. Push = local commits → remote. Pull = remote commits → local branch.
2. Clone already sets `origin` ; push/pull use it.
3. Explain a rejected push: remote moved, you must pull first.
4. Know `-u` sets the upstream tracking branch.

Original practice: Two clones of the same repo. Commit in A, push. Clone B still has old files until `git pull` in B.

5. Hands-on exercise

```
git push -u origin main
# on GitHub, confirm the commit
# from a second folder:
git clone https://github.com/<you>/ccnaauto-labs.git ccnaauto-labs-b
cd ccnaauto-labs-b
# make a commit on GitHub in the browser or in clone A, then:
git pull
```

Practice both directions once.

1.8.e Branch

1. What Cisco expects me to know

Utilize branch: create, list, and switch branches so work is isolated from `main`.

2. Detailed explanation

A **branch** is a pointer to a commit. Creating a branch is cheap (a 41-byte ref, not a full copy of files). When you commit, the **current** branch pointer moves forward.

Why branch: keep `main` releasable; develop a RESTCONF change without mixing it into an unrelated YAML fix; enable pull-request review.

Switch vs checkout. Modern Git: `git switch <name>` to change branches, `git switch -c <name>` to create and switch. Older: `git checkout -b <name>`. Both appear in the wild.

You cannot switch with uncommitted conflicting changes (Git will stop you). Commit or stash first. Stash is useful but not a named 1.8 sub-objective; prefer commit on the feature branch.

3. Syntax and examples

```
git branch          # list local; * marks current
git switch -c feature/hostname
git switch main
git branch -d feature/hostname  # delete after merge; -D forces
```

`git branch -vv` shows tracking remotes.

Naming: `feature/hostname`, `fix/yaml-indent`. Slashes are allowed in branch names.

4. Exam-style understanding

You should be able to:

1. Define a branch as a movable pointer to a commit.
2. Create and switch with `git switch -c` or `git checkout -b`.
3. Explain the advantage: isolated history until merge.
4. Read `git status` "On branch feature/hostname".

Original practice: Commits while on `feature/hostname` do not move `main` until merge.

5. Hands-on exercise

```
git switch -c feature/hostname
# edit a file, add, commit
git switch main
git log --oneline --graph --all
```

Confirm `main` does not include the new commit until 1.8.f.

1.8.f Merge and handling conflicts

1. What Cisco expects me to know

Utilize merge and **handle conflicts**: combine a branch into another; when both sides edit the same lines, resolve markers, then commit.

2. Detailed explanation

Merge creates a new commit (except fast-forward) that has **two parents** and a tree combining both histories.

Fast-forward: `main` has not moved since you branched; Git just slides `main` to the feature tip. No merge commit.

True merge: `main` gained commits while you worked; Git weaves them.

Conflict: Git cannot automatically combine a hunk. It writes **conflict markers** into the file:

```
<<<<<< HEAD
url = "https://router/restconf/data/Cisco-IOS-XE-native:native/hostname"
=====
HOST = "https://router"
url = f"{HOST}/restconf/data/Cisco-IOS-XE-native:native/hostname"
>>>>>> feature/hostname
```

HEAD is “the branch you merged **into**” (often **main**). The other label is the incoming branch. You **edit** to the correct final content, **remove** the markers, **git add** the file, and **git commit** to complete the merge.

Abort: **git merge --abort** if you want to undo an in-progress merge.

Do not leave markers in production files. That is a failed resolution.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
  M[main] --> C[merge commit]
  F[feature/hostname] --> C
```

3. Syntax and examples

Clean merge:

```
git switch main
git merge feature/hostname
```

Force a conflict (lab):

```
git switch main
echo "alpha" > conflict.txt
git add conflict.txt && git commit -m "Main says alpha"
git switch -c feature/conflict
echo "bravo" > conflict.txt
git add conflict.txt && git commit -m "Feature says bravo"
git switch main
echo "charlie" > conflict.txt
git add conflict.txt && git commit -m "Main now charlie"
git merge feature/conflict
# conflict.txt contains markers
```

Resolve by choosing the correct content (or combining), then:

```
git add conflict.txt
git commit -m "Merge feature/conflict; keep charlie"
```

git status during a conflict says “You have unmerged paths.”

4. Exam-style understanding

You should be able to:

1. State merge integrates two branches.
2. Recognize <<<<<<, =====, >>>>>>.
3. Describe the fix: edit, add, commit.
4. Know fast-forward vs merge commit at a high level.
5. Know that both sides changing **different** files usually merges cleanly.

Original practice: Same line, two branches, merge, conflict. Different lines in the same file often auto-merge.

5. Hands-on exercise

Perform the conflict sequence above in `ccnaauto-labs`. Then merge a **non-conflicting** `feature/hostname` as in the lab setup. Read `git log --graph --oneline` after both.

1.8.g diff

1. What Cisco expects me to know

Utilize diff: view unstaged, staged, and commit-to-commit changes. Read **unified diff** (headers, hunks, `+ / -`). This also supports exam topic 5.12-style interpretation later.

2. Detailed explanation

A **diff** is a line-oriented comparison. Git's default unified format shows enough context to apply a patch.

- `git diff` — working tree vs index (unstaged).
- `git diff --staged` (`--cached`) — index vs last commit (what the next commit will contain).
- `git diff HEAD~1` — last commit vs current HEAD.
- `git diff main ... feature/hostname` — changes on the feature since it diverged.

Reading the format (see `labs/04_git/example.diff`):

Piece	Meaning
--- a/hostname.py	Old file
+++ b/hostname.py	New file
@@ -1,6 +1,7 @@	Hunk: old starts line 1, 6 lines; new starts line 1, 7 lines
leading space	Unchanged context line
-	Line removed from old
+	Line added in new

Diff is how code review works. Push/pull move commits; diff **explains** them.

3. Syntax and examples

labs/04_git/example.diff :

```
--- a/hostname.py
+++ b/hostname.py
@@ -1,6 +1,7 @@
 import requests

-url = "https://router/restconf/data/Cisco-IOS-XE-native:native/hostname"
+HOST = "https://router"
+url = f"{HOST}/restconf/data/Cisco-IOS-XE-native:native/hostname"
 response = requests.get(url, auth=("admin", "admin"), verify=False)
 print(response.json())
```

Interpretation: one long URL line was replaced by a `HOST` constant and an f-string. `requests.get` and `print` are unchanged (space prefix). Line count went from 6 to 7 in that hunk (`+1,7`).

Commands:

```
git diff
git diff --stat
git diff HEAD~1
```

`--stat` summarizes files and insert/delete counts without every line.

4. Exam-style understanding

You should be able to:

1. Point to a `-` line and say it was removed.
2. Point to a `+` line and say it was added.
3. Explain `@@` hunk headers at a basic level.
4. Choose `git diff` vs `git diff --staged` based on whether add already happened.

5. Use diff output to describe a change in plain language (as in the hostname URL refactor).

Original practice: After `git add`, `git diff` is empty but `git diff --staged` is not.

5. Hands-on exercise

1. Read `labs/04_git/example.diff` line by line and write a one-sentence summary.
 2. Change a tracked file, run `git diff`, then `git add`, then `git diff --staged`.
 3. After a commit, `git diff HEAD~1`.
 4. Open <https://git-scm.com/docs/git-diff> and skim the unified format section.
-

2.0 Understanding and Using APIs — 20%

Understanding and Using APIs is **one of the two heaviest domains** on CCNA Automation (200-901 CCNAAUTO v1.1). At **20%** of the exam, it outweighs Software Development and Design (15%). If Domain 1 taught you JSON and functions, Domain 2 is where those skills become **HTTP conversations with a platform**.

An API (Application Programming Interface) is a contract: you send a request that follows documented rules; the server returns a structured response. For this exam the dominant style is **REST over HTTP** with **JSON** bodies. You will **construct** requests from documentation, **interpret** status codes, headers, and bodies, **troubleshoot** failures, **utilize** authentication, **compare** API styles, and **construct** Python using the `requests` library.

Study this domain deeper than a glossary. Practice against free endpoints: <https://httpbin.org> (echoes your request), <https://developer.cisco.com/> (real product docs and sandboxes), <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status> (status codes), and <https://requests.readthedocs.io/> (Python client). Lab files: `labs/03_rest_api/rest_client.py` and `labs/03_rest_api/troubleshoot_http.py`.

The exam is 120 minutes. API items are often “here is a doc snippet and an HTTP exchange — what is wrong?” Speed comes from pattern recognition: 401 vs 403, `json=` vs a string body, missing `Accept` header, rate-limit `Retry-After`.

2.1 Construct a REST API request to accomplish a task given API documentation

1. What Cisco expects me to know

Construct means you are given **API documentation** (method, path, headers, query parameters, body schema, auth) and you must **build the request** that performs the task: correct verb, URI, headers, and body. You are not inventing an unofficial URL.

REST here means: **resources** identified by **URIs**, manipulated with **HTTP methods**, typically **stateless**, often **JSON**.

2. Detailed explanation

A REST request has:

1. **Method** — GET (read), POST (create or controller-style action), PUT (replace), PATCH (partial update), DELETE (remove).
2. **URI** — scheme, host, path, query string. Path identifies the resource (`/orgs/{id}/devices`). Query (`?perPage=10`) is not the resource identity; it filters, paginates, or selects.
3. **Headers** — `Accept`, `Content-Type`, `Authorization`, product-specific keys.
4. **Body** — JSON object for POST/PUT/PATCH; GET/DELETE usually have no body.

Stateless: each request carries its own auth and parameters. The server does not need your previous GET to understand this POST (cookies/sessions exist in the web, but REST APIs you study behave as token-per-request).

Documentation-driven construction. Docs typically show:

```
GET https://api.meraki.com/api/v1/organizations/{organizationId}/devices
Header: X-Cisco-Meraki-API-Key: <key>
Header: Accept: application/json
```

Your job: substitute `{organizationId}`, add the key, use GET, do not invent `/devices/list`. If the doc says POST with a JSON body `{"name": "HQ"}`, sending query parameters instead is wrong.

RESTCONF is REST applied to YANG data. Docs specify a path like `/restconf/data/Cisco-IOS-XE-native:native/hostname` and `Accept: application/yang-data+json`. Construction still means: copy the documented path and headers, then fill host and auth.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    D[API documentation] --> R[Method + URI]
    D --> H[Headers]
    D --> B[Body schema]
    R --> HTTP[HTTP request]
    H --> HTTP
    B --> HTTP
    HTTP --> S[Server]
```

Idempotency (construction implication). GET, PUT, DELETE are typically idempotent: repeating them does not create extra resources. POST often creates a new object each time. If the task is “ensure hostname is edge-01”, PUT/PATCH (or a documented “update” POST) is the right construction; firing POST create twice may duplicate.

3. Syntax and examples

Anatomy

```
GET /get?device=csr1kv&vrf=mgmt HTTP/1.1
Host: httpbin.org
Accept: application/json
```

- Method GET
- Path `/get`
- Query `device=csr1kv` and `vrf=mgmt`
- Header `Accept` asks for JSON

From docs to `requests (lab get_with_query)`

Docs: “GET `/get` accepts arbitrary query parameters and returns them in `args`.”

```
import requests
r = requests.get(
    "https://httpbin.org/get",
    params={"device": "csr1kv", "vrf": "mgmt"},
    timeout=15,
)
```

`params=` builds the query string and encodes spaces. Do not concatenate `?device=` by hand unless you know encoding rules.

POST JSON (lab `post_json`)

Docs: “POST `/post` accepts a JSON body.”

```
r = requests.post(
    "https://httpbin.org/post",
    json={"hostname": "edge-01", "role": "wan"},
    timeout=15,
)
```

`json=` sets `Content-Type: application/json` and serializes the dict. Using `data={"hostname": ...}` without the header is a construction error (often 415 later).

Cisco-shaped example (pattern only; host from sandbox docs)

Documentation says:

- Method: GET
- URL: `https://{host}/restconf/data/Cisco-IOS-XE-native:native/hostname`
- Headers: `Accept: application/yang-data+json`
- Auth: HTTP Basic

Construction:

```
url = f"https://{host}/restconf/data/Cisco-IOS-XE-native:native/hostname"
headers = {"Accept": "application/yang-data+json"}
r = requests.get(url, headers=headers, auth=(user, password), verify=False, timeout=15)
```

`verify=False` is a **lab** concession for self-signed certs, not a production default. Docs that require TLS still want HTTPS.

Choosing the method from the task wording

Task in the prompt	Typical method
Retrieve the list of devices	GET
Create a Webex room	POST
Replace the entire hostname resource	PUT
Change one field on a device	PATCH
Remove a webhook subscription	DELETE

Always override this table if the vendor doc says otherwise (some “create” operations are PUT to a known URL).

4. Exam-style understanding

You should be able to:

1. Read a doc table and write the method + path + required headers.
2. Substitute path parameters `{id}` from the scenario.
3. Put filters in the **query string** when docs say query params.
4. Put create/update fields in the **JSON body** when docs show a schema.
5. Avoid mixing: a GET with a JSON body “because JSON is REST” is usually wrong.

Original practice scenario. Docs:

```
POST /networks/{networkId}/vlans
Body JSON: { "id": 30, "name": "GUEST" }
Header: X-Cisco-Meraki-API-Key
```

Task: create VLAN 30 named GUEST in network `N_123`. Constructed request: POST to `/networks/N_123/vlans`, header with API key, body `{"id": 30, "name": "GUEST"}`. Wrong constructions: GET; `/vlans/30` if docs say POST to collection; form-encoded body.

5. Hands-on exercise

Free: <https://httpbin.org> and Postman or `labs/03_rest_api/rest_client.py`.

```
cd labs\03_rest_api
python rest_client.py
```

Then construct, without looking at the file:

1. GET `https://httpbin.org/get` with query `device=edge-01`.
2. POST `https://httpbin.org/post` with JSON `{"hostname": "edge-01"}`.
3. Open <https://developer.cisco.com/>, pick **Meraki API** or **Catalyst Center** getting started, and write (on paper) one GET exactly as the docs specify — method, path, headers — even if you do not send it yet.

Postman: create request, set method, URL, Params tab vs Body raw JSON, Headers. Save the request. You constructed it.

2.2 Describe common usage patterns related to webhooks

1. What Cisco expects me to know

Describe webhooks conceptually: they are **HTTP callbacks**. You **subscribe** a URL; the provider **POSTs** (usually) when an event happens. Patterns: subscription lifecycle, payload shape, **challenge/validation**, retries, security. Connect this to **Observer** (1.6).

2. Detailed explanation

Polling is the opposite pattern: your script GETs `/events` every 30 seconds. **Webhooks** invert it: the platform is the HTTP **client** for a moment, and **your** automation is the HTTP **server**.

Common usage patterns

1. **Subscribe**. You call the vendor's REST API (this is still 2.1) to register `targetUrl`, event types (message created, device down), and maybe a secret. Example conceptual resource: `POST /webhooks` with `{"url": "https://nso.example/hook", "events": ["alarms"]}`.
2. **Receive**. Your endpoint must be reachable (public HTTPS, or a tunnel for labs). It must answer quickly (2xx) so the provider does not mark you unhealthy.
3. **Challenge / validation (handshake)**. Many providers prove you control the URL: they send a GET or POST with a `challenge` string; you must echo it back. If you fail, the subscription is not activated. This stops people from pointing webhooks at victim servers (DDoS amplifier / unsolicited traffic).
4. **Event delivery**. Later, JSON body describes the event. You parse (1.2) and act (create a ticket, send Webex, shut an interface — carefully).
5. **Retry**. If your endpoint returns 5xx or times out, platforms typically **retry** with backoff. Your handler should be **idempotent** where possible: processing the same event twice should not create two change tickets.
6. **Unsubscribe / rotate**. Delete the webhook resource when the receiver dies, or you will generate failures and lockouts. Rotate secrets.
7. **Security patterns**. HMAC signatures in a header, shared secrets, allow-list source IPs, TLS. You authenticate **the sender** (is this really Meraki/Webex?) not only your users.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant You as You automation
    participant API as Platform REST API
    participant Hook as Your webhook URL
    You->>API: POST /webhooks subscribe
    API->>Hook: Challenge / validation
    Hook->>API: Echo challenge 200
    Note over API: Event occurs
    API->>Hook: POST event JSON
    Hook->>API: 200 OK
    Hook->>Hook: Act on event
```

Where Cisco uses this idea. Webex webhooks for messages/memberships; dashboard platforms notifying on network events; ChatOps bots. You do not need every product's field list; you need the **pattern**.

Webhook vs websocket vs polling

Pattern	Who initiates after setup	Typical exam association
Polling	You, on a timer	Simple scripts, rate-limit heavy
Webhook	Platform, per event	Observer, callbacks
WebSocket	Persistent connection	Streaming (know it exists; webhook is the named topic)

3. Syntax and examples

There is no single webhook standard. Teaching syntax is **subscribe with REST, receive POST JSON**.

Subscribe (you are the REST client — 2.1 / 2.9)

```
import requests

r = requests.post(
    "https://httpbin.org/post",
    json={
        "name": "ccnaauto-lab",
        "targetUrl": "https://example.com/hooks/alerts",
        "resource": "alarms",
        "event": "created",
    },
    headers={"Authorization": "Bearer TOKEN"},
    timeout=15,
)
```

httpbin will echo the JSON; a real API would return a webhook `id` for later DELETE.

Challenge response (you are the HTTP server)

Pseudo-handler: if the body has `"challenge"`, return that string as the response body with `200` and `Content-Type: text/plain` (or whatever the vendor documents). Only after that will event POSTs arrive.

Event POST you might receive

```
POST /hooks/alerts HTTP/1.1
Host: example.com
Content-Type: application/json
X-Signature: sha256= ...

{"event": "device_down", "serial": "Q2XX-1111", "ts": "2026-08-16T15:00:00Z"}
```

Your code: verify signature, `json.loads` body, branch on `event`, return `204` or `200` quickly, do slow work asynchronously if needed.

Retry pattern. First POST → your server 503. Provider waits, POSTs again with the same event id. If you created a ticket on the first partial attempt, check event id before creating another.

4. Exam-style understanding

You should be able to:

1. Define webhook as an HTTP callback on an event (push), not a poll.
2. Describe subscribe → (optional challenge) → event POST → 2xx → retry on failure.
3. Relate to Observer: platform is subject; your URL is observer.
4. Explain why challenge exists: prove control of the URL.
5. Explain why 2xx matters: suppress retries / mark healthy.

Original practice scenario. A Webex bot never fires. Checklist: subscription created? Public HTTPS URL? Challenge handler implemented? Returning 401 on events because you reused user-auth instead of signature check? Returning 500 and the platform gave up after N retries?

5. Hands-on exercise

Free tools: httpbin + a local receiver.

1. POST a fake subscription body to <https://httpbin.org/post> using `rest_client.py` style `json=`.
2. Use Python stdlib to listen locally:

```

from http.server import BaseHTTPRequestHandler, HTTPServer
import json

class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", "0"))
        body = json.loads(self.rfile.read(length) or b"{}")
        print("event", body)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok")

HTTPServer(("127.0.0.1", 8765), Hook).serve_forever()

```

3. In another terminal: `python -c "import requests; requests.post('http://127.0.0.1:8765', json={'event': 'up'})"`.
4. Write the sequence diagram in your notes. Optional: ngrok or Cloudflare Tunnel if you later test a real Webex webhook (free tier; not required today).

2.3 Describe the constraints when consuming APIs

1. What Cisco expects me to know

Describe limits that exist **when you are the client**: rate limits, pagination, payload size, versioning, TLS, timeouts, idempotency, required headers, auth scopes. Consuming an API is not “unlimited GET in a `while True`.”

2. Detailed explanation

Vendors protect shared control planes. Your script must live inside those constraints or you will see **429**, truncated lists, `413`, TLS errors, or silent misses (page 2 never fetched).

Rate limits. A quota such as “N requests per second per token.” Exceeding it yields **429 Too Many Requests**, often with `Retry-After`. Constraint: backoff, cache GETs, use webhooks instead of polling.

Pagination. Large collections are not one JSON array of 50,000 devices. APIs return a page plus a `next` link, cursor, or `page / offset` query params. Constraint: loop until there is no next page, or you only automated the first 100 devices.

Payload size. POST bodies and responses have limits. Sending a 20 MB config blob may fail. Responses may truncate. Constraint: chunk, compress if documented, or use a bulk API.

Versioning. Paths include `/api/v1/` or headers `Accept: application/vnd.example.v2+json`. Constraint: pin the version in your client; do not assume v1 lives forever. Breaking changes happen on a new version.

TLS / certificates. Production APIs require HTTPS. Self-signed sandbox certs fail verification (`verify=True` default in `requests`). Constraint: install the CA, or lab-only `verify=False`. Corporate proxies intercept TLS.

Timeouts and retries. Networks fail. `requests` without `timeout=` can hang the exam lab **and** a production controller. Constraint: set timeouts; retry **idempotent** GETs; be careful retrying POST.

Idempotency. Constraint: design writes so a retried PUT is safe. Some APIs support `Idempotency-Key` headers.

Auth and scope. A valid token might still be forbidden for DELETE (403). Constraint: least privilege; different keys for read-only inventory vs change.

Schema and content types. Constraint: send `Content-Type: application/json` (or `yang-data+json`). Wrong type → **415**. Extra unknown fields may be ignored or **400**.

Filtering vs client-side loops. Docs may allow `?name=edge-01`. Constraint: use server-side filters to stay under rate limits.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    C[Your client] --> L{Constraints}
    L --> R[Rate limit / 429]
    L --> P[Pagination]
    L --> S[Payload size]
    L --> V[Versioned path]
    L --> T[TLS]
    L --> I[Idempotent retries]
```

3. Syntax and examples

Timeout (always in this course)

```
r = requests.get(url, timeout=15)
# or (connect timeout, read timeout)
r = requests.get(url, timeout=(5, 30))
```

Pagination loop (pattern)

```
url = "https://api.example.com/v1/devices"
while url:
    r = requests.get(url, headers=headers, timeout=15)
    r.raise_for_status()
    data = r.json()
    for device in data["items"]:
        print(device["name"])
    url = data.get("next") # None when done
```

If you only `GET` once, you silently skipped remaining items — a **constraint bug**, not an HTTP error.

Rate limit handling

```
import time

r = requests.get(url, headers=headers, timeout=15)
if r.status_code == 429:
    wait = int(r.headers.get("Retry-After", "5"))
    time.sleep(wait)
```

Version in the URI

```
https://api.meraki.com/api/v1/organizations
```

v1 is a constraint: your code is coupled to that contract.

TLS

```
requests.get(url, timeout=15) # verify=True
requests.get(url, timeout=15, verify=False) # lab only
```

4. Exam-style understanding

You should be able to:

1. Name several constraints: rate limits, pagination, payload size, versioning, TLS, timeouts, idempotency.
2. Map 429 → rate limit; incomplete inventory → missed pagination; 415 → content type.
3. Explain why a tight poll loop is a bad consumer.
4. Explain why `timeout` is part of consuming APIs correctly.
5. Distinguish “API is down” (5xx) from “you ignored the rules” (4xx / missing pages).

Original practice scenario. Script lists 100 access points; the site has 350. Status 200. Root cause: `perPage=100` and no follow of `next`. Not a Cisco bug — a consumer constraint miss.

5. Hands-on exercise

1. Call `GET https://httpbin.org/delay/3` with `timeout=1` and with `timeout=15`. Observe timeout vs success.
 2. Call `GET https://httpbin.org/status/429` and feed it through `labs/03_rest_api/troubleshoot_http.py` logic.
 3. Read any Cisco API “rate limit” or “pagination” paragraph on <https://developer.cisco.com/> (Meraki and Catalyst Center both document this). Copy the constraint into your notes in your own words.
 4. Write a three-line checklist you will apply to every new API: version, auth header, page loop, timeout, backoff.
-

2.4 Explain common HTTP response codes associated with REST APIs

1. What Cisco expects me to know

Explain the codes that show up with REST APIs — what they **mean**, not only the number. Focus on: **200, 201, 204, 400, 401, 403, 404, 409, 415, 429, 500, 503**. Official overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

2. Detailed explanation

HTTP status codes are three-digit integers in the **status line**. Classes:

Range	Class	Client takeaway
1xx	Informational	Rare in REST JSON APIs
2xx	Success	Request understood and accepted
3xx	Redirect	Follow <code>Location</code> or fix the client
4xx	Client error	Your request/auth/path/body
5xx	Server error	Platform or upstream; retry may help

Success codes you must separate

Code	Name	Typical REST meaning
200	OK	GET succeeded; PUT/PATCH/POST sometimes return 200 with a body
201	Created	POST created a resource; often <code>Location</code> header with the new URI
204	No Content	Success, empty body (DELETE or PUT with nothing to say)

If you `r.json()` on a **204**, parsing fails — the code was still success.

Client errors you must separate

Code	Name	Typical REST meaning
400	Bad Request	Malformed JSON, missing field, invalid VLAN ID
401	Unauthorized	Authentication failed: no/bad credentials
403	Forbidden	Authenticated, but not allowed (RBAC, license, scope)
404	Not Found	Bad path, wrong id, resource deleted, or hidden (some APIs 404 instead of 403)
409	Conflict	State clash: duplicate name, VLAN exists, edit conflict
415	Unsupported Media Type	Content-Type wrong (sent XML or form, expected JSON)
429	Too Many Requests	Rate limit (2.3)

401 vs 403 is the highest-value distinction. 401: the server does not accept who you claim to be (or you claimed nobody). 403: it knows who you are and still says no.

Server errors

Code	Name	Typical REST meaning
500	Internal Server Error	Unhandled failure on the server
503	Service Unavailable	Overloaded, maintenance, retry later

5xx does **not** prove your JSON is wrong. Check 4xx first when the body schema is suspicious, but do not “fix” a payload in response to 503.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TD
    S[Status code] --> A{Class}
    A -->|2xx| OK[Parse body unless 204]
    A -->|401| AUTH[Fix credentials / token / key]
    A -->|403| RBAC[Fix permissions]
    A -->|404| PATH[Fix URI / id]
    A -->|400 or 415| BODY[Fix JSON / Content-Type]
    A -->|409| STATE[Resource already exists or clash]
    A -->|429| WAIT[Backoff Retry-After]
    A -->|5xx| SRV[Retry later; check status page]

```

3. Syntax and examples

Status line:

```

HTTP/1.1 201 Created
Location: /networks/N_123/vlans/30
Content-Type: application/json

```

httpbin can mint codes (lab `status_codes`):

```
r = requests.get("https://httpbin.org/status/404", timeout=15)
print(r.status_code, r.reason) # 404 NOT FOUND
```

Meaning attached to a constructed request

You sent	Likely code if wrong
GET with typo in path	404
POST valid JSON, missing auth header	401
POST with intern's read-only key	403
POST <code>{id: 30}</code> without quotes around <code>id</code> — invalid JSON	400
POST XML to a JSON-only API	415
POST same VLAN twice	409 (if vendor uses conflict)
Loop GET 1000/min	429
Sandbox down	500 or 503

201 vs 200 on create. Some APIs return 200 for POST. Docs win. The **concept** of 201 is “created.”

4. Exam-style understanding

You should be able to:

1. Explain each listed code in one sentence.
2. Contrast 200 / 201 / 204.
3. Contrast 401 / 403 / 404.
4. Contrast 400 / 409 / 415 / 429.
5. Contrast 500 / 503 vs 4xx (who should change what).

Original practice scenario. Response `401` with body `{"error": "invalid token"}` — do not debug the YANG path. Response `404` with a correct token — debug the URI. Response `204` after DELETE — success; do not report “empty body means failure.”

5. Hands-on exercise

```
cd labs\03_rest_api
python rest_client.py
```

Watch the “Status code lab” section print 200, 201, 400, 401, 403, 404, 429, 500. Then:


```
python troubleshoot_http.py
```

Write a one-line “what I would change” for each of 401, 404, 429, 200. Bookmark <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

2.5 Troubleshoot a problem given the HTTP response code, request and API documentation

1. What Cisco expects me to know

Troubleshoot = identify **what is wrong** and a **solution**, given three artifacts: the **request**, the **response code** (and usually headers/body), and **API documentation**. This is Domain 2’s applied skill. 2.4 is the codebook; 2.5 is the diagnosis.

2. Detailed explanation

Work a **fixed order** so you do not guess:

1. **Compare the request to the docs** — method, path (including prefix `/api/v1` or `/restconf/data`), path parameters, query vs body, required headers, auth scheme.
2. **Classify the status code** (2.4).
3. **Read the body** — vendors often return `{"message": "Unknown network"}` which beats guessing.
4. **Read headers** — `WWW-Authenticate`, `Retry-After`, `Content-Type`, `Location`.
5. **Propose one change** — the smallest fix that matches both the code and the docs.

Common mismatch patterns

Evidence	Likely fault	Solution
401, docs say API key header, request used Basic	Wrong auth mechanism	Send the documented header/token
403, 401 already ruled out	Wrong RBAC / scope / org	Use a key with permission; correct org id
404, host is correct	Path, id, or API version	Copy path from docs; verify the resource exists with GET list
400, JSON looks “fine”	Field name, type, or enum	Match schema ("enabled": true not "Enabled": "yes")
415	Content-Type	application/json or application/yang-data+json
409	Duplicate or bad state	GET first; use PATCH; pick another name
429	Too fast	Honor Retry-After ; slow the loop
500/503, request matches docs	Platform	Retry; different sandbox; do not rewrite JSON blindly
200 but empty list	Pagination or filter	Follow next ; drop bad query param
TLS error, never a status code	Certificate	Lab verify=False or trust the CA

Do not stop at the number. 404 on `GET /device` vs documented `GET /devices` is a path typo. 404 on `GET /devices/Q2XX` with a valid key may be a **wrong serial** — still a client problem.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TD
    A[Request + code + docs] --> B[Code class]
    B -->|401/403| C[AuthN vs AuthZ vs header name]
    B -->|404| D[Path / id / version vs docs]
    B -->|400/415/409| E[Body and Content-Type vs schema]
    B -->|429| F[Rate limit]
    B -->|5xx| G[Server; retry]
    B -->|2xx but wrong data| H[Pagination / parse / wrong resource]

```

3. Syntax and examples

Lab diagnostic helper: `labs/03_rest_api/troubleshoot_http.py`.

```
def diagnose(response, api_doc_expected: str) → str:
    code = response.status_code
    if code == 401:
        return "Authentication failed. Check username/password, token, or API key."
    if code == 403:
        return "Authenticated but not authorized. The credential lacks permission."
    if code == 404:
        return f"Resource not found. Confirm the URL path. Docs expected: {api_doc_expected}"
    if code == 400:
        return "Bad request. Inspect JSON body/query params against the API schema."
    if code == 415:
        return "Unsupported media type. Set Content-Type to application/json (or yang-data+json)."
    if code == 429:
        retry = response.headers.get("Retry-After", "unknown")
        return f"Rate limited. Wait Retry-After={retry} seconds, then retry."
    if 500 ≤ code ≤ 599:
        return "Server-side failure. Retry later; do not assume your payload is wrong."
    if 200 ≤ code ≤ 299:
        return "Success. Parse the body."
    return f"Unexpected status {code}."
```

This is a **starting** tree. Real troubleshooting still diffs the request against docs.

Worked original example 1

- **Docs:** GET /api/v1/organizations/{orgId}/devices header X-Cisco-Meraki-API-Key.
- **Request:** GET /api/v1/organizations/devices with that header (org id missing — path collapsed).
- **Code:** 404.
- **Wrong:** “API is down.”
- **Right:** path missing {orgId}. **Solution:** insert the organization id segment.

Worked original example 2

- **Docs:** POST JSON, Content-Type: application/json.
- **Request:** POST with body hostname=edge-01 and Content-Type: application/x-www-form-urlencoded.
- **Code:** 415 or 400.
- **Solution:** send a JSON object with json= in requests or raw JSON in Postman.

Worked original example 3

- **Docs:** Basic auth, RESTCONF, Accept: application/yang-data+json.
- **Request:** GET correct URL, Basic auth, Accept: application/json.
- **Code:** 406 Not Acceptable (if implemented) or 400/415-family. Some devices still answer. If 404, RESTCONF might be disabled — that is also a “request vs platform capability” issue.
- **Solution:** use the documented Accept value; confirm feature enablement if the path never exists.

Worked original example 4

- **Request:** matches docs.

- **Code:** 429, header `Retry-After: 8`.
- **Solution:** wait 8 seconds (and slow the client). Not a new API key.

4. Exam-style understanding

You should be able to:

1. Produce a **fault + fix** pair, not only a code definition.
2. Use docs as the source of truth for path and headers.
3. Prefer body error messages when present.
4. Avoid “500 means my VLAN is wrong.”
5. Handle the 2xx-but-wrong-result case (pagination, parsing `text` instead of JSON).

Original practice: Given `401` and a request that used `Authorization: Bearer` while docs show `X-Cisco-Meraki-API-Key`, the fix is the header, not the URL.

5. Hands-on exercise

```
cd labs\03_rest_api
python troubleshoot_http.py
```

Then, using Postman or Python, **manufacture** faults against httpbin and write the diagnosis:

Request	Expected code	Your written fix
GET <code>/status/401</code>	401	Supply valid basic/token/key
GET <code>/status/404</code>	404	Correct the documented path
POST <code>/post</code> with <code>data="not-json"</code> and JSON Content-Type	200 from httpbin (it accepts)	Switch to a strict API or inspect that httpbin is a poor 415 demo
GET <code>/status/415</code>	415	Set Content-Type / Accept per docs
GET <code>/status/429</code>	429	Backoff

For a stricter 415 demo, RESTCONF on a sandbox with the wrong `Content-Type` is ideal when you reach Domain 3/5. Until then, reason from docs + codes using the tables above.

Craft one **wrong path** GET to <https://httpbin.org/status/404> and one **wrong auth** GET to `/basic-auth/admin/s3cret` **without** `auth=`. Confirm 401. Fix with `auth=("admin", "s3cret")` as in `rest_client.py`.

2.6 Interpret the parts of an HTTP response (response code, headers, body)

1. What Cisco expects me to know

Interpret means **read provided output** and extract meaning from the three parts: **status code**, **headers**, **body**. You may be shown a raw HTTP dump or `requests` prints. You do not construct here; you read.

2. Detailed explanation

An HTTP response is:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 348
Retry-After: 0
Server: gunicorn

{"args": {"device": "csr1kv"}, "url": "https://httpbin.org/get?device=csr1kv"}
```

1. Response code (status line). HTTP/1.1 is the version. 200 is the code. OK is the reason phrase (informational; the number is authoritative). Interpretation starts here (2.4).

2. Headers. Name/value metadata. Case-insensitive names. Useful headers:

Header	What you interpret
Content-Type	How to parse the body (application/json , application/yang-data+json , text/plain)
Location	URI of a created resource (201)
Retry-After	Seconds (or HTTP date) to wait (429/503)
WWW-Authenticate	Auth scheme the server expects (Basic realm=...)
Set-Cookie	Session (less common in token REST)
Allow	Methods on this resource (after 405)
X-Request-Id / Tracking-Id	Correlation for TAC / vendor support
Content-Length	Body size; 0 with 204 is normal

Headers are **not** the JSON fields. `enabled: true` in a header would be exotic; it belongs in the body.

3. Body. Bytes after the blank line that ends headers. May be JSON, XML, HTML error page, or empty. Interpretation:

- If Content-Type is JSON, parse to dict/list (1.2, `response.json()`).

- Error bodies often contain `message`, `error`, `ietf-restconf:errors`.
- Empty body + 204 = success.
- HTML body + 200 on an API URL often means you hit a **web UI** or proxy login, not the API — interpret as “wrong URL or auth portal.”

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TB
    R[HTTP response] --> C[Status code]
    R --> H[Headers]
    R --> B[Body]
    C --> I[Success vs class of failure]
    H --> I
    B --> I

```

requests mapping

HTTP part	requests attribute
Status code	<code>response.status_code</code>
Reason	<code>response.reason</code>
Headers	<code>response.headers</code> (case-insensitive dict)
Body text	<code>response.text</code>
Body bytes	<code>response.content</code>
Parsed JSON	<code>response.json()</code>

Lab `show_response` in `rest_client.py` prints URL, status, Content-Type, then JSON.

3. Syntax and examples

Interpret this original dump

```

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="Restricted"
Content-Type: application/json

{"authenticated": false}

```

- Code: authentication failed.
- Header: server wants **Basic** auth, not an API key.
- Body: confirms not authenticated. **Action:** add `Authorization: Basic ...` (2.7).

Interpret this original dump

```
HTTP/1.1 201 Created
Location: /api/v1/rooms/abc123
Content-Type: application/json

{"id": "abc123", "title": "CCNAAUTO"}
```

- Code: resource created.
- Location: GET this path to fetch the room.
- Body: JSON with the new id. **Action:** store id for later membership APIs.

Interpret this original dump

```
HTTP/1.1 204 No Content
Content-Length: 0
```

- Success, no body. Do not call `.json()`.

Interpret requests output

```
Status: 200 OK
Content-Type: application/json
{'args': {'device': 'csr1kv', 'vrf': 'mgmt'}, ...}
```

Query params were accepted and echoed. Construction of GET+params worked.

Trap: pretty-printed Python dict vs JSON. If the exam shows `True / None`, you are looking at **Python**. If it shows `true / null`, you are looking at **JSON** (1.1).

4. Exam-style understanding

You should be able to:

1. Point to code, headers, and body in a raw response.
2. Say what `Content-Type` implies for parsing.
3. Use `Location` and `Retry-After` correctly.
4. Know `204` $\Rightarrow$ empty body is OK.
5. Distinguish `response.text` (string) from `response.json()` (structure).

Original practice: A dump with 200 and HTML `<title>Login</title>` — interpret as “not the JSON API”; check base URL and auth. A dump with 200 and JSON `[]` — success with empty collection, not necessarily 404.

5. Hands-on exercise

Run `python labs/03_rest_api/rest_client.py` and for **each** printed block label:

1. The numeric code and class (2xx/4xx).
2. One header you would use in troubleshooting.

3. Whether the body is JSON and one field you would read in a script.

In Postman, send `GET https://httpbin.org/get` and click **Status**, **Headers**, **Body** panes separately. Say one sentence per pane.

Optional: `curl -i https://httpbin.org/get` (`-i` includes headers). Interpret the raw text.

2.7 Utilize common API authentication mechanisms: basic, custom token, and API keys

1. What Cisco expects me to know

Utilize three mechanisms: **HTTP Basic**, **custom token** (often Bearer), and **API keys**. Know **how they appear on the wire** (which header, which encoding) and how to send them with `requests`. Never commit secrets; use `labs/.env`.

2. Detailed explanation

APIs must know **who** the caller is (authentication). Authorization (what you may do) is separate (403 vs 401).

HTTP Basic. The client sends username and password on every request, combined as `username:password`, encoded with **Base64** (not encryption — anyone who sees the header can decode it). Hence **TLS is mandatory**.

```
Authorization: Basic YWRtaW46czNjcmV0
```

`YWRtaW46czNjcmV0` is Base64 for `admin:s3cret`. The server decodes and checks. RESTCONF on IOS XE often uses Basic in labs. `requests auth=(user, pass)` builds this header for you.

API keys. A long secret string issued by the platform (Meraki dashboard, some SaaS). Common pattern: a **custom header**:

```
X-Cisco-Meraki-API-Key: 1234567890abcdef
```

Sometimes a query parameter `?apiKey=` (worse: keys land in logs). Sometimes `Authorization: Bearer <key>`.

Utilize means: put the key **where the docs say**, not where another product puts it.

Custom tokens. After you authenticate (POST username/password to `/login` or an OAuth endpoint), the server returns a **token** (opaque string or JWT). Subsequent requests send:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ...
```

Catalyst Center (DNA Center) classic flow: POST to `/dna/system/api/v1/auth/token` with Basic, receive a token, then send `X-Auth-Token: <token>` — that is a **custom token header**, not the Meraki key header. “Custom token” on the blueprint covers Bearer **and** vendor-specific token headers.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph basic ["Basic"]
        B1["username:password"] --> B2["Base64"]
        B2 --> B3["Authorization: Basic ..."]
    end
    subgraph key ["API key"]
        K1["Dashboard-issued key"] --> K2["X-Cisco-Meraki-API-Key: ..."]
    end
    subgraph tok ["Custom token"]
        T1["POST login / OAuth"] --> T2["token string"]
        T2 --> T3["Authorization: Bearer ... or X-Auth-Token"]
    end
```

Comparison

Mechanism	What you send	Typical Cisco-shaped use	Risk
Basic	Base64 user:pass every request	RESTCONF lab devices	Password in header; must be HTTPS
API key	Static secret in a header	Meraki	Key = password; rotate; never Git
Custom token	Short-lived string after login	Catalyst Center, Webex, OAuth	Expire/refresh; still HTTPS

Webex: you create an integration or bot and get a token used as `Authorization: Bearer`. That is custom/Bearer token, not Basic.

3. Syntax and examples

Lab functions in `labs/03_rest_api/rest_client.py`.

Basic with `requests`

```
r = requests.get(
    "https://httpbin.org/basic-auth/admin/s3cret",
    auth=("admin", "s3cret"),
    timeout=15,
)
```

Manual equivalent (you should understand, rarely write):

```
import base64
token = base64.b64encode(b"admin:s3cret").decode("ascii")
headers = {"Authorization": f"Basic {token}"}
```

API key header (Meraki pattern, echoed by `httpbin /headers`)

```
headers = {
    "X-Cisco-Meraki-API-Key": "fake-key-for-lab",
    "Accept": "application/json",
}
r = requests.get("https://httpbin.org/headers", headers=headers, timeout=15)
```

httpbin returns the headers it received so you can **see** the key name. On a real Meraki call the URL would be `https://api.meraki.com/api/v1/ ...`.

Bearer / custom token

```
headers = {"Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.lab"}
r = requests.get("https://httpbin.org/bearer", headers=headers, timeout=15)
```

Catalyst Center **pattern** (host from sandbox docs):

```
auth_url = f"https://{host}/dna/system/api/v1/auth/token"
r = requests.post(auth_url, auth=(user, password), verify=False, timeout=15)
token = r.json()["Token"]
headers = {"X-Auth-Token": token, "Content-Type": "application/json"}
requests.get(f"https://{host}/dna/intent/api/v1/network-device", headers=headers, verify=False,
timeout=15)
```

Two mechanisms in sequence: Basic to **obtain** a custom token, then token header to **utilize** the API.

Wrong mixes (will 401)

- Meraki key placed in `Authorization: Basic`
- Bearer token sent as `X-Cisco-Meraki-API-Key`
- Basic user/pass in the JSON body when docs say the Authorization header

4. Exam-style understanding

You should be able to:

1. Build a Basic header conceptually (Base64 of `user:pass`).
2. Place an API key in a **custom header** when documented (Meraki).
3. Place a token in `Authorization: Bearer` or a vendor token header.
4. Choose the mechanism from docs, not from habit.
5. Explain why Base64 is not confidentiality.

Original practice: Docs: "Authenticate with header X-Cisco-Meraki-API-Key." Request includes `auth=` ("key", "") Basic. Result 401. Fix: `headers={"X-Cisco-Meraki-API-Key": key}`.

5. Hands-on exercise

```
cd labs\03_rest_api
python rest_client.py
```

Study the Basic, API-key, and Bearer sections of the printout.

1. Decode a Basic blob: in Python `import base64; base64.b64decode("YWRtaW46czNjcmV0")`.
2. Send `/basic-auth/admin/s3cret` **without** auth; confirm 401; add `auth=`.
3. Read Meraki getting started: <https://developer.cisco.com/meraki/api-v1/getting-started/> — note the **exact** header name.
4. Put a fake key in `labs/.env` (not in Git). Load it later with `python-dotenv` when you hit Domain 3 labs.

2.8 Compare common API styles (REST, RPC, synchronous, and asynchronous)

1. What Cisco expects me to know

Compare four ideas that are not all the same axis: **REST** vs **RPC** (style of interface), and **synchronous** vs **asynchronous** (style of waiting). Advantages, disadvantages, use cases.

2. Detailed explanation

REST (Representational State Transfer). You manipulate **resources** (nouns) at URIs with a uniform set of HTTP methods. `GET /interfaces/GigabitEthernet1` retrieves a representation (JSON). `PATCH` updates fields. Benefits: cacheable GETs, uniform interface, maps to HTTP well, easy to document per resource. Drawbacks: awkward for “reboot the box” (not a noun) — vendors add POST actions or RPC-like URLs. Statelessness simplifies scale.

RPC (Remote Procedure Call). You call **functions** (verbs) on a server: `CreateVlan(id=30)`, `GetHostname()`. NETCONF is XML RPC (`<rpc><get-config>`). JSON-RPC and gRPC are RPC families. SOAP is XML RPC over HTTP. Benefits: natural for operations and transactions; strong schemas (YANG, protobuf). Drawbacks: each API invents method names; harder to cache; you must know the procedure catalog. RESTCONF is “REST-like access to the same YANG models NETCONF RPCs use.”

	REST	RPC
Center	Resources / URIs	Procedures / methods
HTTP methods	Uniform GET/POST/PUT/PATCH/DELETE	Often POST everything
Example	<code>GET /vlans/30</code>	<code>POST /rpc body getVlan {id:30}</code>
Cisco-shaped	Meraki, RESTCONF, Webex	NETCONF, some SOAP, gRPC telemetry
Advantage	Predictable verbs, web-friendly	Expressive actions, transactional configs
Disadvantage	Verbs-as-nouns feel forced	Less uniform; client per method

Synchronous. The client sends a request and **waits** for the complete result in that HTTP response. `GET /hostname` → 200 `{"hostname": "edge-01"}`. Simple. Blocks the caller. Bad for “upgrade IOS” that takes 10 minutes — you would timeout (2.3).

Asynchronous. The call **starts** work and returns quickly with a **job id** (202 Accepted is a common HTTP pattern) or you use a webhook/callback (2.2) when done. You poll `GET /jobs/{id}` or wait for a notification. Benefits: long tasks, scale. Drawbacks: more states (queued, running, failed), more code, eventual consistency.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant C as Client
    participant A as API
    Note over C,A: Synchronous
    C->>A: POST /vlans
    A->>C: 201 + VLAN JSON
    Note over C,A: Asynchronous
    C->>A: POST /os-upgrade
    A->>C: 202 + jobId
    C->>A: GET /jobs/jobId
    A->>C: 200 running
    A->>C: webhook completed
```

Cross-cutting. REST can be sync (most CRUD) or async (202 + job). RPC can be sync (NETCONF `<rpc-reply>` in the same SSH session) or async (notification streams). Do not say “REST is synchronous and RPC is asynchronous” — those are **different comparisons**.

Use cases

Need	Style
CRUD on inventory objects	REST, synchronous
Replace whole config datastore	NETCONF RPC (edit-config)
Image install	Asynchronous job + poll or webhook
Chat message send	REST POST, usually synchronous 200
Streaming telemetry	Often RPC/gRPC, asynchronous stream

3. Syntax and examples

REST synchronous

```
r = requests.post("https://httpbin.org/post", json={"id": 30, "name": "GUEST"}, timeout=15)
print(r.status_code) # 200 from httpbin; a real API might 201
print(r.json()["json"])
```

You waited; the body is the result.

RPC-shaped HTTP (illustrative)

```
POST /api/rpc HTTP/1.1
Content-Type: application/json

{"method": "setHostname", "params": {"hostname": "edge-01"}, "id": 1}
```

One URL, method **inside** the body. Compare to REST `PUT /hostname` with `{"hostname": "edge-01"}`.

Asynchronous REST pattern

```
POST /api/v1/images/activate HTTP/1.1

HTTP/1.1 202 Accepted
Location: /api/v1/jobs/7f3c
Content-Type: application/json

{"jobId": "7f3c", "status": "accepted"}
```

Follow with GET `/api/v1/jobs/7f3c` until `status` is `success` or `failed`, or register a webhook.

NETCONF as RPC (preview of later domains)

```
<rpc message-id="101">
  <get-config>
    <source><running/></source>
  </get-config>
</rpc>
```

The operation is `get-config`, not `GET /running`. Same YANG data RESTCONF would expose as GET.

4. Exam-style understanding

You should be able to:

1. Contrast REST (resources, uniform verbs) vs RPC (named procedures).
2. Contrast sync (wait for result) vs async (job id / callback).
3. Give an advantage and disadvantage of each.
4. Avoid mixing the two axes.
5. Map NETCONF → RPC, Meraki → REST, OS upgrade → often async, GET hostname → sync.

Original practice: “POST `/reboot` that returns 200 when the box is already back” would be a **synchronous** lie if reboot takes 3 minutes — the API should be **async** 202. “POST `/rpc` with `getInterface`” vs GET `/interfaces/Gi1` is REST vs RPC.

5. Hands-on exercise

1. Table: REST vs RPC vs sync vs async with one Cisco-flavored example each.
2. Using `httpbin`, perform a synchronous POST and interpret 200 + echoed JSON (`rest_client.py`).
3. Simulate async: GET `https://httpbin.org/status/202` and write the next client step (GET `/jobs/{id}` or `webhook`).
4. Open a NETCONF example in `labs/06_yang_netconf_restconf/sample_netconf_get-config.xml` and a RESTCONF JSON in `sample_restconf_get.json`. Say which is RPC-shaped and which is REST-shaped.

2.9 Construct a Python script that calls a REST API using the requests library

1. What Cisco expects me to know

Construct a Python script with `requests`: send GET/POST (and friends), pass `params`, `json=`, `headers`, `auth`, `timeout`, then read `status_code` and `.json()`. Official docs: <https://requests.readthedocs.io/>. The teaching script is `labs/03_rest_api/rest_client.py`.

2. Detailed explanation

`requests` is the de facto HTTP library for this exam. It is **not** in the Python standard library (`pip install requests`, already in `labs/requirements.txt`).

Mental model: you build a Python function that (1) composes URL + headers + body from documentation (2.1), (2) adds auth (2.7), (3) enforces timeout (2.3), (4) inspects `status_code` (2.4–2.6), (5) parses JSON (1.2).

Core call signatures

```
requests.get(url, params=None, headers=None, auth=None, timeout=..., verify=True)
requests.post(url, json=None, data=None, headers=None, auth=None, timeout=...)
```

- `params` : dict → query string.
- `json` : dict/list → JSON body + Content-Type.
- `data` : form fields or raw bytes — **not** the same as `json=` .
- `headers` : dict of extra headers.
- `auth` : tuple (`user`, `password`) for Basic.
- `timeout` : seconds (or tuple). **Always set it.**
- `verify` : TLS validation.

Response object — you will use:

Attribute / method	Use
<code>r.status_code</code>	int 200, 401, ...
<code>r.reason</code>	OK , Not Found
<code>r.headers</code>	header map
<code>r.text</code>	decoded body string
<code>r.content</code>	raw bytes
<code>r.json()</code>	parse JSON → dict/list
<code>r.raise_for_status()</code>	throw if 4xx/5xx
<code>r.url</code>	final URL after redirects / params

Construction errors in Python

Bug	What happens
<code>json.dumps(payload)</code> passed to <code>json=</code>	Double-encoded string in JSON
<code>data=payload_dict</code> without headers	Form encoding, likely 415 on JSON APIs
No <code>timeout</code>	Can hang forever
<code>r.json()</code> on 204 or HTML	<code>ValueError</code>
Ignoring <code>status_code</code>	Treat HTML error pages as success

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    A["requests.get/post"] --> B[Response]
    B --> C["status_code"]
    B --> D["headers"]
    B --> E["json() or text"]
```

3. Syntax and examples

Install/import:

```
import requests
```

GET with query parameters

```
r = requests.get(
    "https://httpbin.org/get",
    params={"device": "csr1kv", "vrf": "mgmt"},
    timeout=15,
)
print(r.status_code)
print(r.json()["args"])
```

`params` produces `?device=csr1kv&vrf=mgmt`. `r.json()` is a dict; `["args"]` is httpbin's echo of the query.

POST JSON body

```
payload = {"hostname": "edge-01", "role": "wan"}
r = requests.post("https://httpbin.org/post", json=payload, timeout=15)
print(r.json()["json"]) # echoed body as object
```

`json=payload` is the correct construction. Contrast:

```
# usually wrong for REST JSON APIs
r = requests.post(url, data=payload, timeout=15)
```

Headers + API key

```
headers = {
    "Accept": "application/json",
    "X-Cisco-Meraki-API-Key": "fake-key-for-lab",
}
r = requests.get("https://httpbin.org/headers", headers=headers, timeout=15)
```

Basic auth


```
r = requests.get(
    "https://httpbin.org/basic-auth/admin/s3cret",
    auth=("admin", "s3cret"),
    timeout=15,
)
```

Bearer token

```
headers = {"Authorization": "Bearer TOKEN"}
r = requests.get("https://httpbin.org/bearer", headers=headers, timeout=15)
```

Branch on status (tie to 2.5)

```
if r.status_code == 200:
    data = r.json()
elif r.status_code == 401:
    raise SystemExit("Check auth")
else:
    raise SystemExit(f"Unexpected {r.status_code}: {r.text[:200]}")
```

Complete original script (pattern you should be able to write from docs)

```
"""Set a hostname via a fictional REST API – structure matches exam construction."""
import os
import requests

BASE = os.environ.get("API_BASE", "https://httpbin.org")
url = f"{BASE}/post"
headers = {"Accept": "application/json"}
body = {"hostname": "edge-01"}

response = requests.post(url, json=body, headers=headers, timeout=15)
print(response.status_code)
if response.status_code ≥ 400:
    print(response.text)
else:
    print(response.json())
```

Against httpbin this POST always echoes; against IOS XE RESTCONF you would change URL, Accept: application/yang-data+json, and auth=.

4. Exam-style understanding

You should be able to:

1. Write `requests.get` / `requests.post` with `timeout`.
2. Choose `params` vs `json` vs `headers` vs `auth` for a documented need.
3. Read `status_code` before trusting `.json()`.
4. Explain `json=` vs `data=`.

5. Assemble auth from 2.7 into the same script.

Original practice: Docs: GET `/organizations/{id}/devices` , header API key, query `perPage=10` . Construct:

```
requests.get(
    f"https://api.meraki.com/api/v1/organizations/{org}/devices",
    headers={"X-Cisco-Meraki-API-Key": key},
    params={"perPage": 10},
    timeout=15,
)
```

Missing `params` , putting `perPage` in JSON on a GET, or using Basic instead of the key header are construction failures.

5. Hands-on exercise

```
cd "C:\Users\Reydel\Documents\04_Learning\CCNA Automation"
.\.venv\Scripts\Activate.ps1
cd labs\03_rest_api
python rest_client.py
```

Then **write from scratch** (do not copy-paste blindly) a file `my_get.py` in that folder:

1. GET <https://httpbin.org/get> with `params={"lab": "2.9"}` .
2. Print `status_code` and `json()["args"]` .
3. Add `timeout=15` .
4. Extend: POST a dict of one interface (from `labs/02_data_formats/interfaces.json` first object) to `/post` using `json=` .
5. Add a 401 case: GET `/basic-auth/admin/s3cret` without auth; print a message using the same logic as `troubleshoot_http.py` .

Read <https://requests.readthedocs.io/> for GET query strings and JSON POST. When this is fluent, Domain 3 Cisco platform APIs are the same script with different URLs and headers from <https://developer.cisco.com/>.

Domain 1 and 2 study close-out

You should now be able to move data from **XML/JSON/YAML** into Python, structure code and tests, talk process and Git with your hands on a repo, and drive **REST** with documented requests, HTTP literacy, auth, and `requests` .

Minimum lab pass for these two domains

Objective cluster	Command / action
1.1–1.2	<code>python labs/02_data_formats/parse_formats.py</code>
1.3, 1.5	<code>python -m unittest labs/01_python_basics/test_subnet.py</code> and run <code>functions_classes_modules.py</code>
1.7–1.8	Checklist in <code>labs/04_git/workflow.yaml</code> ; read <code>labs/04_git/example.diff</code>
2.1, 2.4, 2.6, 2.7, 2.9	<code>python labs/03_rest_api/rest_client.py</code>
2.5	<code>python labs/03_rest_api/troubleshoot_http.py</code>

Bookmark set

- <https://developer.cisco.com/>
- <https://httpbin.org>
- <https://docs.python.org/3/library/json.html>
- <https://git-scm.com/docs>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- <https://requests.readthedocs.io/>

Next domains reuse this chapter constantly: Cisco platform APIs are 2.1+2.7+2.9 with vendor paths; NETCONF is XML + RPC; Ansible is YAML; CI is Git + tests. If a later topic feels opaque, the gap is often here — especially Domain 2.

3.0 Cisco Platforms and Development — 15%

Domain 3 is where the exam stops treating APIs as an abstract REST skill and starts treating them as **Cisco products you can talk to**. You already know HTTP methods, JSON, auth headers, and status codes from Domain 2. Here you apply that knowledge to real controller and device APIs: Meraki, Catalyst Center, ACI, Catalyst SD-WAN, NSO, compute (UCS / Intersight), collaboration (Webex / CUCM), security platforms, and on-box IOS XE / NX-OS programmability.

The exam does **not** make you a product administrator. The verb on most of 3.2–3.7 is **Describe**. That means: what the platform is for, what its API is used for, how you typically authenticate, and which resources you would query. 3.1 and 3.9 are **Construct**: given SDK or API docs, you write working Python. 3.8 is **Apply**: YANG, NETCONF, and RESTCONF in a Cisco environment — this one needs extra technical depth.

Credentials in this chapter are placeholders. Always-On and reservable sandboxes at <https://devnetsandbox.cisco.com/> now often issue **per-session** usernames and passwords. Copy them into `labs/.env` from `labs/.env.example`. Never paste blog passwords from 2019 into a script.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    subgraph north [Northbound APIs]
        Py[Python / SDK]
        REST[REST JSON]
        NC[NETCONF XML]
        RC[RESTCONF JSON/XML]
    end
    subgraph controllers [Controllers and SaaS]
        Meraki
        CatC[Catalyst Center]
        APIC[ACI APIC]
        vManage[Catalyst SD-WAN]
        NSO
        Webex
    end
    subgraph devices [Devices]
        IOSXE[IOS XE]
        NXOS[NX-OS]
    end
    Py --> REST
    Py --> NC
    Py --> RC
    REST --> Meraki
    REST --> CatC
    REST --> APIC
    REST --> vManage
    REST --> NSO
    REST --> Webex
    NC --> NSO
    NC --> IOSXE
    NC --> NXOS
    RC --> IOSXE
    RC --> NXOS
```

3.1 Construct a Python script that uses a Cisco SDK given SDK documentation

1. What Cisco expects me to know

Construct means you can read SDK documentation (classes, methods, required arguments, return types) and write a Python script that authenticates, calls a method, and uses the result. You are not expected to memorize every method of every Cisco SDK. You are expected to recognize the **SDK pattern**: instantiate a client with credentials, call a resource method, handle the response object.

Cisco publishes Python SDKs for many platforms (Meraki `meraki`, Catalyst Center `dnacentersdk`, Webex `webexpythonsdk`, ACI `cobra` / `acicobra`, and others). The exam can give you a snippet of documentation and ask you to complete a script. Lab `labs/05_cisco_apis/sdk_pattern.py` is a miniature version of that pattern so you can practice without depending on a package that changes.

2. Detailed explanation

An SDK (Software Development Kit) wraps raw HTTP so you do not assemble URLs and headers by hand. Under the hood it still does REST: it sets `Accept: application/json`, injects an API key or Bearer token, serializes Python dicts to JSON, and deserializes JSON back to Python objects.

When you read SDK docs, extract four facts:

1. **How the client is constructed.** Constructor arguments are almost always base URL (sometimes implied), API key or username/password, and optional `verify` for TLS.
2. **How resources are namespaced.** Meraki uses `dashboard.organizations.getOrganizations()`. Catalyst Center SDK uses `dnac.devices.get_device_list()`. Webex SDK uses `api.rooms.list()`.
3. **Required vs optional parameters.** Path parameters (org id, room id) are required. Query parameters (`timespan`, `family`) are optional.
4. **What comes back.** A `list` of dicts, a generator, or a response object with `.response` or `.items`.

Compare that to raw `requests`:

Approach	You write	You must know
Raw REST	URL, headers, JSON body, status check	HTTP details
SDK	Client + method + kwargs	Class/method names from docs

An SDK does **not** remove the need to understand the product model. If Meraki is `org` → `network` → `device`, the SDK methods still follow that hierarchy. If Catalyst Center needs a token first, the SDK obtains it in `__init__` and you never see `POST /dna/system/api/v1/auth/token` — unless you read the source.

Typical failure modes when using an SDK:

- Passing a REST path as a method argument (`"/organizations"`) when the method already encodes the path.
- Forgetting that some SDKs return generators (you must iterate or call `list()`).
- Disabling TLS verify in a lab (`verify=False`) and then leaving it that way in a real script.
- Mixing SDK versions: `webexteamssdk` was renamed; current package is `webexpythonsdk`. `dna_center` hostnames still appear in older docs; the product is Catalyst Center.

3. Syntax and examples

Pattern (local teaching SDK) — same idea as `labs/05_cisco_apis/sdk_pattern.py`:

```

from dataclasses import dataclass
import requests

@dataclass
class InventorySDK:
    base_url: str
    token: str
    verify: bool = True

    def _headers(self) → dict:
        return {
            "Authorization": f"Bearer {self.token}",
            "Accept": "application/json",
        }

    def get_devices(self, family: str | None = None) → list[dict]:
        params = {"family": family} if family else None
        r = requests.get(
            f"{self.base_url}/devices",
            headers=self._headers(),
            params=params,
            timeout=15,
            verify=self.verify,
        )
        r.raise_for_status()
        return r.json()

sdk = InventorySDK(base_url="https://controller.example.com/api", token="lab-token")
# devices = sdk.get_devices(family="Switches and Hubs")

```

Meraki official SDK (install with `pip install meraki`; docs at <https://developer.cisco.com/meraki/api-v1/getting-started/>):

```

import os
import meraki

dashboard = meraki.DashboardAPI(
    api_key=os.environ["MERAKEI_API_KEY"],
    base_url="https://api.meraki.com/api/v1",
    suppress_logging=True,
)

orgs = dashboard.organizations.getOrganizations()
org_id = orgs[0]["id"]
devices = dashboard.organizations.getOrganizationDevices(org_id)
for device in devices:
    print(device["name"], device["serial"], device.get("lanIp"))

```

Catalyst Center SDK (`dnacentersdk`; getting started: <https://developer.cisco.com/docs/catalyst-center/getting-started/>):

```
import os
from dnacentersdk import DNACenterAPI

dnac = DNACenterAPI(
    username=os.environ["CATALYST_CENTER_USER"],
    password=os.environ["CATALYST_CENTER_PASSWORD"],
    base_url=f"https://{os.environ['CATALYST_CENTER_HOST']}",
    version="2.3.7.6",
    verify=False, # lab sandboxes often use a private CA
)
devices = dnac.devices.get_device_list()
for row in devices["response"]:
    print(row["hostname"], row["managementIpAddress"])
```

Notice the class is still named `DNACenterAPI`. The product was rebranded to **Cisco Catalyst Center**; many URLs and SDK names still contain `dna`.

Webex SDK (docs: <https://developer.webex.com/docs/getting-started>):

```
import os
from webexpythonsdk import WebexAPI

api = WebexAPI(access_token=os.environ["WEBEX_TOKEN"])
me = api.people.me()
room = api.rooms.create(title="CCNAAUTO SDK lab")
api.messages.create(roomId=room.id, text=f"Hello from {me.displayName}")
```

When SDK docs show a method signature such as `getOrganizationDevices(organizationId, **kwargs)`, the exam-style task is: create the client, obtain `organizationId` from a prior call, then call that method. You map documentation parameter names onto Python kwargs.

4. Exam-style understanding

Given a documentation snippet, ask:

- What object do I instantiate, and with which credentials?
- Which method matches the required operation (list devices, send a message, get a token)?
- Which value from call A becomes an argument to call B? (org id → device list; room id → messages)
- Does the return value need indexing (`["response"]` , `["items"]` , or a plain list)?

Common traps:

- Using `requests.get` inside a question that already imported an SDK — they want the SDK method.
- Hard-coding a token in source. The professional pattern is `os.environ[ ... ]` (see `labs/.env.example`).
- Confusing **library name** (`dnacentersdk`) with **product name** (Catalyst Center) with **URL prefix** (`/dna/`).

If two answers both “work,” prefer the one that matches the documentation’s method name exactly.

5. Hands-on exercise

1. Read `labs/05_cisco_apis/sdk_pattern.py` and run it. Confirm the fake SDK injects an `Authorization: Bearer` header (httpbin echoes headers).
 2. Open <https://developer.cisco.com/meraki/api-v1/getting-started/> and locate `getOrganizations` and `getOrganizationDevices`. Write a five-line script that uses `meraki.DashboardAPI` instead of raw requests. Compare it to `labs/05_cisco_apis/meraki_list_devices.py`.
 3. Optional: `pip install dnacentersdk`, reserve a Catalyst Center sandbox at <https://devnetsandbox.cisco.com/>, put host/user/password in `labs/.env`, and list devices with `DNACenterAPI` as shown above.
-

3.2 Describe the capabilities of Cisco network management platforms and APIs (Meraki, Cisco Catalyst Center, ACI, Cisco Catalyst SD-WAN, and NSO)

1. What Cisco expects me to know

Describe the five named platforms: what job each one does in a network, what its API is for, how you typically authenticate, and which resources you would list or change. You will not configure a full SD-WAN overlay or design an ACI fabric on this exam. You will distinguish “cloud dashboard for branch/wireless” from “on-prem campus controller” from “data-center policy fabric” from “WAN overlay manager” from “service orchestrator.”

Official starting points:

- Meraki: <https://developer.cisco.com/meraki/api-v1/getting-started/>
- Catalyst Center: <https://developer.cisco.com/docs/catalyst-center/getting-started/>
- ACI: <https://developer.cisco.com/docs/aci/>
- NSO: <https://developer.cisco.com/docs/nso/>

2. Detailed explanation

Meraki is a **cloud-managed** networking family (MR wireless, MS switching, MX security/SD-WAN, cameras, sensors). The source of truth is the Meraki Dashboard in Cisco’s cloud, not an on-prem controller you install. The Dashboard API is REST + JSON at `https://api.meraki.com/api/v1`. Authentication is a Dashboard API key in the header `X-Cisco-Meraki-API-Key`. The object tree is **organization** → **networks** → **devices / clients / SSIDs / VLANs**. Use it to inventory hardware, list clients seen on a network, pull traffic analytics, and change a subset of dashboard settings programmatically. Rate limits are strict (typically a small number of GETs per second per org); production scripts must backoff on HTTP 429.

Cisco Catalyst Center (formerly **Cisco DNA Center**; REST paths still start with `/dna/`) is an **on-premises** (or Cisco-hosted) campus/branch controller: inventory, assurance, software image management, Plug and Play, templates, and intent-based policy. You authenticate with HTTP Basic against `POST /dna/system/api/v1/auth/token` and then send `X-Auth-Token` on subsequent calls. Intent APIs live under `/dna/intent/api/v1/...`. A representative read is `GET /dna/intent/api/v1/network-device`. Use it when the campus is Catalyst

switching/wireless and you need a single API for inventory, health, and clients — not when the estate is Meraki cloud.

ACI (Application Centric Infrastructure) is a **data-center leaf/spine fabric**. The controller is **APIC** (Application Policy Infrastructure Controller). The API is REST (JSON or XML) against the APIC; there is also the Cobra Python object model. You log in with `POST /api/aaaLogin.json` and receive a token cookie (`APIC-Cookie`). Everything is a managed object in a MIT (Management Information Tree): tenants, application profiles, EPGs, bridge domains, contracts. You query by class (`/api/class/fvTenant.json`) or distinguished name. Use ACI APIs to automate fabric policy, not campus QoS.

Cisco Catalyst SD-WAN (formerly Viptela / Viptela-based SD-WAN; people still say “vManage”) is the WAN overlay: vBond, vSmart, vManage, and WAN edge routers (vEdge / Catalyst 8xxx / ISR with SD-WAN image). The **vManage API** is REST. You typically `POST` to a session/token endpoint, then reuse a `JSESSIONID` cookie or token header to list devices, templates, and statistics. Use it to inventory WAN edges, push device templates, and read overlay health — not to manage a Meraki MX (that is Dashboard) and not to manage an ACI leaf.

NSO (Network Services Orchestrator) is a **multi-vendor service orchestrator**, not a device controller for one domain. You define **service models** (YANG) that map to **device models**. Northbound, NSO speaks REST, RESTCONF, and NETCONF. Southbound, it talks to devices via CLI, NETCONF, SNMP, and vendor APIs. Use NSO when the requirement is “provision a L3VPN / VLAN / ACL as a service across many devices,” not “get a wireless client list from one dashboard.” Docs: <https://developer.cisco.com/docs/nso/>.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph merakiCloud [Meraki cloud]
        Dash[Dashboard API]
    end
    subgraph campus [Campus]
        CC[Catalyst Center /dna/]
    end
    subgraph dc [Data center]
        APIC2[APIC REST / Cobra]
    end
    subgraph wan [WAN overlay]
        vM[vManage API]
    end
    subgraph orch [Multi-vendor]
        NSO2[NSO REST / NETCONF]
    end
    Dash --> MR[MR / MS / MX]
    CC --> Cat[Catalyst switches / WLC]
    APIC2 --> Leaf[ACI leaf / spine]
    vM --> Edge[SD-WAN edges]
    NSO2 --> Mix[IOS XE / NX-OS / others]
```

3. Syntax and examples

Meraki — list organizations (raw REST):

```
GET https://api.meraki.com/api/v1/organizations
X-Cisco-Meraki-API-Key: <dashboard-api-key>
Accept: application/json
```

Catalyst Center — token then devices:

```
POST https://{host}/dna/system/api/v1/auth/token
Authorization: Basic base64(user:password)

GET https://{host}/dna/intent/api/v1/network-device
X-Auth-Token: <Token from previous JSON>
```

ACI — login then class query:

```
POST https://{apic}/api/aaaLogin.json
Content-Type: application/json

{"aaaUser": {"attributes": {"name": "admin", "pwd": "<password>"}}}
```

```
GET https://{apic}/api/class/fabricNode.json
Cookie: APIC-Cookie=<token>
```

Catalyst SD-WAN (vManage) — session then device list (shape; exact path can vary by version):

```
POST https://{vmanage}/j_security_check
Content-Type: application/x-www-form-urlencoded

j_username=user&j_password=pass
```

Then GET /dataservice/device with the session cookie. Some versions also expose a token endpoint; always follow the API doc for the sandbox version you reserved.

NSO — REST northbound (example shape):

```
GET https://{nso}/restconf/data/tailf-ncs:devices/device
Accept: application/yang-data+json
Authorization: Basic ...
```

NSO can also be driven with NETCONF RPCs against the same service/device tree. That is the point: NSO is a YANG datastore in front of many devices.

4. Exam-style understanding

Match the **requirement** to the **platform**:

Requirement	Platform	Why
List cloud-managed APs and MX appliances	Meraki	Dashboard is the inventory
Campus assurance, image upgrade, intent API	Catalyst Center	/dna/intent
Tenant / EPG / contract in a leaf-spine fabric	ACI / APIC	Policy MIT
List WAN edge routers in an overlay	Catalyst SD-WAN / vManage	Overlay control plane
Provision a service across mixed vendors	NSO	Service model + southbound

Auth-style recognition:

- Header API key → Meraki
- Token from /dna/system/api/v1/auth/token + X-Auth-Token → Catalyst Center
- aaaLogin.json + cookie → ACI
- vManage session/cookie → Catalyst SD-WAN
- YANG/RESTCONF or NETCONF northbound → NSO (also used on devices; context tells you which)

Do not mix **Meraki Dashboard** with **Catalyst Center** just because both can list wireless clients. One is cloud-key REST; the other is on-prem token REST under /dna/.

5. Hands-on exercise

1. Skim the five doc landing pages listed in section 1. For each, write one sentence: purpose, auth, one resource path.
2. Run labs/05_cisco_apis/meraki_list_devices.py with a Dashboard key in labs/.env (personal non-prod org or a Meraki sandbox if listed).
3. Reserve **Catalyst Center** at <https://devnetsandbox.cisco.com/>, copy per-session credentials, run labs/05_cisco_apis/catalyst_center_devices.py.
4. Optional: reserve **ACI** or **SD-WAN** sandbox and issue only the login + one GET from Postman. You are proving you can authenticate and read inventory, not designing a fabric.

3.3 Describe the capabilities of Cisco compute management platforms and APIs (UCS Manager and Intersight)

1. What Cisco expects me to know

Describe two compute-management planes: **UCS Manager** (on-system / fabric-interconnect domain) and **Cisco Intersight** (cloud SaaS operations). Know what each manages (blades, rack servers, profiles, firmware), that UCS Manager's historic API is XML-based, and that Intersight is REST with API keys. You are not racking a chassis on this exam.

2. Detailed explanation

UCS Manager runs on a pair of **Fabric Interconnects** and manages a UCS domain: blade chassis, rack-mount servers, service profiles (identity, firmware, SAN/LAN connectivity), pools, and policies. The northbound API is the **UCS XML API** (HTTPS, XML documents, cookies after `aaaLogin`). Cisco also ships a **UCS Python SDK** that wraps those XML methods so you can query `computeBlade`, `computeRackUnit`, and service-profile objects without hand-writing XML. Use UCS Manager APIs when the servers live in a classic UCS domain and the source of truth is the FI pair.

Cisco Intersight is a **cloud operations** platform for Cisco compute (UCS, HyperFlex, some networking, and third-party targets). You claim devices to Intersight; then you inventory, firmware, policy, and (in Intersight Service for Terraform / workflows) automate from the cloud. The API is **REST + JSON**. Authentication uses an **API key id + secret** (often as HTTP signatures) rather than a UCS Manager XML login. Use Intersight when the requirement is SaaS-wide compute ops, multi-domain inventory, or cloud-triggered workflows — not when you must talk only to an air-gapped FI with no cloud connector.

Relationship: a UCS domain can be managed locally by UCS Manager **and** claimed into Intersight. Intersight does not replace every XML API call on day one; it is the cloud control plane. For the exam, keep the split clean: **UCS Manager = domain XML/SDK, Intersight = cloud REST/API keys**.

3. Syntax and examples

UCS Manager XML login (shape):

```
<aaaLogin inName="admin" inPassword="***" />
```

Sent as an XML document over HTTPS to the FI. Subsequent queries use the returned cookie and class IDs such as `computeBlade` or `lsServer` (service profile). The Python SDK hides this:

```
# Conceptual UCS SDK usage – connect to FI, query blades
# from ucsm.sdk.ucshandle import UcsHandle
# handle = UcsHandle("10.10.20.40", "admin", password)
# handle.login()
# blades = handle.query_classid("computeBlade")
```

Intersight REST (shape):

```
GET https://intersight.com/api/v1/compute/PhysicalSummaries
Authorization: <Intersight API key signature>
Accept: application/json
```

You create keys in the Intersight UI (Settings → API Keys). Scripts use the official `intersight-python` SDK or signed REST. Never commit the secret key file.

4. Exam-style understanding

Question in disguise	Answer
On-prem FI pair, blades, service profiles, XML	UCS Manager
Cloud SaaS, API keys, multi-domain compute inventory	Intersight
Python wrapping XML class IDs	UCS Python SDK
REST JSON to <code>intersight.com</code>	Intersight API

If a scenario says “no cloud connectivity,” Intersight is the wrong primary tool. If it says “one dashboard for UCS domains in three data centers,” Intersight is the intended capability.

5. Hands-on exercise

1. Open DevNet and search **UCS Manager** and **Intersight** API docs. Note auth type for each (XML login vs API key).
2. If an **Intersight** or **UCS** sandbox is listed at <https://devnetsandbox.cisco.com/>, reserve it and perform **one** authenticated GET (inventory). Otherwise, read a public Intersight OpenAPI spec page and write down three resource names (`compute/PhysicalSummaries` , firmware, hyperflex).
3. In your notes, draw a two-box diagram: FI domain (UCSM) versus Intersight cloud, with an optional “claim” arrow.

3.4 Describe the capabilities of Cisco collaboration platforms and APIs (Webex, Webex devices, Cisco Unified Communications Manager including AXL and UDS interfaces)

1. What Cisco expects me to know

Describe three collaboration surfaces: **Webex** (cloud messaging/meetings/calling APIs), **Webex devices** (room/desk devices, xAPI / cloud APIs), and **Cisco Unified Communications Manager (CUCM)** with **AXL** (admin SOAP) and **UDS** (user-facing REST). You should know which API you use to create a space and post a message versus which API you use to add a phone DN on a cluster.

Webex getting started: <https://developer.webex.com/docs/getting-started>

2. Detailed explanation

Webex (formerly Webex Teams / Spark) is Cisco’s cloud collaboration platform. The REST API at `https://webexapis.com/v1` uses an **OAuth or bot Bearer token**: `Authorization: Bearer <token>`. Core resources for this exam:

- `/people` — users; `GET /people/me` is the identity check

- `/rooms` — spaces (the product UI says “spaces”; the API still says rooms)
- `/memberships` — participants in a space
- `/messages` — posts in a space (and direct messages)
- `/webhooks` — notifications when messages or memberships change (Domain 2 overlap)

Bots and integrations are first-class: you create a bot at the developer portal, copy its token, and it posts as that bot identity.

Webex devices (Room Kit, Board, Desk) expose **xAPI** (in-room: SSH/HTTP to the codec; cloud: Control Hub / xAPI over the Webex cloud). You can set volume, place a call, send a message to the on-screen UI, or read people-count. For the exam, remember: **cloud Webex REST is for spaces/messages; device xAPI is for the hardware in the room.**

CUCM is the on-premises call-control cluster (phones, DNs, partitions, route patterns). Two APIs appear on the blueprint:

- **AXL (Administrative XML)** — SOAP/XML over HTTPS. This is the **admin** interface: add a user, add a phone, change a DN. You enable the AXL web service on CUCM and authenticate as an AXL-privileged application user. Payloads are XML envelopes, not JSON.
- **UDS (User Data Services)** — **REST** interface aimed at **user-facing** apps: directory lookup, user settings, extension mobility-style data. UDS is not the tool you use to provision 5,000 phones; AXL is.

Do not confuse Webex Calling APIs (cloud PSTN/calling) with CUCM AXL. The blueprint groups Webex and CUCM together as collaboration, but the **protocol split** (REST Bearer vs SOAP AXL vs REST UDS) is the scoring detail.

3. Syntax and examples

Webex — list spaces and post a message:

```
GET https://webexapis.com/v1/rooms
Authorization: Bearer <token>

POST https://webexapis.com/v1/messages
Authorization: Bearer <token>
Content-Type: application/json

{"roomId": "Y2lzY29zcGFyazovL3VzL1JPT00v ... ", "text": "Interface Gig1 is down"}
```

Python equivalent is in `labs/05_cisco_apis/webex_rooms.py`.

CUCM AXL — SOAP shape (not JSON):

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns="http://www.cisco.com/AXL/API/14.0">
  <soapenv:Body>
    <ns:getPhone>
      <name>SEPAAAABBBBCCCC</name>
    </ns:getPhone>
  </soapenv:Body>
</soapenv:Envelope>
```

Posted to `https://{cucm}:8443/axl/` with HTTP Basic.

UDS — REST directory (shape):

```
GET https://{cucm}:8443/cucm-uds/users
Accept: application/xml
```

UDS responses are typically XML even though the style is REST. That is a frequent “gotcha” if you expected JSON.

4. Exam-style understanding

Task	Interface
Create a Webex space, add a person, post text	Webex REST <code>/rooms</code> <code>/memberships</code> <code>/messages</code>
Mute a Room Kit or set do-not-disturb on the codec	Webex devices / xAPI
Add a phone and DN on a CUCM cluster	AXL (SOAP admin)
Let a custom app look up a user in the corporate directory	UDS

If the payload in a question is an XML SOAP envelope with `getPhone`, the answer is AXL, not Webex. If the header is `Authorization: Bearer` and the host is `webexapis.com`, it is Webex.

5. Hands-on exercise

1. Create a free Webex account and a **Bot** at <https://developer.webex.com/docs/getting-started>. Put the token in `labs/.env` as `WEBEX_TOKEN`.
2. Run `labs/05_cisco_apis/webex_rooms.py`. Confirm it creates a space, posts a message, lists memberships, then deletes the space.
3. In the Webex API Reference, open **Rooms**, **Messages**, and **Memberships**. Note required fields (`title`, `roomId`, `personEmail`).
4. Optional CUCM: only if a CUCM sandbox is available — send one AXL `getVersion` (or equivalent) from Postman. Skip if you have no cluster; reading a sample AXL envelope is enough to recognize SOAP vs REST.

3.5 Describe the capabilities of Cisco security platforms and APIs (XDR, Firepower, Secure Connect, Secure Endpoint, ISE, and Secure Malware Analytics)

1. What Cisco expects me to know

Describe each named security platform at capability level: what it does, what its API is used for, and the kind of resource you would query (incidents, events, sessions, samples). This is not a CCNP Security course. Product names have shifted; follow the **official 200-901 list** below, and treat older names as aliases you might still see in docs.

2. Detailed explanation

Cisco XDR (Extended Detection and Response) correlates detections across email, endpoint, network, and cloud into **incidents** a SOC can hunt. APIs are REST: list incidents, pull related observables, update investigation status. Think “single investigation object that stitches Firepower events + Secure Endpoint events + email,” not “configure an ACL on a firewall.” Older materials may mention **SecureX**; XDR is the blueprint name for this correlation layer.

Firepower here means the **Firepower / FTD** threat-defense family managed by **FMC (Firepower Management Center)** or cloud management. The **FMC REST API** is used for access policies, network objects, and events (intrusion, connection, malware). Auth is typically a token from a username/password login against FMC, then a header on later calls. Use it to automate object creation or pull events — not to replace the entire FMC GUI on the exam.

Cisco Secure Connect is on the official v1.1 blueprint. It is Cisco’s **SASE / secure access** offering: users and branches connect to a cloud-delivered security stack (secure web gateway, zero-trust access, DNS-layer security, and related services) instead of backhauling everything to a DC firewall. APIs expose reporting, identity/connector status, and policy for that cloud edge. **Cisco Umbrella** still appears in older exam matrices and in lots of sample code because DNS-layer security and much of the Secure Access DNA came from Umbrella. For this exam, if the blueprint says Secure Connect, describe **cloud SASE/secure access APIs**; if a lab or older question says Umbrella, recognize it as the DNS/cloud-security ancestor/component of that story — do not ignore the official name.

Cisco Secure Endpoint is the **endpoint** AMP successor (you will still see **AMP for Endpoints** in URLs and SDKs). APIs list **computers**, **events** (detections, quarantines), and **file lists**. Auth is typically an API client id/key. Use it to ask “which hosts saw this SHA256?” not “which switch port is this MAC on?” (that is Catalyst Center / Meraki).

Cisco ISE (Identity Services Engine) is the **policy and identity** engine: 802.1X, profiling, guest, pxGrid. Two developer surfaces matter:

- **ERS (External RESTful Services)** — REST for admin objects: network devices, endpoint groups, internal users, and policies.
- **pxGrid** — a publish/subscribe and query bus so other platforms (Firepower, XDR, Rapid Threat Containment) learn session/identity context.

Use ISE APIs when the data is **who/what/where on the network** (session, identity group), not packet captures.

Cisco Secure Malware Analytics is the **Threat Grid** malware sandbox: you submit a file or URL, the service detonates it, and you retrieve behavioral indicators, network callbacks, and scores via API. Use it when the workflow is “detonate this sample,” not “block this IP on FMC” (though XDR/FMC may consume the result).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    EP[Secure Endpoint events] --> XDR[XDR incidents]
    FP[Firepower / FMC events] --> XDR
    SC[Secure Connect / access telemetry] --> XDR
    ISE[ISE sessions / pxGrid] --> XDR
    TG[Secure Malware Analytics] --> XDR
```

3. Syntax and examples

Illustrative REST shapes (hosts and exact paths vary by region and version — use the product’s API reference):

```
GET https://{xdr}/iroh/iroh-intel/investigate/incidents
Authorization: Bearer <xdr-token>
```

```
POST https://{fmc}/api/fmc_platform/v1/auth/generatetoken
Authorization: Basic ...
```

```
GET https://api.amp.cisco.com/v1/computers
Authorization: Basic <secure-endpoint-client-credentials>
```

```
GET https://{ise}:9060/ers/config/networkdevice
Accept: application/json
Authorization: Basic <ers-admin>
```

```
POST https://panacea.threatgrid.com/api/v2/samples
```

(Secure Malware Analytics / Threat Grid sample submit; API key as a query or header per current docs.)

You do not need to memorize every path. You need to recognize **incident correlation (XDR)**, **firewall policy/events (Firepower/FMC)**, **SASE/cloud access (Secure Connect)**, **endpoint computers/events (Secure Endpoint)**, **identity/policy (ISE ERS/pxGrid)**, **sandbox detonation (Secure Malware Analytics)**.

4. Exam-style understanding

Need	Platform
Stitch detections into one incident	XDR
Access-control policy objects / IPS events	Firepower / FMC
Cloud user/branch secure access (SASE)	Secure Connect
Host quarantine events, SHA256 on a laptop	Secure Endpoint
802.1X session, ANC, network device objects	ISE
Detonate a suspicious binary	Secure Malware Analytics

Name-change traps: AMP → Secure Endpoint; Threat Grid → Secure Malware Analytics; Umbrella → related to Secure Connect / Secure Access; DNA Center is **not** a security platform in 3.5 (it is 3.2).

5. Hands-on exercise

1. Make a six-row table in your notes: platform, one-line job, auth style, one resource.
2. Browse DevNet docs for **ISE ERS** and **Secure Endpoint** (AMP) — they have the most approachable REST examples.
3. If a **SecureX / XDR** or **ISE** sandbox appears at <https://devnetsandbox.cisco.com/>, reserve it and perform a single authenticated list call. Otherwise, skip live labs; 3.5 is Describe, not Construct.

3.6 Describe the device level APIs and dynamic interfaces for IOS XE and NX-OS

1. What Cisco expects me to know

Describe how you program **the device itself** (not a controller) on **IOS XE** and **NX-OS**: NETCONF, RESTCONF, gNMI (IOS XE), on-box Python / Guest Shell, NX-API CLI (JSON-RPC), and NX-API REST. Know default transports (NETCONF 830/tcp SSH, RESTCONF 443 HTTPS) and that these APIs are **model-driven or CLI-encoded**, unlike expect/SSH scraping.

NX-OS developer docs: <https://developer.cisco.com/docs/nx-os/>

2. Detailed explanation

IOS XE (Catalyst 9000, Catalyst 8000 / CSR-like routers, many ISR/ASR with XE) enables a programmability stack:

- **NETCONF** — XML RPCs over SSH, port **830/tcp**. YANG models (IETF + Cisco-IOS-XE-native).
- **RESTCONF** — HTTPS, usually **443**, YANG identified in the URL, JSON or XML encoding, media type `application/yang-data+json`.

- **gNMI / gRPC** — streaming telemetry and set/get using protobuf; common for high-scale telemetry, less for “first REST lab.”
- **Guest Shell** — a CentOS/Alma-like container on the device (`guestshell enable`) where you run **on-box Python**, EEM-triggered scripts, and even `pip` in that environment. The script runs **on the router**, not on your laptop.
- **On-box Python** — IOS XE can also run Python in other on-box contexts; Guest Shell is the one you should be able to name.
- **Model-driven telemetry** — periodic or on-change YANG subscriptions (dial-out/dial-in).

You still use SSH/CLI, SNMP, and RESTCONF together; the exam wants you to pick the **API that matches the job** (structured config get → NETCONF/RESTCONF; on-box reaction to syslog → Guest Shell Python).

NX-OS (Nexus switching, especially **Nexus 9000**) adds Cisco’s own HTTP APIs on top of the same industry stack on many platforms:

- **NX-API CLI** — HTTPS JSON-RPC (or XML) that sends **CLI commands** and returns structured output. This is “CLI over HTTP,” not YANG. You POST a JSON body with `"cmd": "show version"` to `/ins`.
- **NX-API REST** — object-based REST for DME (Data Management Engine) objects on NX-OS; different from NX-API CLI.
- **NETCONF / RESTCONF** — available on many Nexus 9K images; same YANG idea as IOS XE, different native models (`Cisco-NX-OS-device` vs `Cisco-IOS-XE-native`).
- **on-box Python / Guest Shell** — similar idea to IOS XE on supported Nexus platforms.
- **NX-SDK / NX-API** — for custom on-switch applications (beyond CCNA Automation depth).

Dynamic interfaces in this objective means the **programmable interfaces that change with the device model** (YANG-driven NETCONF/RESTCONF, NX-API object model), not a “dynamic interface” as in Frame Relay. Open-source and Cisco tools discover capabilities at run time (`hello` in NETCONF, YANG library in RESTCONF).

Enablement (conceptual IOS XE):

```
netconf-yang
restconf
ip http secure-server
```

Without those, your laptop script gets connection refused or 404. Sandbox images usually have them on.

3. Syntax and examples

RESTCONF on IOS XE (see `labs/06_yang_netconf_restconf/restconf_get_interfaces.py`):

```
GET https://{host}:443/restconf/data/ietf-interfaces:interfaces
Accept: application/yang-data+json
Authorization: Basic ...
```

NETCONF on IOS XE (see `labs/06_yang_netconf_restconf/netconf_get_config.py`): TCP 830, XML `<get-config>`, datastore `running`.

NX-API CLI JSON-RPC:

```
POST https://{nexus}/ins
Content-Type: application/json

{
  "ins_api": {
    "version": "1.0",
    "type": "cli_show",
    "chunk": "0",
    "sid": "1",
    "input": "show version",
    "output_format": "json"
  }
}
```

Guest Shell (IOS XE CLI):

```
guestshell enable
guestshell run python3
```

Inside Guest Shell you can `import cli` (Cisco-provided) to run IOS commands from Python on-box.

4. Exam-style understanding

Feature	IOS XE	NX-OS
NETCONF 830	Yes (enable <code>netconf-yang</code>)	Yes on many 9K
RESTCONF 443	Yes (<code>restconf</code>)	Yes on many 9K
JSON-RPC CLI over HTTP	Not the headline API	NX-API CLI
On-box Linux + Python	Guest Shell	Guest Shell / bash (platform-dependent)
Native YANG	Cisco-IOS-XE-native	Cisco-NX-OS-device

If the question shows a JSON body with `ins_api` and `cli_show` , that is **NX-API CLI**, not RESTCONF. If the URL contains `/restconf/data/` and `ietf-interfaces` , that is RESTCONF (either OS). If the person is SSH'd to the router running Python locally, that is **on-box / Guest Shell**, not your laptop `requests` script.

5. Hands-on exercise

1. Launch **IOS XE on Catalyst 8000 Always-On** (or reservable IOS XE) at <https://devnetsandbox.cisco.com/>. Copy **this session's** host/user/password into `labs/.env` . Do not reuse passwords from old blog posts.
2. Run `python labs/06_yang_netconf_restconf/restconf_get_interfaces.py` and `python labs/06_yang_netconf_restconf/netconf_get_config.py` .
3. Open <https://developer.cisco.com/docs/nx-os/> and find an NX-API CLI example. Compare the JSON body to RESTCONF. Write one sentence: "NX-API CLI sends CLI; RESTCONF sends YANG resource paths."

4. Optional: reserve a Nexus sandbox and POST `show version` via NX-API in Postman.

3.7 Describe the appropriate DevNet resource for a given scenario (Sandbox, Code Exchange, support, forums, Learning Labs, and API documentation)

1. What Cisco expects me to know

Describe which **DevNet** resource you would use for a given job: try a live API, copy sample code, take a guided lab, read a reference, or ask for help. The list on the blueprint is: **Sandbox, Code Exchange, support, forums, Learning Labs, and API documentation**.

2. Detailed explanation

Sandbox — <https://devnetsandbox.cisco.com/> — hosted labs: Always-On (shared, sometimes busy, credentials often **per-session**) and reservable (you get a VPN or proxy and a private topology for a few hours). Use a sandbox when you need a **real APIC / Catalyst Center / IOS XE / vManage** and you do not have hardware. Sandboxes are not production. They reset. They are the correct answer to “I need to test NETCONF against IOS XE for free.”

Code Exchange — <https://developer.cisco.com/codeexchange/> — a catalog of **sample projects** (often GitHub) tagged by product. Use it when you want a **starting repository** (Ansible role, Python collector, Terraform provider example), not the authoritative method-by-method reference.

API documentation — product-specific reference (OpenAPI, resource lists, auth). Examples: Meraki API v1, Catalyst Center APIs, NX-OS, NSO, Webex. Use docs when you need **exact paths, schemas, and status codes**. Docs do not give you a live device; the sandbox does.

Learning Labs — guided, in-browser or step-by-step DevNet courses (REST, Python, specific APIs). Use them when you need a **tutorial path**, not a copy-paste production SDK.

Forums — DevNet Community / Cisco community boards. Use them when you have a **specific error** (401 on token, YANG path 404) and docs plus sandbox still fail. Forums are not authoritative API contracts.

Support — TAC / DevNet support / product support depending on entitlement. Use support when you have a **production outage or a defect**, not when you are still learning `requests.get`. For exam purposes: sandbox + docs + Code Exchange are self-serve; support is the escalation path for customers with a contract.

3. Syntax and examples

There is no CLI for this objective. Practice the **mapping**:

Need a live Catalyst Center → Sandbox
Need the JSON schema for GET /networks/{id}/clients → API documentation
Need a GitHub sample that already uses dnacentersdk → Code Exchange
Need a 20-minute walkthrough of RESTCONF → Learning Labs
Need help with a 500 from a shared Always-On box after checking docs → Forums
Production NSO bug on a paid deployment → Support / TAC

Bookmark:

- Sandbox: <https://devnetsandbox.cisco.com/>
- Code Exchange: <https://developer.cisco.com/codeexchange/>
- Meraki docs: <https://developer.cisco.com/meraki/api-v1/getting-started/>
- Catalyst Center docs: <https://developer.cisco.com/docs/catalyst-center/getting-started/>
- ACI docs: <https://developer.cisco.com/docs/aci/>
- NSO docs: <https://developer.cisco.com/docs/nso/>
- Webex docs: <https://developer.webex.com/docs/getting-started>

4. Exam-style understanding

The exam likes **scenario** → **resource**:

- “You want to try the Meraki API but have no org” → Sandbox (or a free Dashboard org; sandbox is the DevNet answer).
- “You want community Python that lists SD-WAN devices” → Code Exchange.
- “You need the exact query parameter name” → API documentation.
- “You are new and want a guided NETCONF lab” → Learning Labs.
- “Docs disagree with what the sandbox returns and you need discussion” → Forums.
- “A production API is down for a customer with a contract” → Support.

Do not pick Code Exchange when the question asks for **normative** request/response fields — that is API documentation. Do not pick Sandbox when the question asks where to **find sample code** — that is Code Exchange.

5. Hands-on exercise

1. Log in to <https://devnetsandbox.cisco.com/> and note one Always-On and one reservable lab. Read the **lab instructions** for credentials (session-specific).
 2. Open <https://developer.cisco.com/codeexchange/> and search `meraki`. Star or clone nothing proprietary; just observe that samples are full projects, not API references.
 3. Open Catalyst Center getting-started docs and find the token URL. Confirm it matches `labs/05_cisco_apis/catalyst_center_devices.py`.
-

3.8 Apply concepts of model driven programmability (YANG, RESTCONF, and NETCONF) in a Cisco environment

1. What Cisco expects me to know

Apply means you can use YANG, RESTCONF, and NETCONF together in a Cisco (typically IOS XE) environment: what each artifact is, how they relate, how data is encoded, which datastore you read or write, which RPC or HTTP method you use, which ports and auth apply, and how to read a reply. This is the deepest “describe/apply” topic in Domain 3. Labs: `labs/06_yang_netconf_restconf/` (`sample.yang` , `restconf_get_interfaces.py` , `netconf_get_config.py` , sample JSON/XML replies).

2. Detailed explanation

YANG is a **data modeling language** (RFC 7950). It is not a transport and not an encoding. A YANG module describes the shape of configuration and operational state: containers, lists, leafs, types, keys, and whether a node is `config true` (writable) or `config false` (operational only). Cisco devices and controllers load standard IETF modules (`ietf-interfaces` , `ietf-yang-library`) plus vendor modules (`Cisco-IOS-XE-native` , `Cisco-NX-OS-device`). Your script never “sends YANG” on the wire. It sends **XML or JSON that must validate against a YANG model**.

Core YANG building blocks (see the teaching excerpt `labs/06_yang_netconf_restconf/sample.yang`):

- `module / namespace / prefix` — identity of the model
- `container` — a grouping of nodes (no key)
- `list` — repeating entries with a `key`
- `leaf` — a single value with a `type`
- `config false` — operational state (e.g. `oper-status`), not saved in running-config

NETCONF (RFC 6241) is a **network configuration protocol**. Transport for Cisco IOS XE is **SSH on TCP port 830**.

Payloads are **XML**. The client and server exchange `<hello>` with **capabilities** (which YANG modules, which datastores, which operations). Then the client sends **RPCs** (Remote Procedure Calls) and the server returns `<rpc-reply>` .

Important NETCONF operations:

RPC	Purpose
<code><get-config></code>	Read configuration from a datastore (<code>running</code> , <code>candidate</code> , <code>startup</code>)
<code><get></code>	Read config and operational state (filtered)
<code><edit-config></code>	Write configuration into a datastore
<code><copy-config></code>	Copy one datastore to another
<code><delete-config></code>	Delete a datastore (not <code>running</code>)
<code><lock></code> / <code><unlock></code>	Exclusive access to a datastore
<code><commit></code>	Candidate → running (when candidate capability exists)
<code><discard-changes></code>	Drop uncommitted candidate edits
<code><close-session></code> / <code><kill-session></code>	End sessions

Datastores:

- **running** — the active configuration (what the device is using)
- **candidate** — a scratch copy you edit then `<commit>` (if advertised)
- **startup** — what loads at boot (like startup-config)
- **operational** — not a NETCONF configuration datastore; operational state is read with `<get>` (and RESTCONF operational resources). YANG `config false` nodes live here conceptually.

IOS XE often exposes running (and candidate depending on image/config). Do not assume every box has a writable candidate.

RESTCONF (RFC 8040) is **HTTP(S) access to the same YANG data**. On IOS XE it is typically **HTTPS TCP 443**, Basic or token auth as configured, with media types:

- `application/yang-data+json`
- `application/yang-data+xml`

RESTCONF maps YANG to URLs:

```
https://{host}/restconf/data/{module}:{container}/{list}={key}/ ...
```

Example: `ietf-interfaces:interfaces/interface=GigabitEthernet1`

HTTP methods:

Method	Meaning on a data resource
GET	Read (config or oper depending on path)
POST	Create a child resource
PUT	Create or replace this resource
PATCH	Merge/edit
DELETE	Remove the resource
OPTIONS / HEAD	Discovery / headers

RESTCONF also has `/restconf/operations/{rpc}` for YANG-defined RPCs (not the same as NETCONF's XML `<get-config>` wrapper, but the idea of "invoke an operation" is analogous).

How they relate:

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TB
    YANG[YANG models: IETF + Cisco native]
    YANG --> NETCONF
    YANG --> RESTCONF
    NETCONF -->|"XML RPCs over SSH :830"| Device
    RESTCONF -->|"HTTP methods over TLS :443"| Device
    Device --> Running[(running)]
    Device --> Candidate[(candidate)]
    Device --> Startup[(startup)]
    Device --> Oper[(operational state)]

```

Encoding: NETCONF is XML on the wire. RESTCONF can be JSON or XML; Cisco labs almost always use JSON. JSON uses module prefixes in keys (`ietf-interfaces:interfaces`). XML uses namespaces (`xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"`). Same data; different serialization.

Auth: NETCONF uses SSH username/password (or keys). RESTCONF uses HTTPS + HTTP Basic (common on IOS XE labs) or other HTTP auth the platform enables. Both should be treated as management-plane credentials — put them in `labs/.env`, never in Git.

Reading responses:

- NETCONF success: `<rpc-reply>` containing `<data>` (for gets) or `<ok/>` (for edits). Errors: `<rpc-error>` with `error-tag` (`invalid-value`, `data-missing`, `access-denied`).
- RESTCONF success: 200 (GET), 201 (created), 204 (no content). Errors: 400/404/409/415 with a YANG error JSON body. Wrong `Accept` header often yields 406/415.

Cisco-specific notes:

- Enable `netconf-yang` and `restconf` on IOS XE.

- Native model `Cisco-IOS-XE-native:native` is a large tree (hostname, interfaces, ip, etc.). IETF models are smaller and more portable.
- Filters matter. A full `<get-config>` of `running` can be huge; subtree filters (as in `netconf_get_config.py`) request one interface.

3. Syntax and examples

YANG excerpt (from `labs/06_yang_netconf_restconf/sample.yang`):

```
container interfaces {
  list interface {
    key "name";
    leaf name { type string; }
    leaf description { type string; }
    leaf enabled { type boolean; default "true"; }
  }
}
container interfaces-state {
  config false;
  list interface {
    key "name";
    leaf oper-status { type enumeration { enum up; enum down; enum testing; } }
  }
}
```

`interfaces` is configuration. `interfaces-state` is operational (`config false`). RESTCONF would expose them as different data resources; NETCONF `<get-config>` would **not** return `oper-status` .

NETCONF `<get-config>` (client sends this RPC):

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <get-config>
    <source><running/></source>
    <filter>
      <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
        <interface>
          <name>GigabitEthernet1</name>
        </interface>
      </interfaces>
    </filter>
  </get-config>
</rpc>
```

Typical reply (see `labs/06_yang_netconf_restconf/sample_netconf_get-config.xml`):

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <data>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
      <interface>
        <name>GigabitEthernet1</name>
        <description>MANAGEMENT</description>
        <enabled>true</enabled>
      </interface>
    </interfaces>
  </data>
</rpc-reply>
```

Python NETCONF (`ncclient` , from the lab):

```
from ncclient import manager

with manager.connect(
    host=host, port=830, username=user, password=password,
    hostkey_verify=False, device_params={"name": "iosxe"},
    allow_agent=False, look_for_keys=False,
) as m:
    reply = m.get_config(source="running", filter=FILTER)
    print(reply.xml)
```

RESTCONF GET and JSON body (see `sample_restconf_get.json`):

```
GET /restconf/data/ietf-interfaces:interfaces HTTP/1.1
Host: ios-xe.example:443
Accept: application/yang-data+json
Authorization: Basic ...
```

```
{
  "ietf-interfaces:interfaces": {
    "interface": [
      {
        "name": "GigabitEthernet1",
        "description": "MANAGEMENT",
        "enabled": true
      }
    ]
  }
}
```

RESTCONF PATCH (change description — original example):

```
PATCH /restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1
Content-Type: application/yang-data+json

{"ietf-interfaces:interface": {"description": "UPLINK-TO-CORE"}}
```

Python RESTCONF (from the lab):

```
url = f"https://{HOST}:{PORT}/restconf/data/ietf-interfaces:interfaces"
r = requests.get(
    url, auth=(USER, PASSWORD),
    headers={"Accept": "application/yang-data+json"},
    verify=False, timeout=30,
)
print(r.status_code, r.json())
```

4. Exam-style understanding

Keep three layers separate: **model (YANG)**, **protocol (NETCONF vs RESTCONF)**, **encoding (XML vs JSON)**.

If you see...	It is...
leaf, list, key, config false	YANG
Port 830, <get-config>, <rpc-reply>	NETCONF
/restconf/data/, application/yang-data+json, GET/PATCH	RESTCONF
source>running	Configuration datastore read
config false / interfaces-state	Operational data; use <get> or RESTCONF oper path, not <get-config>

Traps:

- Saying YANG is a protocol. It is not.
- Using JSON on NETCONF (standard NETCONF is XML).
- Expecting RESTCONF on port 830.
- Editing operational state.
- Forgetting Accept: application/yang-data+json (IOS XE may reject a generic application/json).
- Confusing **candidate** **commit** with RESTCONF PATCH to running (both can change the box; the locking/commit workflow is a NETCONF candidate feature).

5. Hands-on exercise

1. Read labs/06_yang_netconf_restconf/sample.yang and mark which nodes are config vs operational.
2. Launch IOS XE from <https://devnetsandbox.cisco.com/>. Fill IOSXE_* in labs/.env from **this** reservation.
3. Run restconf_get_interfaces.py. Compare live JSON to sample_restconf_get.json.
4. Run netconf_get_config.py. Compare XML to sample_netconf_get-config.xml. List the first few NETCONF capabilities printed by the script.
5. In Postman or curl, GET Cisco-IOS-XE-native:native/hostname with the yang-data Accept header. Then GET a nonsense path and read the error JSON — that is how you debug 404s on exam day.

3.9 Construct code to perform a specific operation based on a set of requirements and given API reference documentation such as these:

Parent objective: you are given **requirements + API docs** (not a memorized URL list) and you **construct** working code. The three examples on the blueprint are 3.9.a, 3.9.b, and 3.9.c. Treat each as its own construct skill. Always load secrets from the environment (`labs/.env.example`).

3.9.a Obtain a list of network devices by using Meraki, Cisco Catalyst Center, ACI, Cisco Catalyst SD-WAN, or NSO

1. What Cisco expects me to know

Construct a script that authenticates to **one** of these platforms and returns a device inventory. You must follow the platform's auth and resource hierarchy. The exam can specify which platform; the skill is the same: read the reference, obtain a handle (key, token, cookie), GET the device collection, print identifiers (name, serial, management IP).

2. Detailed explanation

Meraki. Header `X-Cisco-Meraki-API-Key`. Base `https://api.meraki.com/api/v1`. Devices can be listed per organization (`GET /organizations/{orgId}/devices`) or per network (`GET /networks/{netId}/devices`). You often list organizations first, pick an id, then list devices. Fields you care about: `name`, `serial`, `model`, `lanIp`, `networkId`.

Catalyst Center. `POST /dna/system/api/v1/auth/token` with Basic auth → JSON `Token`. Then `GET /dna/intent/api/v1/network-device` with `X-Auth-Token`. The list is usually under `response`. Fields: `hostname`, `managementIpAddress`, `softwareType`, `serialNumber`, `family`.

ACI. `POST /api/aaaLogin.json` → token cookie. Devices in the fabric are `fabricNode` (leaves, spines, controllers). `GET /api/class/fabricNode.json`. Parse `imdata` → `fabricNode` attributes (`name`, `role`, `model`). This is not a campus switch inventory; it is fabric nodes.

Catalyst SD-WAN. Authenticate to vManage (form login or token per version). Then a device list under the dataservice API (commonly `GET /dataservice/device`). Fields include `hostname`, `system-ip`, `reachability`, `device-type`. Use the doc for **your** vManage version; path names have shifted across releases.

NSO. Devices are entries in the NSO device tree, not “discovered via CDP” unless you built that. RESTCONF: `GET /restconf/data/tailf-ncs:devices/device` (or REST `/api/running/devices`). You get NSO's view of managed devices (address, authgroup, ned-id, oper-state).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant Script
    participant API
    Script->>API: Authenticate (key / token / cookie)
    API->>Script: Session
    Script->>API: GET device collection
    API->>Script: JSON list
    Script->>Script: Print name / IP / serial
```

3. Syntax and examples

Meraki — `labs/05_cisco_apis/meraki_list_devices.py`:

```
orgs = get("/organizations")
org_id = orgs[0]["id"]
devices = get(f"/organizations/{org_id}/devices")
for d in devices:
    print(d.get("name"), d.get("model"), d.get("serial"), d.get("lanIp"))
```

Catalyst Center — `labs/05_cisco_apis/catalyst_center_devices.py`:

```
tok = token() # POST /dna/system/api/v1/auth/token
payload = get("/dna/intent/api/v1/network-device", tok)
for d in payload.get("response", []):
    print(d.get("hostname"), d.get("managementIpAddress"))
```

ACI (original example):

```
s = requests.Session()
s.verify = False
login = s.post(
    f"https://{apic}/api/aaaLogin.json",
    json={"aaaUser": {"attributes": {"name": user, "pwd": password}}},
    timeout=30,
)
login.raise_for_status()
nodes = s.get(f"https://{apic}/api/class/fabricNode.json", timeout=30)
for item in nodes.json().get("imdata", []):
    attr = item["fabricNode"]["attributes"]
    print(attr.get("name"), attr.get("role"), attr.get("model"))
```

SD-WAN / NSO: follow the sandbox API doc for the exact GET. The construct pattern does not change: session first, then list, then iterate.

4. Exam-style understanding

Given a doc page, you should be able to fill in:

1. Base URL and auth header/body
2. The GET path for devices
3. The JSON key that holds the array (`response` , `imdata` , top-level list)
4. Two identifying fields to print

If the requirement says “Meraki” and an answer uses `X-Auth-Token` , it is wrong. If it says “Catalyst Center” and the path has no `/dna/` , be suspicious. If it says “ACI” and the script never calls `aaaLogin` , it will 403.

5. Hands-on exercise

1. Copy `labs/.env.example` to `labs/.env` .
2. Run `python labs/05_cisco_apis/meraki_list_devices.py` with a Dashboard key.
3. Reserve Catalyst Center; run `python labs/05_cisco_apis/catalyst_center_devices.py` .
4. Optional: ACI or SD-WAN sandbox — list nodes/devices once in Postman, then reproduce with 20 lines of Python.

3.9.b Manage spaces, participants, and messages in Webex

1. What Cisco expects me to know

Construct code that uses the Webex REST API (or SDK) to **create/list/delete spaces (rooms)**, **add/list participants (memberships)**, and **create/list messages**. Auth is a Bearer token. Docs:

<https://developer.webex.com/docs/getting-started>. Lab: `labs/05_cisco_apis/webex_rooms.py` .

2. Detailed explanation

Webex UI **spaces** = API **rooms**. You cannot post a message without a `roomId` (or a person id for 1:1). You cannot add a participant without `roomId` plus `personEmail` or `personId` .

REST resources:

Resource	Path	Typical methods
Identity	<code>/people/me</code>	GET
Spaces	<code>/rooms</code>	POST create, GET list, GET by id, PUT title, DELETE
Participants	<code>/memberships</code>	POST add, GET list by roomId, DELETE membership id
Messages	<code>/messages</code>	POST text/markdown/files, GET by roomId, DELETE

Pagination uses `items` arrays. Always check `status_code` (201 create, 204 delete, 400 missing roomId, 401 bad token, 403 bot not in space).

Bots cannot add people to spaces in all the same ways a human token can; if a membership POST fails, that is often a **scope / bot limitation**, not a wrong URL. Personal access tokens from the getting-started page are the easiest for this lab.

3. Syntax and examples

From the lab:

```
me = api("GET", "/people/me")
room = api("POST", "/rooms", json={"title": "CCNAAUTO lab space"})
room_id = room["id"]
api("POST", "/messages", json={"roomId": room_id, "text": "Hello from the 200-901 lab"})
messages = api("GET", "/messages", params={"roomId": room_id})
memberships = api("GET", "/memberships", params={"roomId": room_id})
api("POST", "/memberships", json={"roomId": room_id, "personEmail": "colleague@example.com"})
api("DELETE", f"/rooms/{room_id}")
```

Headers on every call:

```
Authorization: Bearer <WEBEX_TOKEN>
Content-Type: application/json
```

Base URL: `https://webexapis.com/v1` (settable via `WEBEX_BASE` in `.env`).

4. Exam-style understanding

Order of operations is a scoring item: create room → capture `id` → post message with that `roomId` → list memberships. Posting a message with only `text` and no `roomId` / `toPersonEmail` is invalid.

Markdown vs text: `"markdown": "***down**"` vs `"text": "down"`. Files use a `files` URL array. You do not need file uploads for the core objective.

If a question uses the SDK, `api.rooms.create(title= ...)` then `api.messages.create(roomId= ... , text= ...)` is the same flow.

5. Hands-on exercise

1. Create a bot or personal token at <https://developer.webex.com/docs/getting-started>.
 2. Set `WEBEX_TOKEN` in `labs/.env`.
 3. Run `python labs/05_cisco_apis/webex_rooms.py`.
 4. Extend the script: POST a membership for a second email you control, GET memberships, then DELETE the membership id before deleting the room.
-

3.9.c Obtain a list of clients / hosts seen on a network using Meraki or Cisco Catalyst Center

1. What Cisco expects me to know

Construct a script that lists **clients/hosts** (end users, phones, printers — not the network devices of 3.9.a) from **Meraki** or **Catalyst Center**. You must use the correct resource: Meraki network clients vs Catalyst Center client/health APIs.

2. Detailed explanation

A **device** in 3.9.a is infrastructure (switch, AP, gateway). A **client/host** in 3.9.c is something **seen on** the network: a MAC that associated to an SSID or learned on a switch.

Meraki. After you have a `networkId`:

```
GET /networks/{networkId}/clients?timespan=86400
X-Cisco-Meraki-API-Key: ...
```

`timespan` is seconds of history. Useful fields: `mac`, `ip`, `description`, `ssid`, `vlan`, `status`. Organization-wide client search also exists, but the exam-friendly path is **network** → **clients**. The lab `meraki_list_devices.py` already does this after listing networks.

Catalyst Center. Client data is **assurance** data, not the network-device inventory. Paths vary by version; common ones:

- `GET /dna/intent/api/v1/client-health`
- `GET /dna/data/api/v1/clients` (newer data APIs)

The lab tries both and prints payload keys if one 404s. You still use the same **token** as 3.9.a. Client objects include MAC, IP, hostname, connection status, and health score.

Do not use ISE ERS here unless the question explicitly moves to identity; 3.9.c names Meraki or Catalyst Center.

3. Syntax and examples

Meraki (from the lab):

```
networks = get(f"/organizations/{org_id}/networks")
net_id = networks[0]["id"]
clients = get(f"/networks/{net_id}/clients", params={"timespan": 86400})
for c in clients:
    print(c.get("description") or c.get("mac"), c.get("ip"), c.get("ssid"))
```

Catalyst Center (from the lab):

```

tok = token()
for path in (
    "/dna/intent/api/v1/client-health",
    "/dna/data/api/v1/clients",
):
    payload = get(path, tok)

```

When reading Catalyst Center JSON, walk `response` lists; do not assume the Meraki-style top-level array.

4. Exam-style understanding

Inventory	Platform call
APs, switches, MX	Meraki <code>/devices</code> or Catalyst <code>/network-device</code>
Laptops/phones seen	Meraki <code>/clients</code> or Catalyst <code>client-health/clients</code>

If you print switch hostnames for a “clients seen” requirement, you answered 3.9.a by mistake. `timespan` is a Meraki query parameter; Catalyst Center uses its own filters/time windows in assurance APIs.

5. Hands-on exercise

1. Run `labs/05_cisco_apis/meraki_list_devices.py` and confirm the **Clients seen** section prints MACs/IPs.
 2. On Catalyst Center sandbox, run `catalyst_center_devices.py` and note which client path returned 200.
 3. Write a four-line comment at the top of each script mapping it to 3.9.a vs 3.9.c (the labs already do this in the module docstring).
-

4.0 Application Deployment and Security — 15%

Domain 4 is how software **gets from a laptop to a place users can hit it**, and how you keep that software from leaking secrets or getting pwned. CCNA Automation is not CCIE Data Center and not a full OSCP. You need working vocabulary for **edge vs cloud, VMs vs bare metal vs containers, CI/CD, Docker, unit tests, TLS and secrets, firewalls/DNS/load balancers/reverse proxies, OWASP Top 10, Bash, and DevOps (CAMS/CALMS)**.

Local labs: `labs/07_docker/` (Dockerfile + `app.py`), `labs/01_python_basics/test_subnet.py`, and Bash on WSL Ubuntu as described in `CCNAAUTO_LAB_SETUP.md`.

4.1 Describe the benefits of edge computing

1. What Cisco expects me to know

Describe why an application (or part of it) would run **at the edge** — near users, devices, or branch sites — instead of only in a centralized cloud region. Benefits, not a full MEC architecture course.

2. Detailed explanation

Edge computing places compute and often storage **close to where data is produced or consumed**: a factory PLC network, a retail store, a cell tower site, a campus IDF, a vehicle. The opposite pattern is “send every sensor sample to `us-east-1` and wait.”

Benefits you should be able to explain:

- **Latency.** A robot control loop or a Wi-Fi location API cannot wait 80 ms plus jitter to a distant region. Edge keeps the round trip on the LAN or a nearby PoP.
- **Bandwidth and cost.** Cameras at 4K do not need to backhaul 24/7. Infer “person detected” at the edge; send events, not raw video.
- **Autonomy / resilience.** A store must still scan inventory if the WAN to the cloud dies. Edge apps keep a local control loop.
- **Data locality and compliance.** Some industries cannot export raw PII or plant data. Process locally; send aggregates.
- **Real-time decisions.** QoS, SD-WAN path selection, and radio scheduling are edge problems; they are not batch ETL.

Cisco-relevant picture: SD-WAN edges, industrial switches, UCS/Intersight at a remote site, and apps in a local Kubernetes cluster or even a container on a Catalyst device (beyond exam depth) are all “compute near the network edge.” Domain 3’s controllers may still sit in a DC; **the workload** sits near the user.

Edge is **not** the same as CDN-only (caches of static files), though CDNs are one edge pattern. Edge computing includes **running your logic**, not only caching JPEGs.

3. Syntax and examples

There is no required CLI. A design sketch you can draw:

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    Sensors --> EdgeApp[Edge app: filter / infer]
    EdgeApp -->|events only| Cloud[Central cloud: store / train]
    EdgeApp --> Actuators
    Users --> EdgeApp
```

Example: a Python service on a store NUC reads Meraki camera motion metadata locally and POSTs a Webex message when a loading dock is blocked. The video stays on-site; the cloud sees a JSON event.

4. Exam-style understanding

If the scenario stresses **milliseconds**, **WAN failure**, **too much video/IoT volume**, or **data that must not leave the site**, the benefit is edge computing. If the scenario is “elastic capacity for a global web app with no latency sensitivity,” public cloud centralization may be the better attribute (4.2) — do not force “edge” into every answer.

5. Hands-on exercise

1. Write five bullets: latency, bandwidth, autonomy, locality, real-time. Add one original example each (warehouse scanner, stadium Wi-Fi analytics, clinic imaging, mine safety sensors, branch SD-WAN).
2. Optional: run `labs/07_docker/app.py` locally and then in Docker — that is still “central laptop,” but it sets up 4.3.c. Mentally place the same container on a branch host as an edge deploy.

4.2 Describe the attributes of different application deployment models (private cloud, public cloud, hybrid cloud, and edge)

1. What Cisco expects me to know

Describe four **where-do-we-run-it** models: private cloud, public cloud, hybrid cloud, and edge. Attributes: who owns the hardware, tenancy, connectivity, elasticity, and operational responsibility.

2. Detailed explanation

Public cloud. A provider (AWS, Azure, GCP, Cisco cloud services) offers compute/storage/network as a service over the Internet. **Multi-tenant** hardware you do not unbox. Attributes: rapid elasticity, opex billing, global regions, you manage the app (and sometimes the OS) but not the data center. APIs and IAM are first-class. Tradeoff: data gravity, egress cost, shared-responsibility security, less physical control.

Private cloud. Cloud-style APIs and pooling **on infrastructure your organization controls** (on-prem UCS + OpenStack/VMware/Nutanix, or a dedicated hosted suite). Attributes: single-tenant (from your perspective), stronger physical control and sometimes easier compliance, you still pay for idle capacity. “Private” does not mean “no APIs”; a private cloud that still requires a ticket to launch a VM is just a virtualized DC.

Hybrid cloud. **Two or more** environments (typically private + public) **orchestrated together**: burst to public, backup to public, identity spanning both, or an app with a stateful database on-prem and a stateless front end in public cloud. Connectivity (VPN, SD-WAN, Direct Connect/ExpressRoute) and **consistent identity/policy** are the hard attributes. Hybrid is not “we have a VM in both places and email the files.”

Edge. Covered in 4.1 as a **placement** model: compute near users/devices. Attributes: smaller failure domain, limited hardware, often intermittent WAN, need for GitOps/containers that can run disconnected. Edge can be part of hybrid (central Kubernetes + site clusters).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

graph TD
    subgraph public [Public cloud]
        R1[Region A]
        R2[Region B]
    end
    subgraph private [Private cloud]
        DC[On-prem UCS / VMware]
    end
    subgraph edge [Edge]
        Store[Branch / plant]
    end
    DC <--> |hybrid connectivity| R1
    Store --> DC
    Store --> R1

```

3. Syntax and examples

Attribute table:

Attribute	Public	Private	Hybrid	Edge
Hardware owner	Provider	You (or dedicated hoster)	Mix	You / telco / colocation
Elasticity	High	Limited by purchased gear	Burst to public	Low–medium
Latency to local users	Region-dependent	Low if users are in that DC	Depends	Lowest for local users
Typical API	Cloud provider	Private cloud API / vCenter / Intersight	Both + network	Device/local k8s / IoT

Example: a university runs Canvas integrations in Azure (public), student-record DBs in the campus private cloud, and a digital-signage player in each building (edge). The integration layer is hybrid.

4. Exam-style understanding

Do not confuse **deployment model** (4.2: public/private/hybrid/edge) with **deployment type** (4.3: VM / bare metal / container). You can run containers in public cloud, VMs in private cloud, and bare metal at the edge.

“We keep PHI on-prem and use AWS for the website” is **hybrid**. “We only use an on-prem OpenStack with self-service VMs” is **private cloud**. “Lambda in AWS only” is **public**. “Inference on a factory server” is **edge**.

5. Hands-on exercise

1. Classify four apps you already use (Webex, a home lab VM, a bank’s mobile app, a store POS) into the four models. Justify with one attribute each.
2. Read your `labs/.env.example`: DevNet Sandbox is **publicly hosted lab infrastructure** (not your private cloud). Your Docker Desktop is **local**, which is closer to edge/laptop than to public cloud.

4.3 Describe the attributes of these application deployment types

This objective splits into virtual machines, bare metal, and containers. Each subtype below has the full five-section treatment.

4.3.a Virtual machines

1. What Cisco expects me to know

Describe what a VM is, what it includes (virtual hardware + guest OS + app), and attributes: isolation, density, live migration, slower start than containers, larger images.

2. Detailed explanation

A **virtual machine** is a full computer emulated by a **hypervisor** (ESXi, Hyper-V, KVM). The guest believes it has CPU, RAM, disk, and NICs. You install a **guest operating system** (Windows, Ubuntu) then your application. Isolation is strong (separate kernels) compared to containers that share a host kernel.

Attributes:

- **Strong isolation** and mixed OS on one host (Windows + Linux guests).
- **Heavier**: minutes to boot, gigabytes per guest OS.
- **Mature networking**: vSwitches, port groups, overlay — maps cleanly to CCNA switching ideas.
- **Snapshots, clones, templates, live migration** (vMotion-style) for operations.

- **Inefficient** if you only needed to run one Python process — you paid for a whole kernel.

Cisco compute (UCS + Intersight, section 3.3) is often the hardware under the hypervisor.

3. Syntax and examples

You do not write a VM on the exam in Python. You should recognize a **guest** vs **host**:

```
Physical server (UCS blade)
├─ Hypervisor
│   ├── VM: Ubuntu + nginx + app
│   └── VM: Windows + AD lab
```

Cloud analog: an AWS EC2 instance **is** a VM from the exam's point of view.

4. Exam-style understanding

If the requirement is “run Windows and Linux on the same server” or “live-migrate a full OS,” the type is **VM**. If the requirement is “pack 200 microservices with second-scale start,” containers win. If the requirement is “maximum I/O, no hypervisor tax,” bare metal wins.

5. Hands-on exercise

1. If Hyper-V or VirtualBox is available, note vCPU/RAM of one VM versus the host. If not, use Docker only and still write the isolation comparison in your notes.
2. Relate: DevNet Sandbox topologies are usually VMs behind the portal — that is why they take minutes to reserve.

4.3.b Bare metal

1. What Cisco expects me to know

Describe running the application (or OS) **directly on physical servers** with no hypervisor (or with the app as the main OS workload). Attributes: performance, dedicated hardware, slower provisioning, weaker multi-tenancy.

2. Detailed explanation

Bare metal means the operating system runs on **physical** CPU/RAM/NIC/disk. Databases with huge I/O, real-time systems, and some telecom VNFs (or CNFs on dedicated nodes) still want this. You gain:

- **No hypervisor overhead** and full access to NICs (SR-IOV, DPDK).
- **Predictable performance** (noisy neighbor is you).
- **Simpler failure story** for some appliances (the firewall *is* the box).

You lose:

- **Density and fast clone** of VMs/containers.
- **Easy multi-tenant** packing.
- **Seconds-scale** scale-out (you wait for PXE/iLO/Intersight to image a server).

UCS service profiles and Intersight firmware jobs exist **because** bare metal still needs automation — you just automate the physical layer instead of `kubectl scale`.

3. Syntax and examples

```
UCS blade + service profile
├─ Ubuntu installed on local disk
└─ PostgreSQL (no hypervisor)
```

Contrast with the same blade running ESXi plus a PostgreSQL VM.

4. Exam-style understanding

Keywords: **physical, no hypervisor, dedicated, highest performance, slowest to provision**. A container on a laptop is not bare metal. A VM in AWS is not bare metal (unless the product is literally a bare-metal instance class — still “provider hardware,” but the exam’s 4.3.b is the **type**, not the 4.2 model).

5. Hands-on exercise

1. List three workloads you would keep bare metal (high-frequency trading NIC, old vendor appliance, GPU training box) and three you would not (stateless API, CI runner, student lab OS).
2. Skim UCS/Intersight from 3.3 and note one API that images or inventories **physical** servers.

4.3.c Containers

1. What Cisco expects me to know

Describe containers as **isolated processes** sharing the host kernel, packaged with a root filesystem (image).

Attributes: fast start, high density, portable images, weaker isolation than VMs, perfect fit for CI/CD and microservices. Docker details are 4.6 and 4.7; here you need the **type** versus VM/bare metal.

2. Detailed explanation

A **container** packages an application plus its libraries into an **image**. A **runtime** (Docker, containerd) starts a process with namespaces (PID, net, mnt) and cgroups (CPU/RAM). There is **no guest kernel**. That is why a container starts in seconds and why a Linux container will not run a Windows kernel inside it on a Linux host.

Attributes:

- **Portable:** `docker run` the same image on a laptop, a VM, or a Kubernetes node.
- **Layered images** (4.6) make CI rebuilds fast.
- **Weaker isolation** than VMs: kernel exploits are a class of risk; still far better than “all processes in one OS user space with no namespaces.”
- **Networking:** virtual Ethernet, port publish (`-p 8080:8080`), overlay networks in Swarm/Kubernetes.
- **Ephemeral filesystem** by default; durable data needs volumes.

Containers run **on** VMs or **on** bare metal. Kubernetes in public cloud is “containers (4.3.c) on VMs (4.3.a) in public cloud (4.2).”

3. Syntax and examples

Lab app: `labs/07_docker/app.py` inside `labs/07_docker/Dockerfile` . Conceptually:

```
Host Linux kernel
├─ Container A: python app.py (port 8080)
└─ Container B: nginx
```

Versus two VMs each with their own kernel.

4. Exam-style understanding

Type	Isolation unit	Typical start	Guest OS
Bare metal	Physical server	Minutes–hours	One OS on iron
VM	Virtual hardware	Seconds–minutes	Yes
Container	Namespaced process	Milliseconds–seconds	No (shares host kernel)

If the question mentions Dockerfile, image, or `docker run` , the type is containers. If it mentions hypervisor and guest OS, VMs.

5. Hands-on exercise

Complete 4.6 and 4.7 hands-on with `labs/07_docker` . After `docker run` , you have felt a container. Write one paragraph contrasting it with a VM you have used.

4.4 Describe components for a CI/CD pipeline in application deployments

1. What Cisco expects me to know

Describe the **pieces** of a CI/CD pipeline: source, build, test, artifact, deploy, and the feedback loop. You are not required to author a production GitHub Actions enterprise platform. You should recognize what each stage is for.

2. Detailed explanation

CI (Continuous Integration) means every change is **merged often** and **automatically built and tested**. The goal is to catch breakage on a small diff, not at the end of a three-month branch.

CD means either **Continuous Delivery** (every good build **can** be released with one click) or **Continuous Deployment** (every good build **is** released automatically). The exam cares that you know the pipeline **automates** build/test/deploy rather than “copy files with FTP on Friday.”

Typical components:

Component	Role
VCS (Git)	Source of truth for code; PRs trigger the pipeline
CI server / runner	GitHub Actions, GitLab CI, Jenkins — executes jobs
Build	Compile, <code>pip install</code> , <code>docker build</code>
Unit tests	Fast tests (objective 4.5)
Artifact repository	Docker registry, PyPI, Nexus — stores versioned outputs
Security scans	Dependency CVE scan, image scan, secret scan (ties to 4.8 / 4.10)
CD / GitOps	Deploy to test then prod (Ansible, Terraform, kubectl, Helm)
Environments	Dev, staging, prod with promotion
Observability	Logs/metrics after deploy so you can roll back

A pipeline is **code** (`workflow.yaml`, Jenkinsfile). The Git lab notes in `labs/04_git/workflow.yaml` remind you that version control and not storing keys in Git are part of the same professional habit.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
  Git[git push] --> CI[CI runner]
  CI --> Build[docker build]
  CI --> Test[python -m unittest]
  Test --> Registry[Image registry]
  Registry --> Staging
  Staging --> Prod
```

3. Syntax and examples

Minimal GitHub Actions-style pipeline (original teaching example):

```

name: ccnaauto-app
on: [push]
jobs:
  test-and-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r labs/requirements.txt
      - run: python -m unittest labs.01_python_basics.test_subnet
      - run: docker build -t ccnaauto-lab:ci labs/07_docker

```

That file is **CI**. Adding a deploy job to a Kubernetes cluster or `docker compose` on a jump host would be **CD**.

4. Exam-style understanding

Match the failure to the missing component: tests never run → no CI test stage; “it worked on my laptop” → no build in a clean runner; prod was updated by hand and drifted → no CD/GitOps; secrets in the YAML → failed 4.8 as well as 4.4.

CI is not only “Jenkins.” Any automated build-on-push is CI. CD is not only Kubernetes.

5. Hands-on exercise

1. Run the unit tests locally (`python -m unittest labs.01_python_basics.test_subnet`) — that is the **test** stage.
2. Run `docker build` (4.7) — that is the **build** stage.
3. Read `labs/04_git/workflow.yaml` checklist and imagine each Git action triggering CI.
4. Optional: add a GitHub Action to a private repo that only runs unittest — no need to deploy.

4.5 Construct a Python unit test

1. What Cisco expects me to know

Construct a unit test using the standard library `unittest` module: subclass `TestCase`, use assertions (`assertEqual`, `assertRaises`, ...), and run with `python -m unittest`. Docs: <https://docs.python.org/3/library/unittest.html>. Lab: `labs/01_python_basics/test_subnet.py` testing `prefix_to_hosts` in `functions_classes_modules.py`.

2. Detailed explanation

A **unit test** verifies a **small unit** (function/class) in isolation. It is not a test that logs into Catalyst Center (that would be integration). Good unit tests are **fast, repeatable, and independent**.

unittest structure:

- `import unittest`
- `class TestSomething(unittest.TestCase):`
- Methods named `test_*`
- Assertions: `self.assertEqual(a, b)`, `self.assertTrue(x)`, `self.assertIn(item, container)`, `self.assertRaises(Error)`
- Optional `setUp` / `tearDown` for fixtures
- `if __name__ == "__main__": unittest.main()`

Discovery: `python -m unittest` finds `test*.py`. You can pass a module path: `python -m unittest labs.01_python_basics.test_subnet`.

TDD (Domain 1 overlap): write the failing test first, then implement. The lab docstring describes that cycle.

Failures print a traceback and `FAIL` vs `ERROR` (assertion vs exception). `OK` means all tests passed.

3. Syntax and examples

From `labs/01_python_basics/test_subnet.py`:

```
import unittest
from functions_classes_modules import prefix_to_hosts

class TestPrefixToHosts(unittest.TestCase):
    def test_slash_24(self):
        self.assertEqual(prefix_to_hosts("192.168.1.0/24"), 254)

    def test_slash_30(self):
        self.assertEqual(prefix_to_hosts("10.0.0.0/30"), 2)

    def test_slash_32(self):
        self.assertEqual(prefix_to_hosts("172.16.0.1/32"), 1)

    def test_invalid_raises(self):
        with self.assertRaises(ValueError):
            prefix_to_hosts("not-an-ip")

if __name__ == "__main__":
    unittest.main()
```

Run:

```
python -m unittest labs.01_python_basics.test_subnet
python -m unittest test_subnet.py
```

Original extra example — testing a tiny helper you might write for REST status handling:

```
def is_success(code: int) → bool:
    return 200 ≤ code < 300

class TestStatus(unittest.TestCase):
    def test_ok(self):
        self.assertTrue(is_success(200))
    def test_created(self):
        self.assertTrue(is_success(201))
    def test_not_found(self):
        self.assertFalse(is_success(404))
```

4. Exam-style understanding

Given a function, you should write a class that calls it and asserts. Common mistakes: naming the method `slash_24` without `test_` (it will not run); using `assert a == b` instead of `self.assertEqual` (works in pytest, not the unittest style the exam names); forgetting `self` as the first argument.

`assertEqual(254, prefix_to_hosts( ... ))` argument order: first is typically expected in some styles; `unittest` prints “expected vs actual” based on `assertEqual(first, second)` — keep expected vs actual consistent with the docs.

5. Hands-on exercise

1. From the repo root (or `labs/01_python_basics`), run `python -m unittest test_subnet.py`.
2. Break `prefix_to_hosts` temporarily and re-run — read a `FAIL`.
3. Add `test_slash_16` expecting `65534` and run until it passes.
4. Keep tests next to code; do not put API keys in tests.

4.6 Interpret contents of a Dockerfile

1. What Cisco expects me to know

Interpret a Dockerfile: what each instruction does, that each instruction creates a **layer**, and that order affects **cache**. Reference: <https://docs.docker.com/reference/dockerfile/>. Lab file: `labs/07_docker/Dockerfile`.

2. Detailed explanation

A **Dockerfile** is a recipe that produces an **image**. The daemon executes instructions **top to bottom**. Unchanged prefix layers are cached on rebuild.

Instructions you must read fluently:

Instruction	Meaning
FROM	Base image; first (or after ARG for FROM)
LABEL	Metadata key=value; not required for the app to run
WORKDIR	cd for later instructions and the default cwd at runtime
COPY / ADD	Copy files into the image. Prefer COPY . ADD can unpack URLs/tar (easy to misuse)
RUN	Execute a command at build time (apt, pip) — becomes a layer
ENV	Environment variable persisted in the image
EXPOSE	Documents a port; does not publish it by itself
CMD	Default process. JSON-exec form ["python", "app.py"] is preferred (no shell)
ENTRYPOINT	Fixed process; CMD then becomes default arguments
USER	Drop root (security)
VOLUME	Mount point for persistent data
ARG	Build-time only variable (not the same as ENV)

Layers: each RUN, COPY, etc. snapshots the filesystem. A change to app.py invalidates COPY app.py and everything **after** it, but not FROM python:3.12-slim if that line did not change. That is why you copy requirements.txt and RUN pip install **before** copying the rest of the source in larger apps.

CMD vs ENTRYPOINT:

- CMD ["python", "app.py"] — replaceable: docker run img bash overrides CMD.
- ENTRYPOINT ["python"] + CMD ["app.py"] — docker run img other.py runs python other.py.

Shell form CMD python app.py runs via /bin/sh -c, which breaks some signals. Prefer exec form.

EXPOSE 8080 plus ENV PORT=8080 does not open a Windows firewall. You still docker run -p 8080:8080.

3. Syntax and examples

Lab Dockerfile:

```
FROM python:3.12-slim
LABEL exam.objective="4.6 Interpret contents of a Dockerfile"
WORKDIR /app
COPY app.py .
ENV PORT=8080
ENV APP_ENV=lab
EXPOSE 8080
CMD ["python", "app.py"]
```

Interpretation, line by line:

1. Start from a slim Python 3.12 image.
2. Metadata only.
3. Subsequent paths and the container cwd are `/app`.
4. Copy `app.py` from the build context into `/app/app.py`. 5–6. Set `PORT` and `APP_ENV` inside the container.
5. Document TCP 8080.
6. Default process: `python app.py` with no shell.

A **worse** Dockerfile for teaching cache (do not copy this into prod blindly):

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Any source change busts the pip layer. Better: copy requirements first, `RUN pip`, then copy the app.

4. Exam-style understanding

Given a Dockerfile, answer: what is the base? what is the working directory? what command starts? which port is documented? what env vars exist?

Traps:

- Believing `EXPOSE` publishes the port to the host.
- Confusing `RUN` (build) with `CMD` (runtime).
- Thinking `COPY . .` copies from the container; it copies from the **build context** (usually the directory of the Dockerfile).
- `FROM ubuntu` then `CMD python` without installing Python — the image would fail at runtime.

5. Hands-on exercise

1. Read `labs/07_docker/Dockerfile` and `app.py` side by side. Confirm `PORT` is read by the app.
 2. Add a comment above each instruction in your notes (not necessarily in the file) stating “build-time” vs “run-time.”
 3. Open <https://docs.docker.com/reference/dockerfile/> and read `FROM`, `COPY`, `RUN`, `CMD`, `ENTRYPOINT`, `ENV`, `EXPOSE`.
 4. Continue into 4.7 to build and run it.
-

4.7 Utilize Docker images in local developer environment

1. What Cisco expects me to know

Utilize means you can **build, list, run, stop, and remove** images/containers on your laptop (Docker Desktop on Windows is fine). You map ports, pass env vars, and inspect logs. This is the practical companion to 4.6.

2. Detailed explanation

Image = immutable template. **Container** = running (or stopped) instance of an image. You can run many containers from one image.

Everyday workflow:

1. `docker build -t name:tag dir` — create an image from a Dockerfile
2. `docker images` — list images
3. `docker run --rm -p host:container name:tag` — start a container
4. `docker ps` — running containers (`-a` includes stopped)
5. `docker logs <id>` — stdout of the app
6. `docker stop` / `docker rm` — lifecycle
7. `docker rmi` — delete an image

`--rm` deletes the container on exit (good for labs). `-d` runs detached. `-e PORT=9090` overrides `ENV`. `-p 8080:8080` binds host port 8080 to container 8080. Without `-p`, `EXPOSE` alone does not make <http://localhost:8080> work.

Pulling: `docker pull python:3.12-slim` fetches from Docker Hub (needs network). The lab `FROM` line does this implicitly on first build.

File sharing: the build **context** is the path you pass to `docker build`. Secrets in that directory can be copied into the image if you `COPY . .` — another 4.8 issue.

3. Syntax and examples

From the repo, with Docker Desktop running:

```
cd labs/07_docker
docker build -t ccnaauto-lab:local .
docker images
docker run --rm -p 8080:8080 ccnaauto-lab:local
```

In another terminal:

```
curl http://127.0.0.1:8080
```

You should see a line containing `CCNAAUTO docker lab` (from `app.py`).

Useful extras:

```
docker run --rm -e PORT=9090 -p 9090:9090 ccnaauto-lab:local
docker ps
docker stop <container>
docker logs <container>
docker rm <container>
docker rmi ccnaauto-lab:local
```

Run an interactive shell **in** an image when debugging:

```
docker run --rm -it ccnaauto-lab:local bash
```

(If `bash` is missing in `python:3.12-slim`, use `sh`.)

4. Exam-style understanding

Match command to intent: build vs run vs list vs delete. Know that **tag** `ccnaauto-lab:local` is name:tag. Know `-p` is required for localhost access. Know the difference between deleting a container (`rm`) and an image (`rmi`) — you cannot `rmi` while a container still references the image.

5. Hands-on exercise

1. Install Docker Desktop if needed (`CCNAUTO_LAB_SETUP.md`).
2. Build and run `labs/07_docker` as above. Hit the URL from a browser or `curl` .
3. Change the greeting in `app.py` , rebuild, rerun. Notice that only layers after `COPY` rebuild.
4. Run `docker images` and identify `python:3.12-slim` (base) vs `ccnaauto-lab:local` (yours).

4.8 Describe application security issues related to secret protection, encryption (storage and transport), and data handling

1. What Cisco expects me to know

Describe how applications fail when they mishandle **secrets**, **encryption in transit and at rest**, and **data** (PII, logs, backups). This is professional practice for every script in this course (`labs/.env.example` exists so you never commit keys).

2. Detailed explanation

Secret protection. API keys, tokens, passwords, private keys, and certificates must not live in Git, Docker layers, screenshots, or chat logs. Patterns:

- **Environment variables** (`MERAKI_API_KEY` in `.env` , loaded by `python-dotenv`) for local labs.
- **Secret stores / vaults** (HashiCorp Vault, cloud secret managers, CI secret variables) for real pipelines.

- **Short-lived tokens** (OAuth, Catalyst Center token that expires) beat immortal passwords in source.
- `.gitignore` the `.env` file; commit only `.env.example` with empty values.
- Never `COPY .env` into a Dockerfile.

If a key leaked, **rotate** it (Meraki dashboard, Webex bot, sandbox session). Assume Git history is public once pushed.

Encryption in transit. TLS (HTTPS, WSS, NETCONF over SSH) stops eavesdropping and, with proper cert validation, impersonation. Lab scripts often use `verify=False` because sandboxes have private CAs — that is a **lab compromise**. In production, install the CA or use proper certificates. RESTCONF on 443 and Webex APIs already use TLS; do not “downgrade to HTTP to make it work.”

Encryption at rest. Disks, database volumes, backups, and laptop SSDs should use encryption (LUKS, BitLocker, cloud volume encryption, database TDE). An API key in a stolen unencrypted laptop disk is a credential dump. Encrypting at rest does **not** replace TLS: a stolen backup still needs TLS when restored over the network.

Data handling. Collect the minimum (need-to-know). Do not log full Authorization headers or packet payloads with passwords. Mask PAN/PII. Define retention. If you dump Catalyst Center client lists to a CSV on a shared laptop, that CSV is now an identity file — treat it as sensitive. GDPR/CCPA-style thinking: know where client MACs and emails go.

3. Syntax and examples

Good (lab pattern):

```
import os
from dotenv import load_dotenv
load_dotenv()
API_KEY = os.getenv("MERAKE_API_KEY")
```

Bad:

```
API_KEY = "1234567890abcdef" # committed to Git
```

Transit:

```
requests.get(url, headers=headers, verify=True, timeout=30)
```

At rest (conceptual): database volume encrypted; S3 bucket with encryption enabled; no plaintext key files on the desktop named `passwords.txt`.

Docker anti-pattern:

```
ENV MERAKE_API_KEY=realkey
```

Anyone with the image can `docker history` or `inspect` that layer.

4. Exam-style understanding

Three buckets: **secrets** (how we store credentials), **transport** (TLS), **storage** (disks/backups) **plus data handling** (what we log and keep). A question about `verify=False` is transport integrity. A question about a key in GitHub is secret protection. A question about unencrypted backups of customer client lists is storage + data handling.

5. Hands-on exercise

1. Confirm `labs/.env` is not committed. Use `labs/.env.example` as the template.
2. Search your scripts: no hardcoded tokens.
3. When you run Catalyst Center labs with `verify=False`, write a note: “replace with a CA in production.”
4. Create a fake “incident”: paste a dummy key into a file, then practice rotating a Webex bot token in the developer portal (revoke + new token).

4.9 Explain how firewall, DNS, load balancers, and reverse proxy in application deployment

1. What Cisco expects me to know

Explain how these four functions sit in front of (or beside) an application: what problem each solves, where it sits in a request path, and what breaks if it is misconfigured. Not ASA CLI mastery — deployment architecture.

2. Detailed explanation

DNS. Users type `app.example.com`. DNS resolves it to a VIP or cloud load-balancer name. Attributes: TTL, CNAME vs A, split-horizon (internal vs public). If DNS is wrong, the app can be healthy and still “down.” DNS is also a control point (Umbrella / Secure Connect DNS-layer security from Domain 3). In deployment, you also use DNS for **service discovery** (`db.internal`).

Firewall. Allows or denies flows (ports 443 to the load balancer, 22 only from a jump net, no direct 5432 from the Internet). Stateful inspection, application awareness (NGFW / Firepower). In app deploy, the firewall **protects north-south** (Internet → app) and **east-west** (app → database). Your Docker `-p 8080:8080` on a laptop bypasses the corporate firewall — do not confuse them.

Load balancer. Distributes TCP/HTTP connections across **several app instances** (containers/VMs). Health checks remove sick backends. SSL/TLS can terminate on the LB (users hit HTTPS; backends may be HTTP on a private net — understand the trust boundary). Algorithms: round-robin, least connections, session persistence (cookies). Without an LB, you have a single point of failure and no horizontal scale.

Reverse proxy. A proxy that **receives client requests** destined for your app (the opposite of a forward proxy that the client configures). nginx, Traefik, Apache, and many LBs act as reverse proxies: TLS termination, URL routing (`/api` to API containers, `/` to static), header injection, buffering, WebSocket upgrade. A reverse proxy is often

the same box as an HTTP load balancer. Exam distinction: **load balancer** stresses distribution and health; **reverse proxy** stresses sitting in front, routing, and hiding internal topology.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant User
    participant DNS
    participant FW as Firewall
    participant LB as Load balancer / reverse proxy
    participant App as App containers
    User->>DNS: app.example.com
    DNS->>User: 203.0.113.10
    User->>FW: TCP 443
    FW->>LB: allowed
    LB->>App: HTTP 8080 to healthy instance
    App->>User: 200 OK
```

3. Syntax and examples

nginx reverse-proxy sketch (original):

```
server {
    listen 443 ssl;
    server_name app.example.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

The Docker lab app listens on 8080; nginx would be the public 443 front.

Firewall policy in words: allow any → lb_vip port 443; allow lb → app_subnet port 8080; deny any → app_subnet port 8080 from the Internet.

DNS: app.example.com CNAME lb-prod.example.com.

4. Exam-style understanding

Symptom	Likely layer
Name does not resolve	DNS
SYN dropped, timeout	Firewall
One backend overloaded, others idle	Load balancer / persistence
Host header or path routing wrong; internal ports hidden	Reverse proxy
Certificate name mismatch	TLS on LB/proxy (also 4.8)

A reverse proxy **can** load-balance; a layer-4 firewall **does not** rewrite `/api` to a container. Keep the primary job of each term clear.

5. Hands-on exercise

1. Run the Docker app on 8080 (4.7). That is the **origin**.
2. Optionally install nginx in WSL or use a cloud sandbox and proxy to 8080 — or skip install and still draw the mermaid path on paper.
3. `nslookup` / `dig` a public site and identify A/CNAME. `dig` is Bash (4.11).
4. List which ports you would open on a firewall for: DNS (53), HTTPS (443), SSH jump (22 from management), database (never from Internet).

4.10 Describe top OWASP threats (such as XSS, SQL injections, and CSRF)

1. What Cisco expects me to know

Describe high-level **OWASP Top 10** application risks, with working understanding of **XSS**, **SQL injection**, and **CSRF** as named on the blueprint. Catalog: <https://owasp.org/www-project-top-ten/>. You explain the **mechanism and the mitigation**, not exploit tooling.

2. Detailed explanation

OWASP Top 10 is a consensus list of the most critical web app risks. The exact ten **reorder between versions** (2017 vs 2021 vs later). For this exam, know the **idea** of the list and **deep-dive the three named threats**. Other Top 10 themes you should recognize in passing: broken access control, cryptographic failures (ties to 4.8), injection in general, insecure design, misconfiguration, vulnerable components, auth failures, integrity failures, logging/monitoring failures, SSRF.

XSS (Cross-Site Scripting). The application **reflects or stores attacker-controlled text as HTML/JS** in another user's browser. The victim's browser trusts the origin (`app.example.com`) and runs the attacker's script: session theft, fake login forms, calls to your API as the user.

- **Reflected XSS:** payload in the URL/query, bounced in the response (`?q=<script> ...`).
- **Stored XSS:** payload saved (comment, Webex-like message rendered as HTML without sanitizing).
- **DOM XSS:** client-side JavaScript unsafely writes `location.hash` into the DOM.

Mitigations: **encode/escape** output (HTML entity encode), use templating that auto-escapes, **Content-Security-Policy**, avoid `innerHTML` with user data, HTTPOnly cookies.

SQL injection (SQLi). User input is concatenated into a SQL string so the attacker changes the query: `' OR 1=1 --` turns an auth check into a tautology, or `UNION SELECT` dumps tables.

Mitigations: **parameterized queries / prepared statements** (never string-format SQL), least-privilege DB accounts, ORM with bind parameters, input validation as defense in depth (not the only control).

CSRF (Cross-Site Request Forgery). The browser **automatically sends cookies** (session) to `bank.example.com` when the user visits `evil.example.com`, which contains `<form action="https://bank.example.com/transfer">`. The bank sees a genuine logged-in request it did not intend.

Mitigations: **anti-CSRF tokens** (synchronizer token in form + session), **SameSite cookies**, prefer `Authorization: Bearer` in SPA clients that are not auto-attached as cookies, re-auth for sensitive actions.

XSS vs CSRF: XSS **runs code in your origin**. CSRF **forges a request** using existing credentials without injecting script into your pages (though XSS can also defeat CSRF tokens).

3. Syntax and examples

Vulnerable SQLi pattern (do not use):

```
# BAD - string concatenation
sql = f"SELECT * FROM users WHERE name = '{username}'"
```

Safe pattern:

```
cursor.execute("SELECT * FROM users WHERE name = %s", (username,))
```

XSS — unsafe render:

```
# BAD - user_comment may contain <script>
return f"<p>{user_comment}</p>"
```

Safer:

```
import html
return f"<p>{html.escape(user_comment)}</p>"
```

CSRF — missing token: a POST `/api/v1/organizations/{id}/shutdown` that trusts only a session cookie. Fix: require a CSRF token header or use a Bearer token that `evil.example.com` cannot read (same-origin policy) plus SameSite.

These examples are **defensive**. They show what to write and what to avoid.

4. Exam-style understanding

Threat	One-line	Primary fix
XSS	Attacker script runs in victim browser as your site	Output encoding, CSP
SQLi	Attacker changes database query	Parameterized queries
CSRF	Attacker site triggers victim's authenticated request	CSRF tokens, SameSite

If the payload is in a comment that other users load, think **stored XSS**. If the payload is `' OR '1'='1`, think **SQLi**. If the victim is logged in and visits another site that POSTs to your app, think **CSRF**.

Do not confuse SQLi with XSS: one hits the **database**, the other hits the **browser**. Both are injection-family issues in a broad sense; the exam names them separately.

5. Hands-on exercise

1. Read the current OWASP Top 10 overview at <https://owasp.org/www-project-top-ten/>. Write the ten names once; you need fluency, not memorized sub-bullet 4 of each.
2. In a local Python snippet, practice `html.escape` and a parameterized query as above. Do not attack any live system.
3. Review `webex_rooms.py`: messages are posted as JSON `text`. If a future UI rendered `text` as HTML, that would be an XSS design issue — good mental model for APIs that later grow a dashboard.

4.11 Utilize Bash commands (file management, directory navigation, and environmental variables)

1. What Cisco expects me to know

Utilize Bash for **moving around a filesystem, creating/copying/removing files**, and **setting/reading environment variables**. On Windows, do this in **WSL Ubuntu** so the exam's Linux-shaped questions match your muscle memory (`CCNAUTO_LAB_SETUP.md`).

2. Detailed explanation

Bash is a shell: it reads commands, expands variables, and runs programs. The exam focus is not writing 200-line Bash frameworks. It is:

Directory navigation

- `pwd` — print working directory
- `ls / ls -la` — list (including hidden `.env`)

- `cd /path` — change directory (`cd ~`, `cd -`, `cd ..`)
- `mkdir -p labs/tmp` — create path
- `tree` (if installed) — visualize

File management

- `touch file` — create empty / update mtime
- `cp src dst` / `cp -r dir1 dir2` — copy
- `mv old new` — rename or move
- `rm file` / `rm -r dir` — delete (`-i` interactive; be careful)
- `cat`, `less`, `head`, `tail` — read
- `chmod`, `chown` — permissions (know that scripts need execute bit)
- `find . -name '*.py'` — search
- redirection: `echo hi > file`, `>>` append, `2>` stderr

Environment variables

- `echo $HOME`, `echo $PATH`
- `export WEBEX_TOKEN= ...` — current session (and children)
- `VAR=value command` — one-shot for a child process
- `env` / `printenv` — list
- `unset VAR`
- `.env` files are **not** automatically Bash; `source labs/.env` works only if the file is valid shell syntax (`KEY=value` , no spaces)

`PATH` is the list of directories searched for commands. If `python` is not found, it is a `PATH` problem, not a Python-is-uninstalled problem necessarily.

Windows PowerShell is a different language (`$env:VAR`, `Get-ChildItem`). The objective says **Bash** — practice in WSL.

3. Syntax and examples

```
pwd
cd ~/Documents/04_Learning/CCNA\ Automation
ls -la labs
mkdir -p labs/tmp
cp labs/.env.example labs/tmp/env.study
mv labs/tmp/env.study labs/tmp/env.copy
cat labs/tmp/env.copy | head
rm labs/tmp/env.copy
rmdir labs/tmp

export MERAKI_API_KEY="not-a-real-key"
echo "$MERAKI_API_KEY"
python3 -c 'import os; print("key set:" , bool(os.getenv("MERAKI_API_KEY")))'
unset MERAKI_API_KEY

# run a lab with env from a file that is valid Bash
set -a
source labs/.env    # only if you created it with KEY=value lines
set +a
```

`set -a` exports all variables that get set while sourcing — useful for `.env` files written as shell.

PATH example:

```
echo "$PATH"
which python3
```

4. Exam-style understanding

Given a required outcome, pick the command: “create a directory” → `mkdir`; “see hidden files” → `ls -a`; “make a variable visible to a Python child” → `export`; “rename” → `mv` not `cp`.

Traps: `rm -r` on the wrong path; spaces in paths without quotes; assuming `export` persists after you close the terminal (it does not — use `~/ .bashrc` for persistence, but do **not** put API keys there if the file is synced); mixing PowerShell `$env:F00` into a Bash answer.

5. Hands-on exercise

1. Open WSL Ubuntu. `cd` to this repo. `ls labs/07_docker`.
 2. `export PORT=8080` and `echo $PORT`.
 3. Copy `.env.example` to a **gitignored** `.env` and `source` it if the syntax is valid.
 4. `chmod +x` a tiny `hello.sh` you write with `echo "$USER"` and run `./hello.sh`.
 5. Use `head / tail` on `test_subnet.py`. That is file management plus navigation.
-

4.12 Describe the principles of DevOps practices

1. What Cisco expects me to know

Describe DevOps as **culture and practices**, not as a job title or a single tool. The usual teaching acronym is **CAMS** or **CALMS**: Culture, Automation, Lean, Measurement, Sharing (Lean is the extra L in CALMS). Tie it to CI/CD (4.4), IaC, and collaboration between people who used to be “dev” vs “ops.”

2. Detailed explanation

DevOps is a way of delivering software and infrastructure **together**, with short feedback loops, so that a network change and an application change are not thrown over a wall.

CALMS / CAMS:

- **Culture.** Shared ownership of production. Blameless postmortems. Network engineers review API changes; developers respect change windows and blast radius. Without culture, Jenkins is just a new silo.
- **Automation.** CI/CD, tests (4.5), Docker builds (4.6–4.7), Ansible/Terraform (Domain 5), API-driven Cisco platforms (Domain 3). If a human must SSH and click, it will not scale and it will drift.
- **Lean.** Small batches, limit WIP, remove waste (manual ticket for every VLAN). Continuous delivery of **small** changes is easier to reverse than a quarterly “big bang.”
- **Measurement.** SLIs/SLOs, pipeline duration, change failure rate, MTTR. You cannot improve what you do not measure. For network automation: time from commit to device config, error rate of REST calls, coverage of unit tests.
- **Sharing.** Docs, runbooks, Code Exchange-style samples, inner-source libraries, pairing. DevNet forums (3.7) are a weak analog; inside a company this is the wiki and the Git repo.

DevOps is **not** “no more change control.” It is **faster, safer** change: automated tests, progressive delivery, instant rollback. It is **not** “developers have root on every router.” Least privilege still applies (4.8).

Related terms: **SRE** (error budgets, toil reduction), **GitOps** (desired state in Git), **platform engineering**. The exam’s word is DevOps principles.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    Culture --> Automation
    Automation --> Lean
    Lean --> Measurement
    Measurement --> Sharing
    Sharing --> Culture
```

3. Syntax and examples

There is no single DevOps CLI. Map principles to artifacts you already have:

Principle	In this course
Culture	Study group reviews of scripts before they touch a sandbox
Automation	<code>python -m unittest</code> , <code>docker build</code> , GitHub Actions sketch in 4.4
Lean	Small PRs: one API script, not a 40-file dump
Measurement	Test pass/fail, HTTP status logs in the labs
Sharing	<code>labs/.env.example</code> , Code Exchange, this study guide

Anti-examples: unique snowflake routers with undocumented ACL changes; production credentials in Slack; a three-week CAB for a one-line YANG default.

4. Exam-style understanding

If the question is “what principle is missing when builds are automated but teams still blame each other for outages?” → **culture**. If configs are applied by hand on 200 devices → missing **automation**. If nobody knows whether deploys got faster → missing **measurement**. If one hero owns all Jenkinsfiles and takes vacation → missing **sharing**.

Do not answer “DevOps” with only “Docker.” Containers are a **tool** that helps automation; they are not the definition.

5. Hands-on exercise

1. Write a five-line CALMS note mapping each letter to one file in this repo.
2. Run tests + docker build in one sitting — that is a mini CI loop (automation + measurement).
3. Share (with yourself / a study partner) a PR-style review of `meraki_list_devices.py`: secrets? timeouts? error handling? That is culture + sharing.
4. Revisit 3.7: Code Exchange is industry-scale **sharing**; Sandbox is **automation-friendly infrastructure** for learning.

Domain 3–4 checkpoint

You should now be able to:

- Read SDK docs and construct a client call (3.1).
- Match Meraki, Catalyst Center, ACI, Catalyst SD-WAN, and NSO to a job and an auth style (3.2).
- Distinguish UCS Manager XML domains from Intersight cloud REST (3.3).
- Use Webex rooms/memberships/messages and recognize CUCM AXL vs UDS (3.4, 3.9.b).
- Place XDR, Firepower, Secure Connect, Secure Endpoint, ISE, and Secure Malware Analytics in a SOC/automation story (3.5).

- Contrast IOS XE RESTCONF/NETCONF/Guest Shell with NX-API CLI (3.6).
- Pick the right DevNet resource (3.7).
- Apply YANG models through NETCONF 830/XML and RESTCONF 443/JSON, including datastores and RPCs (3.8).
- Construct device lists and client lists from Meraki or Catalyst Center (3.9.a, 3.9.c).
- Explain edge, cloud models, VM/bare metal/containers, CI/CD, Docker, unittest, secrets/TLS, firewall/DNS/LB/proxy, OWASP XSS/SQLi/CSRF, Bash, and CALMS DevOps (4.1–4.12).

Next study block in this series is Domain 5 (infrastructure as code and network automation tools) when that chapter is present. Until then, keep credentials in `Labs/.env`, launch devices from <https://devnetsandbox.cisco.com/>, and treat every API error body as a teacher.

5.0 Infrastructure and Automation — 20%

This domain is one of the two heaviest weights on **200-901 CCNAAUTO v1.1**. Treat it as a reading-and-reasoning domain more than a typing domain. Cisco wants you to **describe** why model-driven automation exists, **compare** controller-level and device-level management, **describe** simulation/test tools, CI/CD, IaC, and the capabilities of Ansible, Terraform, and Cisco NSO, then **identify** or **interpret** artifacts: Python that calls Cisco APIs, Ansible playbooks, bash scripts, RESTCONF/NETCONF replies, YANG models, unified diffs, and sequence diagrams.

That last cluster of **interpret** verbs is where people lose points. You are not being asked to become a professional NETCONF developer. You are being asked to look at a snippet and say, accurately, *what is happening*. The labs under `labs/06_yang_netconf_restconf/`, `labs/08_ansible/`, `labs/09_terraform/`, `labs/04_git/example.diff`, and `labs/05_cisco_apis/` exist so you can practice that skill on real-looking output.

A useful mental model for the whole domain:

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    Intent["Intent / desired state<br/>playbook, HCL, YANG, Python"] --> Control["Controller or orchestrator<br/>Catalyst Center, APIC, NSO, Ansible, Terraform"]
    Control --> Protocol["Model-driven protocol<br/>NETCONF / RESTCONF / REST"]
    Protocol --> Device["Network device<br/>IOS XE, NX-OS, ACI leaf, Meraki AP"]
    Device --> State["Operational state<br/>parsed by pyATS, APIs, telemetry"]
    State --> Intent
```

If you can place every objective on that diagram, the rest of this chapter is detail.

5.1 Describe the value of model driven programmability for infrastructure automation

1. What Cisco expects me to know

The verb is **Describe**. You must explain *why* model-driven programmability is more valuable for automation than traditional CLI scraping or ad-hoc SNMP. You do not need to author a full IETF YANG module from scratch. You do need the vocabulary: **YANG is the model**; **NETCONF and RESTCONF are protocols** that carry data shaped by that model; the payoff is **structured, validated, machine-readable configuration and state**.

Connect this objective to 5.10 (interpret a query result) and 5.11 (interpret a YANG model). 5.1 is the “why”; 5.10 and 5.11 are the “read this.”

2. Detailed explanation

For decades, network automation meant logging into a box over SSH, sending CLI commands, and parsing the human-oriented text that came back. That works until it does not:

- CLI output is **not a contract**. A new IOS XE release can insert a column, wrap a line, or rename a field. Your regex breaks.
- CLI is **not validated up front** as a data structure. `interface GigabitEthernet1` plus a typo in a nested command may apply half a change.
- CLI is **vendor- and OS-specific**. The same “set a description on an interface” idea is different on IOS XE, NX-OS, IOS XR, and ASA.
- Screen scraping does not give you **transactions**. You cannot easily say “apply this whole change or none of it.”

Model-driven programmability replaces “speak the device’s human CLI” with “speak a schema.” The schema is **YANG** (Yet Another Next Generation), standardized in [RFC 7950](#). A YANG module declares:

- what configuration leaves exist (hostname, interface name, IP address, enabled flag)
- what operational state leaves exist (oper-status, packet counters)
- types, keys, defaults, and nesting
- which nodes are writable (`config true`) versus read-only (`config false`)

Protocols then **encode** instances of that schema:

Layer	Role	Typical encoding	Typical transport
YANG	Data model (the contract)	N/A — this is the schema	N/A
NETCONF	RPC protocol for config/state	XML	SSH, TCP 830
RESTCONF	HTTP mapping of YANG data	JSON or XML (<code>yang-data+json</code> / <code>yang-data+xml</code>)	HTTPS, typically TCP 443
gNMI / telemetry	Streaming of YANG-modeled state	protobuf / JSON	gRPC (beyond blueprint depth, but know it exists)

The **value** for infrastructure automation is not “XML is nicer than CLI.” The value is:

1. **Structure**. A RESTCONF GET of `ietf-interfaces:interfaces` returns objects with named fields, not a banner and a table. Python `response.json()` just works.
2. **Validation**. The device (and your client, if it loads the YANG) can reject a payload that is missing a list key or uses the wrong type *before* it becomes a half-applied config.
3. **Vendor-neutral models**. IETF modules such as `ietf-interfaces` and `ietf-ip` describe the same idea on any compliant device. Native modules (`Cisco-IOS-XE-native`) still exist when you need Cisco-specific knobs. Automation can prefer IETF models for portability and native models for features.

4. **Discoverability.** NETCONF `<hello>` advertises capabilities and YANG modules. RESTCONF exposes `/restconf/data/` and YANG library resources. You can ask the device what it supports instead of guessing CLI.
5. **Separation of config and state.** YANG marks `config false` containers. NETCONF `<get-config>` reads configuration datastores; `<get>` can include operational state. RESTCONF maps this to different resource paths. Automation can **set** desired config and **assert** live state without mixing the two.
6. **Transactional, targetable changes.** NETCONF `edit-config` against `candidate` plus `commit` is an atomic unit. RESTCONF PATCH can change one leaf. You stop concatenating CLI snippets and hoping the pager did not eat a line.
7. **Idempotent automation.** Because you send *desired data* rather than *a sequence of CLI keystrokes*, a second run that sends the same YANG instance does not keep appending garbage. This is the same idea as Ansible/Terraform desired state (5.5, 5.6), applied at the device API.

CLI is not “wrong.” It remains the operator’s interactive tool and the fallback southbound adapter for older devices (Cisco NSO still speaks CLI to many boxes). Model-driven programmability is what you use when **software** is the operator.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph models [YANG models]
        IETF["ietf-interfaces<br/>ietf-ip"]
        Native["Cisco-IOS-XE-native"]
    end
    subgraph protocols [Protocols]
        NC["NETCONF XML RPCs<br/>port 830 / SSH"]
        RC["RESTCONF HTTP<br/>port 443 / TLS"]
    end
    IETF --> NC
    IETF --> RC
    Native --> NC
    Native --> RC
    NC --> Box["IOS XE device"]
    RC --> Box
```

Misconceptions to kill now:

- YANG is not a protocol. You do not “open a YANG session.”
- NETCONF is not REST. It is SSH-based RPCs with XML.
- RESTCONF is not “the Cisco REST API.” It is the IETF HTTP mapping of YANG. Meraki and Catalyst Center APIs are product REST APIs, not RESTCONF.
- Model-driven does not mean “only IETF.” Native Cisco YANG is still model-driven.

3. Syntax and examples

A tiny YANG leaf from the lab excerpt `labs/06_yang_netconf_restconf/sample.yang` :


```
leaf enabled {  
  type boolean;  
  default "true";  
}
```

That single declaration is the contract: the value must be boolean; if omitted, treat it as `true`. A RESTCONF JSON instance that honors it:

```
{  
  "ietf-interfaces:interfaces": {  
    "interface": [  
      {  
        "name": "GigabitEthernet1",  
        "enabled": true  
      }  
    ]  
  }  
}
```

The same data as NETCONF XML (namespace from the module):

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">  
  <interface>  
    <name>GigabitEthernet1</name>  
    <enabled>true</enabled>  
  </interface>  
</interfaces>
```

Contrast the **CLI scrape** you would otherwise write:

```
GigabitEthernet1 is up, line protocol is up  
Description: MANAGEMENT  
Internet address is 10.10.20.48/24
```

That text has no stable schema. The YANG/JSON form is what an Ansible module, a Python `requests` script, or NSO actually consumes.

Cisco IOS XE RESTCONF documentation (encoding, URLs, media types):

https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1717/b_1717_programmability_cg/restconf-protocol.html

4. Exam-style understanding

These are **original study items**, not Cisco exam questions. Use them to check whether you can *describe the value*, not recite a slogan.

Item A. A teammate says, “We already automate with `paramiko` and `expect`. Why bother with YANG?”

What a strong answer includes: CLI text is unstructured and version-fragile; YANG provides a schema;

NETCONF/RESTCONF carry validated structured data; config and state are distinguished; changes can be transactional.

Item B. Given a diagram labeled YANG → NETCONF → SSH, identify which box is the **model** and which is the **protocol**.

Answer: YANG is the model; NETCONF is the protocol; SSH is transport.

Item C. Which statement is true?

1. RESTCONF replaces YANG.
2. NETCONF is a YANG encoding.
3. YANG models the data; RESTCONF and NETCONF are protocols that encode it.
4. SNMP MIBs are YANG modules.

Answer: 3.

Item D. Why might an automation pipeline prefer `ietf-interfaces` over scraping `show ip interface brief`?

Answer: Stable named fields, JSON/XML parse, IETF portability, explicit enabled/oper-status leaves instead of a formatted table.

5. Hands-on exercise

1. Open `labs/06_yang_netconf_restconf/sample.yang` and list every `container`, `list`, `leaf`, and `key`.
2. Open `labs/06_yang_netconf_restconf/sample_restconf_get.json` and `sample_netconf_get-config.xml`. Confirm they are two encodings of the same interface idea.
3. In a Cisco DevNet Sandbox IOS XE Always-On (or reservable) lab, run `labs/06_yang_netconf_restconf/restconf_get_interfaces.py` after filling `labs/.env`. Notice `Accept: application/yang-data+json`.
4. Read [RFC 8040](#) section 1 (RESTCONF overview) and [RFC 6241](#) section 1 (NETCONF overview). You do not need to memorize the RFCs; you need the layering.

5.2 Compare controller-level to device-level management

1. What Cisco expects me to know

The verb is **Compare**. You must know differences, advantages, disadvantages, and when each approach is appropriate. **Controller-level** means a fabric or campus controller: Cisco Catalyst Center, ACI APIC, SD-WAN vManage, Meraki dashboard. **Device-level** means you talk to one box with SSH, NETCONF, RESTCONF, or a device REST API.

This is not “controllers are always better.” Cisco tests whether you can pick the right *scope*.

2. Detailed explanation

Device-level management is a 1:1 conversation with a switch, router, or firewall.

- Transport: SSH (CLI), NETCONF :830, RESTCONF :443, occasionally a device HTTP API (IOS XE RESTCONF, NX-API).
- Scope: that device's running config and local state.
- Strengths: precise, works in a lab of one box, no controller license, excellent for model-driven leaf changes and troubleshooting a single node.
- Weaknesses: you own inventory, consistency, sequencing, and rollback across 200 devices. A Python `for host in hosts` loop is still device-level management with extra steps.

Controller-level management is a 1:N conversation with a system that already has inventory, topology, and policy.

Controller	Typical domain	What you send it
Cisco Catalyst Center (DNA Center URLs still use <code>/dna/</code>)	Campus/enterprise LAN	Intent APIs: discover devices, images, assurance, templates
Cisco APIC	ACI fabric	Policy: tenants, EPGs, contracts — not per-leaf CLI
Cisco vManage	Catalyst SD-WAN	Templates, policies, device lists for WAN edges
Meraki dashboard	Cloud-managed campus/branch	Org → network → device hierarchy via Dashboard API

Strengths of controllers:

- **Intent and policy** instead of per-interface snippets. “These VLANs exist on this site” rather than 40 `switchport` blocks.
- **Inventory** is the source of truth for hostnames, serials, management IPs, software images.
- **Assurance / telemetry** is already aggregated (Catalyst Center client health, Meraki clients).
- **Blast radius is designed.** APIC pushes a fabric-consistent policy. A device-level NETCONF edit on one leaf can desynchronize the fabric if you fight the controller.

Weaknesses of controllers:

- Another platform to authenticate to, version, and trust.
- APIs are **product REST**, not IETF RESTCONF. You learn Catalyst Center token auth and Meraki `X-Cisco-Meraki-API-Key`, not a single YANG module.
- Some changes are asynchronous (Catalyst Center task IDs). Device-level NETCONF `rpc-reply` with `<ok/>` is simpler.
- If the controller is down, your automation story changes. Device-level still works for break-glass SSH.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph controller [Controller-level]
        Op1[Operator / script] --> CC[Catalyst Center / APIC / vManage / Meraki]
        CC --> D1[Switch A]
        CC --> D2[Switch B]
        CC --> D3[AP / WAN edge]
    end
    subgraph device [Device-level]
        Op2[Operator / script] --> SSH[SSH / NETCONF / RESTCONF]
        SSH --> D4[One IOS XE box]
    end
```

How they coexist. Production networks usually do both. You use Catalyst Center to onboard and image a campus. You still use RESTCONF on a lab CSR or a device that is not in a controller. NSO (5.6) sits in the middle: it is an orchestrator that speaks southbound to devices *and* can be driven northbound like a controller.

ACI nuance. Pushing XML to APIC is controller-level even though it looks like a device API. Pushing NX-OS CLI to a leaf that APIC owns is fighting the controller. Exam items often show an APIC URL (`/api/node/mo/ ...` or `/api/class/ ...`) versus an IOS XE RESTCONF URL (`/restconf/data/ ...`). Recognize the *level*.

3. Syntax and examples

Device-level RESTCONF (one IOS XE host from `labs/.env.example`):

```
url = "https://devnetsandboxiosxe.cisco.com:443/restconf/data/ietf-interfaces:interfaces"
# Talks to that box only.
```

Controller-level Catalyst Center (from `labs/05_cisco_apic/catalyst_center_devices.py`):

```
# Token is for the controller, not for each switch.
POST https://{CATALYST_CENTER_HOST}/dna/system/api/v1/auth/token
GET https://{CATALYST_CENTER_HOST}/dna/intent/api/v1/network-device
```

Controller-level Meraki (from `labs/05_cisco_apic/meraki_list_devices.py`):

```
GET https://api.meraki.com/api/v1/organizations
GET https://api.meraki.com/api/v1/organizations/{org_id}/devices
# Header: X-Cisco-Meraki-API-Key
```

Controller-level ACI (illustrative, original example):

```
# Login to APIC, then query a class – fabric-wide, not one leaf SSH session.
POST https://apic.lab.example/api/aaaLogin.json
GET https://apic.lab.example/api/class/fvTenant.json
```

4. Exam-style understanding

Original study items:

Item A. You must change the hostname on a single always-on IOS XE sandbox router that is not part of Catalyst Center. Device-level or controller-level?

Answer: Device-level RESTCONF or NETCONF.

Item B. You must list every access point in a Meraki organization and their LAN IPs. Device-level or controller-level?

Answer: Controller-level (Dashboard API). You do not SSH to each AP.

Item C. Advantage of controller-level for a 200-switch campus?

Answer: Shared inventory, intent/policy consistency, image/assurance APIs, less per-box drift.

Item D. Risk of device-level NETCONF `edit-config` on an ACI leaf?

Answer: You can create config that APIC does not own, causing drift or a later overwrite.

Item E. Match the URL to the level:

- `/restconf/data/Cisco-IOS-XE-native:native/hostname` → device
- `/dna/intent/api/v1/network-device` → controller
- `/api/v1/organizations/{id}/networks` → controller (Meraki)
- `/api/node/mo/uni/tn-PROD.json` → controller (ACI)

5. Hands-on exercise

1. Read `labs/05_cisco_apis/meraki_list_devices.py` and `catalyst_center_devices.py`. Write one sentence for each: “This script talks to ___ and therefore is ___-level.”
2. Read `labs/06_yang_netconf_restconf/restconf_get_interfaces.py`. Same sentence.
3. Draw the mermaid diagram above on paper and add NSO as a third box that has northbound REST and southbound NETCONF/CLI.
4. Optional: launch a Catalyst Center sandbox from <https://devnetsandbox.cisco.com> and a separate IOS XE sandbox. Hit one intent API and one RESTCONF path in Postman. Feel the difference in authentication (controller token vs device basic auth).

5.3 Describe the use and roles of network simulation and test tools (such as Cisco Modeling Labs and pyATS)

1. What Cisco expects me to know

The verb is **Describe**. Know **what CML and pyATS are for**, not how to license every CML node type. Cisco Modeling Labs (CML, formerly VIRL) **simulates topologies**. pyATS (with Genie) **parses, tests, and compares operational state**. Together they let you validate automation *before* it touches production.

2. Detailed explanation

Cisco Modeling Labs (CML) is a network simulator. You build a topology of virtual routers, switches, and Linux nodes, wire them with links, and start the lab. It is the successor to VIRL. Use it when you need:

- a **safe topology** to practice NETCONF/RESTCONF, routing, VLANs, or Ansible against IOS XE / NX-OS images
- **pre-change validation**: apply a Terraform/Ansible change to CML, then promote the same artifact to production
- **training and CI**: a pipeline spins a small CML lab, runs tests, destroys the lab

CML is not Packet Tracer. It runs real (virtual) network OS images, so APIs and YANG models behave like the hardware they emulate, within resource limits. It is also not a replacement for a controller sandbox: Catalyst Center and APIC have their own DevNet sandboxes.

Docs: <https://developer.cisco.com/modeling-labs/>

pyATS is Cisco's Python test framework for networks. **Genie** sits on top and provides parsers, models, and `learn / diff` of operational state. Use it when you need:

- to **parse** `show` command output or device APIs into structured Python without writing regex
- to **snapshot** ("learn") BGP, interfaces, or config before a change
- to **compare** post-change state to the snapshot (`genie diff`)
- to run **test cases** in a CI job: "after the playbook, every interface in the whitelist is up"

Docs: <https://developer.cisco.com/pyats/>

Roles in an automation workflow:

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant Dev as Engineer
    participant CML as CML topology
    participant Ans as Ansible / RESTCONF
    participant pyATS as pyATS / Genie
    participant Prod as Production
    Dev->>CML: Start virtual topology
    Dev->>Ans: Apply change to CML devices
    Ans->>CML: NETCONF / CLI
    Dev->>pyATS: Learn pre and post state
    pyATS->>Dev: Diff / pass-fail
    Dev->>Prod: Promote same change if tests pass
```

Other tools you may see in the same sentence (describe, do not over-study): VIRL (old name), CML Personal vs Enterprise, Cisco DevNet Sandbox (hosted topologies, not CML on your laptop), Robot Framework (sometimes wraps pyATS). The exam examples in the blueprint are **CML and pyATS**.

pyATS vs Pytest. Pytest is a general Python unit-test runner (domain 1.3). pyATS is a **network-aware** test harness: testbeds (YAML inventory of devices), connections, parsers. You can run pyATS jobs in CI the same way you run

pytest.

3. Syntax and examples

A conceptual pyATS testbed (YAML). This is original study syntax, not a live lab file:

```
testbed:
  name: cnaauto-lab
devices:
  edge-01:
    os: iosxe
    type: router
    connections:
      cli:
        protocol: ssh
        ip: 10.10.20.48
```

A conceptual Genie “learn then diff” flow:

```
pyats learn interface --testbed-file testbed.yml --output pre/
# ... automation runs ...
pyats learn interface --testbed-file testbed.yml --output post/
genie diff pre/ post/
```

CML is GUI- and API-driven. The skill to describe: nodes, links, simulations (start/stop), and reaching a node’s management IP for NETCONF. You do not need CML REST payload memorization for CCNA Automation.

4. Exam-style understanding

Original study items:

Item A. Which tool would you use to build a virtual three-router OSPF topology on a laptop or server?

Answer: Cisco Modeling Labs (CML).

Item B. Which tool would you use to parse `show ip bgp summary` across ten devices and fail CI if a neighbor dropped?

Answer: pyATS / Genie.

Item C. True or false: pyATS replaces YANG.

Answer: False. pyATS consumes CLI/API/YANG-modeled data to test state. YANG remains the device model.

Item D. Why run automation against CML before production?

Answer: Catch playbook/YANG errors, routing loops, and broken APIs without touching live traffic.

5. Hands-on exercise

1. Read <https://developer.cisco.com/pyats/> (overview) and <https://developer.cisco.com/modeling-labs/>. Write four bullet sentences: CML purpose, CML former name, pyATS purpose, Genie purpose.

2. Free path if you do not have CML: use DevNet Sandbox IOS XE as the “simulated” device and install pyATS in WSL (`pip install pyats[full]` or the current documented extra). Skip if disk is tight; reading the docs plus this chapter is enough to **describe** the roles.
 3. Optional: `pip install genie` and run a parser against sandbox CLI later in your study plan. Do not block Domain 5 on a CML license.
-

5.4 Describe the components and benefits of CI/CD pipeline in infrastructure automation

1. What Cisco expects me to know

The verb is **Describe**. Name the **components** of a CI/CD pipeline and the **benefits** when the thing being delivered is infrastructure (playbooks, Terraform, YANG payloads, Python API scripts), not only application binaries. You will not be asked to author a production Jenkinsfile from memory. You should recognize stages and why they exist.

2. Detailed explanation

CI (Continuous Integration) means every change to the automation repo is automatically built and tested. **CD (Continuous Delivery/Deployment)** means those artifacts are then released to a staging or production environment in a repeatable way.

For *infrastructure* automation, the “artifact” is often:

- an Ansible playbook and inventory
- a Terraform configuration plus a **plan**
- a Python package that wraps RESTCONF
- a YANG service model for NSO
- a CML topology definition

Typical components (left to right):

Stage	What happens for network automation	Example
Source	Git commit / pull request	GitHub, GitLab
Lint / static analysis	YAML/HCL/Python syntax, Ansible-lint, <code>terraform fmt</code>	Fail fast on tabs in YAML
Unit / contract tests	pytest for parsers; schema checks for JSON payloads	<code>labs/01_python_basics/test_subnet.py</code> style
Simulate / integration	CML, DevNet Sandbox, or a lab VRF	Apply playbook to CML
Operational tests	pyATS learn/diff, ping, RESTCONF GET assertions	Interface still up
Plan / review	<code>terraform plan</code> , Ansible <code>--check</code> , code review (5.13)	Human approval
Deploy	<code>terraform apply</code> , Ansible, NSO commit, controller intent API	Staging then prod
Feedback	Logs, artifacts, failed test output, monitoring	Chat notification

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart LR
  Git[Git PR] --> Lint[Lint / syntax]
  Lint --> Unit[Unit tests]
  Unit --> Sim[CML or lab apply]
  Sim --> PyATS[pyATS / API asserts]
  PyATS --> Review[Code review + plan]
  Review --> Deploy[Deploy to prod]
  Deploy --> Observe[Observe / rollback]

```

Benefits that matter on this exam:

1. **Repeatability.** The same playbook that passed CI is what production runs. No “it worked on my laptop with a different inventory.”
2. **Faster, safer change.** Tests catch a bad VLAN or a broken RESTCONF path before Friday night maintenance.
3. **Reviewability.** The pipeline enforces that a unified diff (5.12) went through code review (5.13).
4. **Smaller blast radius.** Failed pyATS jobs stop the deploy stage.
5. **Audit.** Git history plus CI logs answer “who changed BGP and did tests pass?”
6. **Rollback as a deploy of a previous commit**, not as tribal CLI knowledge.

CI/CD is how IaC (5.5) becomes operational. A Terraform file in a shared drive is not a pipeline. A Terraform file that cannot merge until `terraform plan` is clean and a reviewer approves *is*.

Infrastructure-specific pitfalls:

- Network changes are not always as easy to canary as a web app. Use CML/lab first, then a maintenance window, then prod.
- Secrets must not be in Git (use CI secret stores). Domain 4 covers secret management; mention it in pipelines.
- `--check` / `terraform plan` are **not** a substitute for operational tests. Plan says “I would create this resource”; pyATS says “BGP still has 4 neighbors.”

3. Syntax and examples

Original GitHub Actions-style sketch (interpret the workflow, do not memorize YAML keys):

```
name: network-ci
on: [pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Lint playbook
        run: ansible-lint labs/08_ansible/playbook.yml
      - name: Terraform fmt and plan
        run: |
          cd labs/09_terraform
          terraform fmt -check
          terraform init
          terraform plan
      - name: Unit tests
        run: pytest labs/01_python_basics/test_subnet.py
```

A bash fragment you might see in a deploy job:

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
ansible-playbook -i inventory.ini playbook.yml --check
ansible-playbook -i inventory.ini playbook.yml
```

`set -e` fails the pipeline on the first error — that is a CI benefit implemented in a shell.

4. Exam-style understanding

Original study items:

Item A. Name three pipeline stages that should run *before* production Ansible.

Answer: lint/syntax, lab/CML apply or `--check`, operational tests (pyATS/API), plus code review.

Item B. Benefit most closely tied to Git + CI together?

Answer: Every merged change is tested the same way; history is auditable.

Item C. `terraform plan` in CI vs `terraform apply` in CD — which is which?

Answer: Plan is CI/review; apply is CD/deploy (often gated).

Item D. Why is a pipeline valuable even if a human still clicks “approve”?

Answer: Humans review a **tested, formatted, diffed** artifact, not an untested laptop script.

5. Hands-on exercise

1. Open `labs/04_git/workflow.yaml` and map each Git operation to a CI moment (clone in the runner, commit on your machine, pull request before merge).
2. In WSL, from the repo root, run a *local* mini-pipeline:
`python -m pytest labs/01_python_basics/test_subnet.py`
then `terraform -chdir="labs/09_terraform" fmt -check` (after Terraform is installed per `CCNAUTO_LAB_SETUP.md`).
3. Write a five-line description of what you would add if you had CML: start lab → apply playbook → pyATS → destroy lab.

5.5 Describe the principles of infrastructure as code

1. What Cisco expects me to know

The verb is **Describe**. **Infrastructure as code (IaC)** means the desired state of infrastructure is **declared in files, versioned in Git, reviewed, and applied repeatably**. You should contrast this with click-ops and with imperative SSH snowflakes. Ansible, Terraform, NSO, and even RESTCONF payloads stored in Git are IaC *if* they follow those principles.

2. Detailed explanation

Principles you should be able to explain:

1. **Declarative desired state.** You write *what* should be true: nginx installed, interface enabled, DNS record present. The tool computes *how*. Imperative scripts (`apt-get install` without checking) can be “code that touches infra” without being good IaC.
2. **Versioned.** The source of truth is Git, not a running box and not a wiki screenshot. You can `git revert` .
3. **Reviewable.** Changes show up as unified diffs (5.12) and go through code review (5.13).
4. **Repeatable / idempotent.** Applying the same definition twice converges to the same state. Ansible modules with `state: present` and Terraform resources are designed for this. A bash script that appends a line to a file every run is not.
5. **Documented by being executable.** The playbook *is* the documentation of how the lab host is built, if it is the path people actually run.
6. **Environment-parameterized.** Inventory, Terraform variables, or NSO device lists separate *code* from *data* (which host, which VLAN).
7. **Testable in a pipeline** (5.4).

Declarative vs imperative (exam favorite):

Style	You write	Example
Imperative	Steps	<code>useradd ccnaauto ; systemctl start nginx</code>
Declarative	End state	<code>user: name=ccnaauto state=present ; service: state=started</code>

Ansible is **procedural-ish** (an ordered list of tasks) but each module is usually **idempotent** and **desired-state**. Terraform is more purely declarative: a graph of resources, not a step list. NSO service models are declarative at the service layer.

What IaC is not:

- Pasting CLI into Notepad and calling it a “template” without version control.
- A Python script that SSH-es unique snowflake commands per device with no inventory and no review.
- Storing production secrets in the same file as the code.

Relationship to model-driven programmability. IaC is the *software engineering* practice. YANG/NETCONF is a *device API* that makes IaC accurate. You can do IaC against CLI (Ansible `ios_command`) but you inherit CLI fragility. Model-driven plus IaC is the combination Domain 5 is selling.

3. Syntax and examples

Terraform in this repo is a miniature IaC definition (`labs/09_terraform/main.tf`): the desired state is “this JSON inventory file exists with these devices.” Re-applying does not keep creating new files with random names; it updates the one resource.

Ansible in this repo (`labs/08_ansible/playbook.yml`) declares package present, user present, file content, service started. A second run should report `ok / changed=0` if nothing drifted.

A **non-IaC** bash anti-pattern:

```
echo "ccnaauto ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
```

Every run appends another line. The IaC version uses a module or a `lineinfile` with a regexp so the line exists once.

4. Exam-style understanding

Original study items:

Item A. List four IaC principles.

Answer: desired state, versioned, reviewable, repeatable/idempotent (plus testable/parameterized).

Item B. Is a Jupyter notebook that someone runs once to create VLANs IaC?

Answer: It is code, but it fails versioned/reviewable/repeatable unless it is in Git, idempotent, and the path people actually use.

Item C. Why does Terraform store state?

Answer: So the tool knows which real-world objects map to which declared resources — required for declarative lifecycle (create/update/destroy).

5. Hands-on exercise

1. Read `labs/09_terraform/main.tf` and `labs/08_ansible/playbook.yml`. For each file, write: declarative or imperative-with-idempotent-tasks? What is the desired state in one sentence?
2. Initialize and apply the Terraform lab (see 5.6 exercise). Change a device name in `main.tf`, run `terraform plan`, and observe the diff — that plan *is* IaC in action.
3. Intentionally make the Ansible playbook non-idempotent (append to a file with `shell: echo x >> file`). Run twice. Restore the module-based version. Feel the principle.

5.6 Describe the capabilities of automation tools such as Ansible, Terraform, and Cisco NSO

1. What Cisco expects me to know

The verb is **Describe** the **capabilities** of three named tools. You are not required to be a HashiCorp or Red Hat certified specialist. You must know what each tool is good at, how it is driven, and how it differs from the other two. Objective **5.8** later asks you to **interpret** an Ansible playbook in more detail; 5.6 is the product map.

Official docs (free):

- Ansible: <https://docs.ansible.com/>
- Terraform: <https://developer.hashicorp.com/terraform/docs>
- Cisco NSO: <https://developer.cisco.com/docs/nso/>

2. Detailed explanation

Ansible

Ansible is a **configuration automation** engine. Default transport to Linux is **agentless SSH** (Windows uses WinRM; network devices use SSH, NETCONF, or HTTP APIs via collections). You write **YAML playbooks**. A playbook contains **plays**; a play maps **hosts** from an **inventory** to **tasks**; each task calls a **module**.

Capabilities to remember:

- Agentless: nothing to install on the target except SSH and Python for Linux; network modules often run on the control node.
- Inventory: which hosts, which groups, which variables.
- Modules: `package`, `user`, `service`, `copy`, `ios_config`, `cisco.ios.ios_interfaces`, and hundreds more.

- Idempotence when modules are used correctly (`state: present` rather than raw `apt-get` every time).
- Procedural-ish execution: tasks run **in order**. That is different from Terraform's dependency graph. People still call Ansible "declarative" because each module expresses desired state.
- Ansible Galaxy / collections for Cisco IOS, NX-OS, ACI, Meraki.
- Check mode (`--check`) for a dry run.
- Not a persistent state database. The world *is* the state; Ansible converges it each run.

Ansible is the tool you reach for when the job is "make this OS or this device look like X": packages, users, services, config snippets.

Terraform

Terraform is a **provisioning** tool. You write **HCL** (HashiCorp Configuration Language). You declare **resources** that belong to **providers** (AWS, Azure, `local`, Cisco Intersight, ACI provider, and so on). Terraform keeps **state** (often `terraform.tfstate`) that maps declarations to real objects.

Lifecycle you must know:

1. `terraform init` — download providers.
2. `terraform plan` — show the diff between declared and actual (via state + API refresh).
3. `terraform apply` — create/update/delete to match the declaration.
4. `terraform destroy` — tear down resources Terraform owns.

Capabilities:

- Strongly **declarative**. You do not write "step 1, step 2"; you write resources and Terraform builds a graph.
- **State** is a first-class concept. Lose the state file and Terraform no longer knows what it created.
- Excellent at cloud objects, DNS records, load balancers, and generating files (as in this repo's `local_file` lab).
- Less natural than Ansible for "install nginx and start it" on a snowflake VM, though you *can* use provisioners (discouraged).
- Plan output is reviewable IaC — it belongs in CI (5.4).

Cisco NSO (Network Services Orchestrator)

NSO is a **service orchestrator** for multi-vendor networks. It is not "Ansible with a Cisco logo."

Core ideas:

- **Device models** (NED — Network Element Driver): how each OS looks in NSO, often YANG, southbound **NETCONF** when the device supports it, otherwise **CLI** or other adapters.
- **Service models**: YANG that describes a *customer-facing service* (L3VPN, VLAN service, CNAME + firewall hole) rather than a single device CLI.
- A configuration database (**CDB**) holds the service instance and the rendered device configs.
- You **commit** a service instance; NSO computes the device deltas and pushes them southbound. Abort/rollback is a transaction at the service layer.

- Northbound: RESTCONF, NETCONF, CLI, Python APIs — NSO itself is model-driven.

NSO's capability is **keep a service's intent and all device configs in sync** across vendors. Ansible can push configs; NSO is built to *own* the service lifecycle (create, modify, delete) and the mapping to many devices.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

graph TD
    subgraph north [Northbound]
        Op[Operator / CI]
        Op --> REST[RESTCONF / Python / CLI]
    end
    REST --> NSO[Cisco NSO<br/>service YANG + CDB]
    NSO --> NETCONF[Southbound NETCONF]
    NSO --> CLI[Southbound CLI NED]
    NETCONF --> XE[IOS XE]
    CLI --> Old[Legacy OS]

```

Comparison table

Capability	Ansible	Terraform	Cisco NSO
Primary language	YAML playbooks	HCL	YANG service + templates / Python
Default style	Ordered idempotent tasks	Declarative resource graph	Declarative service instances + commit
State	No central state file; world is state	<code>tfstate</code> required	CDB
Agent	Agentless SSH / APIs	Provider APIs	Southbound NED (NETCONF/CLI)
Sweet spot	OS + device config drift	Cloud/infra objects, lifecycle	Multi-device network <i>services</i>
Dry run	<code>--check</code>	<code>plan</code>	compare / dry-run commit
Destroy	Playbook with <code>state: absent</code>	<code>terraform destroy</code>	un-deploy service instance

Misconceptions: Ansible is not “only Linux.” Terraform is not “only AWS.” NSO is not a replacement for Catalyst Center in campus assurance; it is orchestration of services onto devices.

3. Syntax and examples

Ansible inventory (`labs/08_ansible/inventory.ini`):

```
[lab]
127.0.0.1 ansible_connection=local
```

The group is `[lab]`. The playbook's `hosts: lab` selects it. `ansible_connection=local` means “do not SSH; run on this machine” — useful for a WSL self-lab.

Ansible playbook (`labs/08_ansible/playbook.yml`) — full interpretation is 5.8. Capability shown here: package, user, file, service modules in one play.

Terraform (`labs/09_terraform/main.tf`):

```
terraform {
  required_version = "≥ 1.5.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "device_inventory" {
  filename = "${path.module}/generated_inventory.json"
  content = jsonencode({
    devices = [
      { name = "edge-01", mgmt = "10.10.20.48" },
      { name = "core-01", mgmt = "10.10.20.49" },
    ]
  })
}
```

Read it as: “The `local` provider can manage files on disk. Resource `local_file.device_inventory` must exist at that path with that JSON content.” `${path.module}` is the directory containing this `.tf` file. `jsonencode` turns a HCL object into a JSON string.

NSO (conceptual northbound RESTCONF, original example):

```
POST /restconf/data/l3vpn:vpn
Content-Type: application/yang-data+json

{"l3vpn:vpn": {"name": "ACME", "rd": "65000:10"}}
```

You will not build NSO in the free local lab. You must still recognize: service name, commit, device list, southbound NETCONF.

4. Exam-style understanding

Original study items:

Item A. Which tool stores a state file mapping resources to real objects?

Answer: Terraform.

Item B. Which tool is agentless and typically uses SSH plus YAML tasks?

Answer: Ansible.

Item C. Which tool models a VPN as a service instance and pushes per-device config via NEDs?

Answer: Cisco NSO.

Item D. You need to install `nginx`, create user `ccnaauto`, and enable the service on lab VMs. Best fit?

Answer: Ansible.

Item E. You need to create three cloud load balancers and later destroy them cleanly. Best fit?

Answer: Terraform.

Item F. True or false: Terraform playbooks are YAML.

Answer: False. Terraform uses HCL. Ansible uses YAML.

5. Hands-on exercise

1. Ansible (WSL):

```
cd labs/08_ansible
```

```
ansible-playbook -i inventory.ini playbook.yml --check
```

Then run without `--check` if you accept installing `nginx` locally. Read the `changed` / `ok` counters on a second run.

2. Terraform:

```
cd labs/09_terraform
```

```
terraform init
```

```
terraform plan
```

```
terraform apply
```

Open `generated_inventory.json`. Change a hostname in `main.tf`, `plan` again, then `apply`. Finish with `terraform destroy` if you want a clean tree.

3. **NSO:** Read the NSO developer docs landing page: <https://developer.cisco.com/docs/nso/> — write five sentences: service model, device model, commit, southbound, northbound. DevNet sometimes lists an NSO sandbox; use it if available, otherwise docs plus this chapter satisfy **describe**.

5.7 Identify the workflow being automated by a Python script that uses Cisco APIs including ACI, Meraki, Cisco Catalyst Center, and RESTCONF

1. What Cisco expects me to know

The verb is **Identify**. Cisco will show you a Python snippet that uses `requests` (or an SDK) against **ACI**, **Meraki**, **Cisco Catalyst Center**, or **RESTCONF**. You must say **what the script is doing** — which platform, which resource,

which HTTP method, and the workflow (auth → list → maybe create). You do not need to memorize every endpoint in those products. You need **recognition patterns**.

This objective is paired with Domain 3 platform APIs. Here the skill is *workflow identification*, not product administration.

2. Detailed explanation

Train yourself to scan a script in this order:

1. **Base URL and path** — the fastest discriminator.
2. **Auth header or login call**.
3. **HTTP methods** and order.
4. **Payload keys** (tenant, orgId, network-device, yang-data).
5. **What is printed or returned** — list devices, set hostname, create tenant.

Recognition cheat sheet:

Platform	URL fragment	Auth pattern	Typical workflow
Meraki	api.meraki.com/api/v1 /organizations /networks /devices /clients	X-Cisco-Meraki-API-Key	List orgs → pick org → list devices/clients
Catalyst Center	/dna/system/api/v1/auth/token /dna/intent/api/v1/	POST token with basic auth, then X-Auth-Token	Get token → list network-device / clients / sites
ACI APIC	/api/aaaLogin.json /api/class/ /api/node/mo/uni/	Cookie / token from aaaLogin	Login → GET class or POST managed object
RESTCONF (device)	/restconf/data/ ietf-interfaces: Cisco-IOS-XE-native:	HTTP Basic to the device , Accept: application/yang- data+json	GET/PATCH a YANG path on one box

Meraki workflow. Cloud controller. Almost always: authenticate with API key on every request (no separate token call). Hierarchy is Organization → Network → Device / Client. A script that fetches /organizations then /organizations/{id}/devices is **inventory listing**, not changing SSIDs.

Catalyst Center workflow. On-prem (or sandbox) controller. You **must** obtain a token first. Paths under /dna/intent/ are intent APIs (inventory, command runner, templates). A script that only GET-lists devices is discovery/inventory. A script that POST-s a template is change.

ACI workflow. APIC is the controller. Login JSON body typically contains aaaUser credentials. After login, GET /api/class/fvTenant.json lists tenants. POST to a distinguished name under uni/ creates policy. If you see fvTenant, fvBD, fvAEPg, you are in ACI even if the word “ACI” never appears.

RESTCONF workflow. Device-level. Media type `application/yang-data+json` (or `+xml`). Path starts with `/restconf/data/` plus `module:container/ ... . interface=GigabitEthernet1` is how RESTCONF encodes a YANG list key in the URL. GET is read; PATCH/PUT/POST/DELETE mutate according to RESTCONF rules.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    Start[Start: Read the script] --> URL1[URL: URL or header?]
    URL1 --> M[M: Meraki org/network/device workflow]
    URL1 --> C[C: Catalyst Center token then intent]
    URL1 --> A[A: ACI login then MO/class]
    URL1 --> R[R: Device RESTCONF YANG resource]
```

SDK pattern. `labs/05_cisco_apis/sdk_pattern.py` wraps `requests`. Exam scripts might call `api.organizations.get_organizations()` (Meraki SDK) or `dnac.devices.get_device_list()`. Identify the **workflow**, not the SDK brand.

3. Syntax and examples

Meraki — list devices (workflow of `labs/05_cisco_apis/meraki_list_devices.py`):

```
orgs = get("/organizations")
org_id = orgs[0]["id"]
devices = get(f"/organizations/{org_id}/devices")
clients = get(f"/networks/{net_id}/clients", params={"timespan": 86400})
```

Identify: *Authenticate with Dashboard API key. List organizations, take the first, list that org's devices, then list clients on the first network for the last day.*

Catalyst Center — inventory (`catalyst_center_devices.py`):

```
tok = token() # POST /dna/system/api/v1/auth/token
devices = get("/dna/intent/api/v1/network-device", tok)
```

Identify: *Obtain a Catalyst Center token, then list network devices from the intent API.*

RESTCONF — read hostname and interfaces (`restconf_get_interfaces.py`):

```
HEADERS = {
    "Accept": "application/yang-data+json",
    "Content-Type": "application/yang-data+json",
}
restconf("Cisco-IOS-XE-native:native/hostname")
restconf("ietf-interfaces:interfaces")
```

Identify: *HTTP Basic to an IOS XE device. GET native hostname leaf, then GET IETF interfaces container. Read-only inventory of that box.*

ACI — original example (not in the lab folder):

```
login = {
  "aaaUser": {"attributes": {"name": user, "pwd": password}}
}
s = requests.Session()
s.post(f"https://{apic}/api/aaaLogin.json", json=login, verify=False)
tenants = s.get(f"https://{apic}/api/class/fvTenant.json", verify=False).json()
```

Identify: *Log in to APIC, then list all tenant objects in the fabric.*

RESTCONF write workflow (original):

```
requests.patch(
  "https://router/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1",
  auth=("admin", "admin"),
  headers={"Content-Type": "application/yang-data+json"},
  json={"ietf-interfaces:interface": {"description": "UPLINK", "enabled": True}},
  verify=False,
)
```

Identify: *Device-level RESTCONF PATCH of one interface's description and enabled flag. Not Meraki, not Catalyst Center.*

4. Exam-style understanding

Original study items. Practice **naming the platform and the workflow in one sentence**.

Item A. Script uses `X-Cisco-Meraki-API-Key` and GET `/networks/{id}/clients`.

Answer: Meraki controller API listing clients on a network.

Item B. Script POSTs to `/dna/system/api/v1/auth/token` then GETs `/dna/intent/api/v1/network-device`.

Answer: Catalyst Center: authenticate, then inventory network devices.

Item C. Script POSTs `aaaLogin.json` then GETs `/api/class/fvTenant.json`.

Answer: ACI APIC: login, list tenants.

Item D. Script GETs `/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1` with `Accept: application/yang-data+json`.

Answer: RESTCONF read of one IETF interface on a device.

Item E. Distractor: Webex `https://webexapis.com/v1/rooms` is **not** in the 5.7 list. If you see it, you are in Domain 3 collaboration APIs, not this objective.

Item F. A script has both `aaaLogin` and later `interface GigabitEthernet`. If the `GigabitEthernet` appears inside a RESTCONF URL, you have two different tools — but ACI scripts do not normally use RESTCONF on the APIC for interface IETF models. Prefer the URL.

5. Hands-on exercise

1. Open every file in `labs/05_cisco_apis/` and `labs/06_yang_netconf_restconf/restconf_get_interfaces.py`. For each, write: platform, auth, resources touched, mutating or read-only.
 2. Cover the ACI gap: in Postman or a scratch file, write (do not need to run) a 15-line `aaaLogin` + `fvTenant` script using the pattern above. Say the workflow out loud.
 3. Shuffle printed snippets with a study partner (or hide filenames) and identify them in under 30 seconds — that is the exam skill.
-

5.8 Interpret the workflow being automated by an Ansible playbook (management packages, user management related to services, basic service configuration, and start/stop)

1. What Cisco expects me to know

The verb is **Interpret**. Cisco tells you the scope: **management packages, user management related to services, basic service configuration, and start/stop**. You will be shown a YAML playbook similar to `labs/08_ansible/playbook.yml`. You must explain **which hosts, which tasks in order, what desired state each module enforces, and whether a re-run would change anything**.

You are not asked to write a 200-task role from scratch. You *are* asked to read YAML indentation, `hosts`, `become`, `vars`, module names, and `state` keys.

2. Detailed explanation

Anatomy of a playbook:

```
- name: Human description of the play
  hosts: group_from_inventory
  become: true          # escalate to root (sudo)
  vars:
    some_key: value
  tasks:
    - name: Human description of the task
      ansible.builtin.module_name:
        argument: value
```

Play vs task. The list item with `hosts:` is a **play**. The list under `tasks:` is ordered **tasks**. Ansible runs task 1 on all matched hosts (by default), then task 2, and so on.

Inventory selects machines. `hosts: lab` means “all hosts in group `lab`.” `hosts: all` means everyone. A hostname can appear in `inventory.ini`.

Modules in the exam scope:

Workflow Cisco named	Typical modules	Key arguments
Management packages	<code>package</code> , <code>yum</code> , <code>apt</code> , <code>dnf</code>	<code>name</code> , <code>state: present / absent / latest</code>
User management related to services	<code>user</code> , <code>group</code>	<code>name</code> , <code>state</code> , <code>shell</code> , <code>create_home</code> , <code>groups</code>
Basic service configuration	<code>copy</code> , <code>template</code> , <code>lineinfile</code> , <code>file</code>	<code>dest</code> , <code>content</code> / <code>src</code> , <code>mode</code>
Start/stop	<code>service</code> , <code>systemd</code>	<code>name</code> , <code>state: started / stopped / restarted</code> , <code>enabled: true</code> (start on boot)

Idempotence. `package: state=present` installs only if missing. `user: state=present` creates only if missing. `service: state=started` starts only if not running. `copy` writes the file if content differs. A second run should be green.

`become: true` is required for package install, user create, and writing `/var/www`. Without it, those tasks fail with permission denied — a common interpretation trap.

FQCN. `ansible.builtin.package` is the fully qualified collection name. Older playbooks say `package:` or `apt:`. Same workflow.

Handlers. You may see `notify: Restart nginx` and a `handlers:` section. A handler runs **once at the end if notified** — typical for “restart service only if the config file changed.” That is still start/stop, just deferred.

What the exam playbook is *not*. It is not Terraform. It is not a Kubernetes manifest. If you see `tasks:` and `ansible.builtin`, it is Ansible.

3. Syntax and examples

Full lab playbook (`labs/08_ansible/playbook.yml`), interpreted line by line:

```

- name: Prepare a lab linux host
  hosts: lab
  become: true
  vars:
    lab_user: ccnaauto
    app_package: nginx

  tasks:
    - name: Ensure nginx is installed
      ansible.builtin.package:
        name: "{{ app_package }}"
        state: present

    - name: Create application user
      ansible.builtin.user:
        name: "{{ lab_user }}"
        shell: /bin/bash
        state: present
        create_home: true

    - name: Deploy a simple index page
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          CCNA Automation Ansible lab
        mode: "0644"

    - name: Start and enable nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

```

Workflow in English:

1. Target the `lab` inventory group; use `sudo`.
2. **Package management:** ensure the package named in `app_package` (nginx) is installed.
3. **User management:** ensure user `ccnaauto` exists, bash shell, home directory.
4. **Basic service configuration:** write `/var/www/html/index.html` with a fixed string, mode 0644.
5. **Start/stop:** nginx process must be running **now** (`started`) and must start at boot (`enabled: true`).

Jinja2 `{{ app_package }}` substitutes variables. If you change `app_package: httpd`, the same play installs Apache instead — still package management.

Start vs stop original snippet:

```

- name: Stop nginx for maintenance
  ansible.builtin.service:
    name: nginx
    state: stopped
    enabled: false

```

Interpret: *Stop the service now and prevent it from starting on boot.* Opposite of the lab playbook's last task.

Package absent:

```
- ansible.builtin.package:
  name: telnet
  state: absent
```

Interpret: *Uninstall telnet if present.* Still “management packages.”

4. Exam-style understanding

Original study items:

Item A. After a successful first run of the lab playbook, what does a second run do to the user `ccnaauto`?

Answer: Nothing material; `state: present` is already satisfied.

Item B. Which task implements “basic service configuration” rather than package install?

Answer: The `copy` to `index.html`.

Item C. `enabled: true` vs `state: started`.

Answer: `state` is now; `enabled` is boot-time.

Item D. `hosts: lab` but inventory only has `127.0.0.1` under `[lab]`. Where does nginx get installed?

Answer: The local machine (especially with `ansible_connection=local`).

Item E. Task order: could you start nginx *before* installing the package?

Answer: The playbook would fail; order matters. Ansible is a task list.

Item F. Given a playbook with `user: name=www-data` and `service: name=nginx state=started`, summarize.

Answer: Ensure a service account exists and nginx is running — user management related to services plus start.

5. Hands-on exercise

1. Read `labs/08_ansible/playbook.yml` and `inventory.ini` without running anything. Write the five-sentence workflow above from memory.
 2. In WSL: `ansible-playbook -i labs/08_ansible/inventory.ini labs/08_ansible/playbook.yml --syntax-check`
 3. Run `--check` then a real apply if you want nginx on the VM. Run twice. Screenshot or copy the recap (`ok=`, `changed=`).
 4. Edit a copy: add a task `state: stopped` for nginx, predict the recap, run it, then restore `started`.
 5. Docs: https://docs.ansible.com/ansible/latest/collections/ansible/builtin/service_module.html — read `state` and `enabled`.
-

5.9 Interpret the workflow being automated by a bash script (such as file management, app install, user management, directory navigation)

1. What Cisco expects me to know

The verb is **Interpret**. Given a bash script, explain the workflow. Cisco's examples: **file management, app install, user management, directory navigation**. You need to read `cd`, `mkdir`, `cp`, `mv`, `rm`, `apt-get` / `yum`, `useradd` / `id`, `chmod`, `chown`, variables, and `if` tests well enough to narrate the script.

This is Linux-for-automation literacy, not a LPIC exam.

2. Detailed explanation

Bash scripts used in network automation labs usually bootstrap a control node or a small Linux helper VM. Read them as a **sequence of side effects**.

Directory navigation. `cd /opt/app` changes the working directory for subsequent relative paths. `cd "$(dirname "$0")"` means “go to the directory where this script lives” — common in CI. `pwd` prints the current directory. Failure to `cd` is why a script “cannot find” a file that exists.

File management.

Command	Workflow
<code>mkdir -p /etc/myapp</code>	Create directory path; <code>-p</code> no error if exists, creates parents
<code>cp src dest</code>	Copy file
<code>mv old new</code>	Rename or move
<code>rm -f file</code>	Delete file, ignore missing
<code>rm -rf dir</code>	Recursive delete — dangerous
<code>chmod 644 file</code>	Permissions
<code>chown user:group file</code>	Ownership
<code>cat > file <<'EOF'</code>	Write a file from a heredoc
<code>touch file</code>	Create empty or update timestamp

App install. Debian/Ubuntu: `apt-get update` then `apt-get install -y nginx`. RHEL: `yum install -y` or `dnf`. `-y` answers yes. `apt-get update` refreshes package **indexes**; it does not upgrade every package (`upgrade` does).

User management. `useradd -m -s /bin/bash ccnaauto` creates a user with home. `id ccnaauto` checks existence. `usermod -aG sudo ccnaauto` adds a group. `passwd` is interactive — pipelines avoid it and use `chpasswd` or SSH keys.

Scripting glue. `set -e` exit on error. `if [ -d /var/www ]; then` tests a directory. `$1` is the first argument. `$@` is all arguments. `#` comments.

Ansible vs bash. A bash script that always runs `apt-get install` is imperative. Ansible `package: state=present` is the IaC form of the same workflow. Exam: interpret the bash, do not rewrite it unless asked.

3. Syntax and examples

Original bootstrap script covering all four Cisco example areas:

```
#!/usr/bin/env bash
# Workflow: navigate, install nginx, create service user, deploy a file.
set -euo pipefail

APP_DIR=/opt/ccnaauto-web
USER_NAME=ccnaauto

cd /tmp
apt-get update
apt-get install -y nginx

if ! id "$USER_NAME" >/dev/null 2>&1; then
    useradd -m -s /bin/bash "$USER_NAME"
fi

mkdir -p "$APP_DIR"
cat > "$APP_DIR/index.html" <<'EOF'
CCNA Automation bash lab
EOF
cp "$APP_DIR/index.html" /var/www/html/index.html
chown "$USER_NAME":"$USER_NAME" "$APP_DIR/index.html"
chmod 644 /var/www/html/index.html

systemctl enable --now nginx
```

Narration: Change to `/tmp`. Refresh apt indexes and install nginx (app install). If user `ccnaauto` does not exist, create with home and bash (user management). Create `/opt/ccnaauto-web` (directory), write `index.html` (file management), copy it into nginx's docroot, set owner and mode. Enable and start nginx.

Directory navigation trap:

```
cd /etc/nginx
cp nginx.conf /tmp/nginx.conf.bak
cd /var/www/html
rm index.html
```

Interpret: Backup the nginx config from `/etc/nginx`, then delete `index.html` from the web root — two different directories; the `rm` does not delete `/etc/nginx/index.html`.

Relative vs absolute:

```
cd /opt
mkdir app
cd app
touch config.yml
```

After this, `config.yml` is `/opt/app/config.yml`. If the script never `cd`s back, the next relative `cp` still uses `/opt/app`.

4. Exam-style understanding

Original study items:

Item A. What does `mkdir -p /var/www/html` do if `/var/www` already exists?

Answer: Creates `html` if needed; does not fail.

Item B. `apt-get install -y git` without `update` — risk?

Answer: May install from stale indexes or fail to find the package.

Item C. Script contains `useradd ccnaauto` with no existence check and no `-m`. Second run?

Answer: Likely fails “user already exists”; home may be missing from the first run.

Item D. Match: `cp`, `useradd`, `dnf install`, `cd` → file, user, app, navigation.

Item E. `rm -rf "$WORKDIR"` when `WORKDIR` is empty accidentally.

Answer: Can delete from the current directory or worse — why `set -u` and quoting matter. Interpret as destructive file management.

5. Hands-on exercise

1. In WSL, create `labs/08_ansible/bootstrap.sh` as a **study copy** of the script above (or run pieces manually). Do not run `rm -rf` experiments as root.
2. Trace `pwd` after each `cd` in the “trap” example on paper.
3. Compare the bash script to `labs/08_ansible/playbook.yml`. Same workflow, different IaC quality. Write three differences (idempotence, become, inventory).

5.10 Interpret the results of a RESTCONF or NETCONF query

1. What Cisco expects me to know

The verb is **Interpret**. This is a high-value Domain 5 skill. You will be shown a **RESTCONF** HTTP exchange or a **NETCONF** XML RPC and must say what was requested and what the device answered. You need media types, URL paths, NETCONF operations (`get` vs `get-config`, `edit-config`), and the meaning of `<rpc-reply>`, `<ok/>`, `<data>`, and JSON `yang-data` bodies.

RFCs: [RFC 8040 RESTCONF](#), [RFC 6241 NETCONF](#).

Cisco IOS XE RESTCONF: https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1717/b_1717_programmability_cg/restconf-protocol.html

2. Detailed explanation

RESTCONF

RESTCONF maps YANG data to HTTP.

- **Entry point:** `/restconf/data/` for the datastore; `/restconf/operations/` for YANG RPCs; `/restconf/yang-library-version` (and related) for what modules exist.
- **Media type:** `application/yang-data+json` or `application/yang-data+xml`. If you send `application/json` without the YANG media type, IOS XE may return **415**.
- **Path:** `/restconf/data/<module>:<container>/ ...`
Example: `/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1`
The `module:` prefix disambiguates. The `=key` syntax selects a **list** entry by its YANG key (`name` for interfaces).
- **Methods:** GET (read), POST (create in a list/container), PUT (replace), PATCH (merge), DELETE (remove).
- **Auth:** typically HTTP Basic over TLS. Port **443** (sometimes a nonstandard sandbox port).
- **Success bodies:** GET returns the resource JSON/XML. PATCH/DELETE may return **204 No Content**. Errors return YANG-modeled error JSON plus 4xx/5xx.

How to interpret a GET result. Look at the top-level key. `ietf-interfaces:interfaces` means “this object is the `interfaces` container from module `ietf-interfaces`.” Nested `interface` is a list (JSON array). Each element must include the key `name`. Leaves such as `enabled` are booleans. Augmentations appear as other module prefixes, for example `ietf-ip:ipv4` inside an interface.

Query parameters you may see: `content=config` vs `content=nonconfig` vs `content=all` (config vs operational state). Depth and fields parameters exist; if the exam shows `?content=config`, they are asking you to notice **configuration only**.

NETCONF

NETCONF is an XML RPC protocol over SSH, default TCP **830**.

Session start: client and server exchange `<hello>` with `<capabilities>`. Capabilities advertise protocol version (`:base:1.0` / `1.1`), datastores (`:candidate` , `:writable-running`), and YANG modules as URNs. Interpreting a hello: “the box supports candidate config and these modules.”

Common RPCs:

RPC	Meaning	Typical reply
get-config	Read configuration from a datastore (running , candidate , startup)	<rpc-reply><data> ... </data> </rpc-reply>
get	Read config and/or operational state (filter selects)	<data> with possibly config false nodes
edit-config	Merge/replace/delete config in a target datastore	<ok/> on success
copy-config	Copy one datastore to another	<ok/>
lock / unlock	Exclusive edit	<ok/>
commit	Candidate → running (if :candidate)	<ok/>
close-session	Graceful disconnect	<ok/>

Filters. A subtree filter in <get-config> limits XML to, for example, one interface. Interpreting a small <data> does **not** mean the device has only one interface; it means the filter asked for one.

get vs get-config. If the XML contains operational leaves such as oper-status or statistics , it is almost certainly <get> (or RESTCONF operational resource), not <get-config> . Configuration leaves (description , enabled) appear in both.

Errors. <rpc-reply> with <rpc-error> : look at <error-tag> (invalid-value , data-missing , access-denied). Do not treat any <rpc-reply> as success.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant C as Client ncclient
    participant D as IOS XE :830
    C->>D: SSH + NETCONF hello
    D->>C: hello + capabilities
    C->>D: rpc get-config source=running + subtree filter
    D->>C: rpc-reply data interfaces XML
    C->>D: close-session
    D->>C: ok
```

3. Syntax and examples

RESTCONF GET matching labs/06_yang_netconf_restconf/sample_restconf_get.json :

```
GET /restconf/data/ietf-interfaces:interfaces HTTP/1.1
Host: devnetsandboxiosxe.cisco.com
Accept: application/yang-data+json
Authorization: Basic ...
```

Result to interpret:

```
{
  "ietf-interfaces:interfaces": {
    "interface": [
      {
        "name": "GigabitEthernet1",
        "description": "MANAGEMENT",
        "enabled": true,
        "ietf-ip:ipv4": {
          "address": [
            {
              "ip": "10.10.20.48",
              "netmask": "255.255.255.0"
            }
          ]
        }
      }
    ]
  }
}
```

Interpretation: The device returned the IETF interfaces container. One list entry, key `GigabitEthernet1`, description `MANAGEMENT`, administratively enabled. The IETF IP module is **augmented** onto the interface: IPv4 address `10.10.20.48/24` (mask form). This is **configuration-shaped** data (address, enabled), not packet counters.

Single-interface URL:

```
/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1
```

Equals sign encodes the list key. A slash-only path would be a different container.

NETCONF `get-config` result (`labs/06_yang_netconf_restconf/sample_netconf_get-config.xml`):

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <data>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
      <interface>
        <name>GigabitEthernet1</name>
        <description>MANAGEMENT</description>
        <enabled>true</enabled>
      </interface>
    </interfaces>
  </data>
</rpc-reply>
```

Interpretation: Successful reply (`data` , no `rpc-error`) to message-id 101. Configuration of IETF interfaces: `GigabitEthernet1`, description `MANAGEMENT`, enabled `true`. Same story as the JSON, without the IPv4 augmentation in this sample.

NETCONF request that produced it (from `netconf_get_config.py`):

```
<rpc message-id="101">
  <get-config>
    <source><running /></source>
    <filter>
      <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
        <interface>
          <name>GigabitEthernet1</name>
        </interface>
      </interfaces>
    </filter>
  </get-config>
</rpc>
```

Interpretation: Read **running** configuration, subtree-filtered to one interface.

Successful edit:

```
<rpc-reply message-id="102">
  <ok/>
</rpc-reply>
```

Interpretation: `edit-config` (or `commit/close`) succeeded. There is **no** data payload; `<ok/>` is the success signal.

Failed edit (original):

```
<rpc-reply message-id="103">
  <rpc-error>
    <error-type>application</error-type>
    <error-tag>invalid-value</error-tag>
    <error-message>Invalid leaf value</error-message>
  </rpc-error>
</rpc-reply>
```

Interpretation: The device rejected the payload. Do not claim the interface was updated.

4. Exam-style understanding

Original study items:

Item A. JSON top-level key `ietf-interfaces:interfaces` vs `Cisco-IOS-XE-native:native`.

Answer: IETF standard interface model vs Cisco native configuration tree.

Item B. RESTCONF path `interface=GigabitEthernet1` — why the equals?

Answer: YANG list key in the URL.

Item C. Distinguish `<get-config>` reply vs `<get>` reply if you see `<oper-status>up</oper-status>`.

Answer: Operational leaf $\Rightarrow$ `<get>` (or operational RESTCONF), not config-only `get-config`.

Item D. HTTP 204 after PATCH.

Answer: Success, no body; the resource was updated (or already matched).

Item E. Content-Type: application/yang-data+json on the **response**.

Answer: Body is YANG-modeled JSON, not a random product REST API.

Item F. NETCONF <hello> lists urn:ietf:params:netconf:capability:candidate:1.0.

Answer: Device supports the candidate datastore; edits can target candidate then commit.

5. Hands-on exercise

1. Compare sample_restconf_get.json and sample_netconf_get-config.xml side by side. List three leaves present in both.
2. Run labs/06_yang_netconf_restconf/restconf_get_interfaces.py against DevNet IOS XE Always-On (labs/.env). Note status code, Content-Type, and hostname JSON.
3. Run netconf_get_config.py . Copy the first eight capabilities. Identify at least one YANG module URN and whether :candidate appears.
4. In Postman: GET https://{host}/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1 with Basic auth and Accept: application/yang-data+json . Interpret the body using this section.

5.11 Interpret basic YANG models

1. What Cisco expects me to know

The verb is **Interpret**. Read a **basic YANG model** and explain the data tree. Required vocabulary: **module**, **namespace**, **prefix**, **container**, **list**, **key**, **leaf**, **leaf-list**, **config true/false**, **grouping**, **uses**, **type**. You will map a snippet of YANG to the JSON/XML you saw in 5.10.

The lab excerpt labs/06_yang_netconf_restconf/sample.yang is a teaching subset of ietf-interfaces , not the full IETF module.

YANG language: [RFC 7950](#).

2. Detailed explanation

YANG is a **schema language**. A **module** is a named schema document. Inside it, statements nest to form a tree.

Header statements:

- module ietf-interfaces — module name. RESTCONF uses this name before the colon.
- namespace "urn:ietf:..." — XML namespace URI. NETCONF XML uses this xmlns .
- prefix if; — short name used when other modules **import** this one (if:interfaces).
- yang-version 1.1; — language version.
- organization , description — metadata; rarely affect the instance data.

Data-definition statements:

Statement	Meaning	Instance encoding
container	Named grouping of child nodes, not a list. No key.	JSON object
list	Dictionary of entries, each uniquely identified by key	JSON array of objects
key	Which leaf (or leaves) uniquely identify a list row	RESTCONF =value in the path
leaf	Single value	JSON scalar
leaf-list	Ordered or unordered set of scalar values	JSON array of scalars
type	string, boolean, uint8, enumeration, inet:ipv4-address, ...	Constrains the leaf
config false	Operational state, not writable config	Appears in <get> / operational RESTCONF
config true	Default: configuration	get-config / writable RESTCONF
default	Value if the leaf is omitted	Device may still omit it in replies
mandatory true	Must be present in config	
grouping	Reusable bundle of nodes (not instantiated by itself)	
uses	Insert a grouping here	Expands like a macro

config false on a container makes the whole subtree operational. In the sample, `interfaces` is config; `interfaces-state` is `config false` (older IETF style; newer models often use a single tree with `config false` leaves). Exam: if you see `config false`, you cannot `edit-config` those leaves.

Identity of a list entry. `list interface { key "name"; leaf name { type string; } ... }` means two interfaces cannot share `name`. RESTCONF: `interface=GigabitEthernet1`. JSON must include `"name": "GigabitEthernet1"`.

Augment / import (recognize, do not author). Another module can `import ietf-interfaces` and `augment` IPv4 under each interface — that is why JSON showed `ietf-ip:ipv4` inside an interface even though `sample.yang` has no IP leaves.

Types. `enumeration` with `enum up; enum down;` means only those strings. `boolean` is `true / false`. Wrong type → NETCONF `invalid-value` or RESTCONF 400.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    M["module ietf-interfaces"]
    M --> C1["container interfaces<br/>config true"]
    M --> C2["container interfaces-state<br/>config false"]
    C1 --> L1["list interface key name"]
    L1 --> N["leaf name string"]
    L1 --> D["leaf description string"]
    L1 --> E["leaf enabled boolean"]
    C2 --> L2["list interface key name"]
    L2 --> N2["leaf name"]
    L2 --> O["leaf oper-status enumeration"]
```

3. Syntax and examples

Lab model (abridged comments):

```
module ietf-interfaces {
  yang-version 1.1;
  namespace "urn:ietf:params:xml:ns:yang:ietf-interfaces";
  prefix if;

  container interfaces {
    list interface {
      key "name";
      leaf name { type string; }
      leaf description { type string; }
      leaf enabled { type boolean; default "true"; }
    }
  }

  container interfaces-state {
    config false;
    list interface {
      key "name";
      leaf name { type string; }
      leaf oper-status {
        type enumeration {
          enum up;
          enum down;
          enum testing;
        }
      }
    }
  }
}
```

Interpretation drills:

- Module name for RESTCONF: `ietf-interfaces`.
- XML ns: `urn:ietf:params:xml:ns:yang:ietf-interfaces`.

- Writable path: `ietf-interfaces:interfaces/interface= ...`
- Read-only path conceptually: `ietf-interfaces:interfaces-state/interface= ... /oper-status`
- Can you PATCH `oper-status` ? **No** — parent is `config false`.
- Can two list entries omit `name` ? **No** — it is the key.

grouping / uses original example:

```
grouping addr-block {
  leaf ip { type string; }
  leaf prefix-length { type uint8; }
}

container ipv4 {
  list address {
    key "ip";
    uses addr-block;
  }
}
```

Interpretation: Each `address` list entry has leaves `ip` (key) and `prefix-length` because `uses addr-block` copies that grouping. `grouping` itself is not a RESTCONF resource.

leaf-list original:

```
leaf-list name-server {
  type string;
}
```

JSON instance: `"name-server": ["1.1.1.1", "8.8.8.8"]` — array of strings, not array of objects. Contrast with `list`.

4. Exam-style understanding

Original study items:

Item A. Difference between `leaf` and `leaf-list`.

Answer: One scalar vs multiple scalars.

Item B. Difference between `leaf-list` and `list`.

Answer: `list` has child nodes and a `key`; `leaf-list` is only repeated scalars.

Item C. Given `container interfaces { list interface { key "name";` which JSON is valid?

A: `"interface": { "name": "Gi1" }` as a single object with no array

B: `"interface": [ { "name": "Gi1" } ]`

Answer: B matches a YANG list (JSON array). Some encodings may vary; IETF JSON encoding of lists is an array.

Item D. prefix `if` — do RESTCONF URLs use `if:interfaces` or `ietf-interfaces:interfaces`?

Answer: RESTCONF uses the **module name**, not the prefix. Prefix is for YANG references inside modules.

Item E. Find the operational leaf in the sample.

Answer: `oper-status` under `interfaces-state`.

5. Hands-on exercise

1. Print `sample.yang` and highlight: module, namespace, prefix, two containers, two lists, keys, leaves, `config false`, types.
2. For each leaf, write one JSON snippet and one XML snippet.
3. Add on paper a `leaf-list alias { type string; }` under `list interface`. Write the JSON for two aliases.
4. Read RFC 7950 section 3 (overview) only: <https://datatracker.ietf.org/doc/html/rfc7950> — enough for CCNA Automation.

5.12 Interpret a unified diff

1. What Cisco expects me to know

The verb is **Interpret**. Read **unified diff** output (`git diff`, `diff -u`, a GitHub pull request) and say what file changed, which lines were removed or added, and where the hunk sits in the file. This is how code review (5.13) and CI (5.4) present change.

Lab file: `labs/04_git/example.diff`.

2. Detailed explanation

Unified diff is a text format. Core pieces:

```
--- a/path/to/old
+++ b/path/to/new
@@ -oldStart,oldCount +newStart,newCount @@ optional hunk header
  context line (unchanged, prefix space)
 -removed line (present in old, absent in new)
 +added line (absent in old, present in new)
```

File headers. `---` is the original (often `a/` from Git). `+++` is the new file (`b/`). If a file is created, the old side may be `/dev/null`. If deleted, the new side may be `/dev/null`.

Hunk header. `@@ -1,6 +1,7 @@` means:

- Old file: starting at **line 1**, the hunk covers **6** lines.
- New file: starting at **line 1**, the hunk covers **7** lines.

The new side has one extra line (`6 → 7`), which matches one `+` line net of removals.

Line prefixes (the first character of each hunk line):

Prefix	Meaning
space	Context — unchanged
-	Removed
+	Added
\	"No newline at end of file" marker (rare)

A **modified** line is usually shown as a `-` old version immediately followed by a `+` new version.

What a diff does not tell you by itself: whether tests passed, whether the change is a good idea, or whether it was already committed. It only shows textual change. `git diff` (unstaged) vs `git diff --cached` (staged) vs `git diff HEAD~1` (last commit) are different **ranges** with the same format.

3. Syntax and examples

Lab diff (`labs/04_git/example.diff`):

```
--- a/hostname.py
+++ b/hostname.py
@@ -1,6 +1,7 @@
  import requests

-url = "https://router/restconf/data/Cisco-IOS-XE-native:native/hostname"
+HOST = "https://router"
+url = f"{HOST}/restconf/data/Cisco-IOS-XE-native:native/hostname"
  response = requests.get(url, auth=("admin", "admin"), verify=False)
  print(response.json())
```

Interpretation:

- File: `hostname.py`.
- Hunk starts at line 1 on both sides. Old length 6, new length 7.
- Unchanged: `import requests`, blank line, `response = requests.get( ... )`, `print( ... )`.
- Removed: a hardcoded full RESTCONF URL.
- Added: `HOST = "https://router"` and an f-string URL that builds the same path from `HOST`.
- Net intent: make the base URL a variable. Auth and `verify=False` did **not** change.

Created file (original example):

```
--- /dev/null
+++ b/labs/09_terraform/generated_inventory.json
@@ -0,0 +1,3 @@
+{
+  "devices": []
+}
```

Interpret: *New file with three lines; nothing removed.*

Two hunks. A file can have @@ -10,4 +10,5 @@ later. Each hunk is independent. Do not assume the whole file is shown.

4. Exam-style understanding

Original study items:

Item A. In @@ -20,3 +20,5 @@, how many lines does the new side of this hunk include?

Answer: 5.

Item B. A hunk has two - lines and one + line. Did the file grow or shrink?

Answer: Shrink by one line in this hunk (net -1).

Item C. Does the lab diff change authentication?

Answer: No. Only the URL construction changed.

Item D. Prefix of an unchanged import requests line?

Answer: A space character (often easy to miss in a question stem).

Item E. --- and +++ both say hostname.py but a/ vs b/.

Answer: Git's way of labeling old vs new tree; same path.

5. Hands-on exercise

1. Read labs/04_git/example.diff and narrate it without looking at this section.
2. In a scratch clone, change one Python file, run git diff, and label every @@ number by opening the file and counting.
3. On GitHub, open any pull request, switch to Files changed, and interpret one hunk. That is the same format as the exam.

5.13 Describe the principles and benefits of a code review process

1. What Cisco expects me to know

The verb is **Describe**. Explain **principles** of code review and **benefits**, especially for infrastructure automation (playbooks, Terraform, Python API scripts). You already know Git from Domain 1; here the human process around a diff (5.12) and a pipeline (5.4) is the point.

2. Detailed explanation

A **code review** is a structured look at a proposed change **before** it becomes the shared main branch (and before it hits production devices). Typical GitHub flow: branch → commit → pull request → reviewers comment on the unified diff → CI runs → approve → merge.

Principles:

1. **Small, focused changes.** A PR that rewrites Ansible and Terraform and a YANG model is hard to review. Prefer one workflow per PR.
2. **The diff is the artifact.** Reviewers read 5.12 output, not a slide deck. “Trust me I ran it” is not a review.
3. **Automation first, humans second.** Linters, `terraform plan`, `pytest`, `ansible-lint`, secret scanners run in CI so humans do not nitpick formatting or catch a hardcoded API key late.
4. **Correctness, safety, and operability.** For network IaC, reviewers ask: idempotent? wrong inventory? `become` too wide? `rm -rf ? verify=False` acceptable only in a lab? blast radius?
5. **Authorship vs approval.** The author does not approve their own change in a disciplined team. At least one other engineer (or a maintained CODEOWNERS file) looks.
6. **Comments are about the code, and they should be actionable.** “This `get-config` filter is unscoped and will pull the entire running config” is a review. “I don’t like YAML” is not.
7. **Recorded outcome.** Approve, request changes, or comment. The Git platform stores who signed off — audit benefit.
8. **Secrets never merge.** Reviewers watch for API keys in diffs. `.env` must stay untracked.

Benefits:

- **Catch bugs** that unit tests miss (wrong Catalyst Center path, Meraki org ID hardcoded).
- **Share knowledge** — junior engineers see how RESTCONF URLs are built.
- **Consistency** with team style and IaC principles (5.5).
- **Safety** for production networks: two people agree before BGP changes.
- **Audit and compliance** — who approved the change to firewall rules.
- **Better CI** — reviewers demand tests that then keep working.

Code review does not replace CML/pyATS (5.3) or CI (5.4). It sits in the pipeline as a gate: **green tests + human approval**.

3. Syntax and examples

There is no single “code review language.” You will see GitHub/GitLab UI. A review comment anchored on a diff line (original example):

```
playbook.yml:22
Reviewer: create_home: true is good. Please also set
groups: sudo
only if this user must become root; otherwise omit it (least privilege).
```

A PR checklist (original):

- [] `ansible-lint` and `terraform plan` are green
- [] No secrets in the diff
- [] Inventory targets lab, not prod
- [] pyATS or RESTCONF GET assertion described

`git log` after merge shows the PR; `git blame` shows who last touched a line — benefits of a reviewed history.

4. Exam-style understanding

Original study items:

Item A. Name three benefits of code review for Ansible playbooks.

Answer: Catch destructive tasks, share module knowledge, ensure inventory/scope, require CI, audit who approved.

Item B. Why review Terraform `plan` output as well as HCL?

Answer: The plan is the real create/destroy set; HCL can hide blast radius until plan.

Item C. Is “the pipeline is green” a substitute for review?

Answer: No. CI cannot judge intent (“this VLAN is the wrong site”). Both.

Item D. Principle most closely tied to 5.12?

Answer: Review the unified diff, line by line.

5. Hands-on exercise

1. Push a branch that only changes a comment in `labs/08_ansible/playbook.yml`. Open a GitHub pull request. Request review from a classmate or review it yourself as if you were a second engineer — write two comments using the principles above.
2. Intentionally put a fake key `X-Cisco-Meraki-API-Key: 12345` in a branch (never a real key). Practice catching it in the diff, then remove it before merge.
3. Read `labs/04_git/workflow.yml` and mark where “open PR / review” belongs relative to clone, commit, merge, push.

5.14 Interpret a sequence diagram that includes API calls

1. What Cisco expects me to know

The verb is **Interpret**. Given a **sequence diagram** that includes **API calls**, identify participants, message order, what is synchronous vs asynchronous, and what comes back. Domain 2 webhooks and Domain 3 controllers show up here as diagrams rather than as Python.

You should be able to **read** UML/Mermaid sequence diagrams: lifelines, arrows, return messages, optional loops, notes.

2. Detailed explanation

A sequence diagram shows **time going down** and **participants going across**. Each vertical line is a **lifeline** (client, API gateway, Catalyst Center, device, webhook listener).

Arrow conventions (UML-ish, also used in Mermaid):

Arrow	Typical meaning
Solid line with filled/open arrowhead to the right	Synchronous call (HTTP request). Caller waits.
Dashed line back	Return / HTTP response.
Open arrow / async message	Fire-and-forget or “response comes later” (webhook, job queued).
Box over a lifeline	Activation — that participant is processing.
<code>alt</code> / <code>opt</code> / <code>loop</code> frames	Conditional or repeated calls (retries, pagination).

How to interpret an API sequence:

1. **Who speaks first?** Usually the automation client.
2. **Is there an auth call before the business call?** Catalyst Center token POST before GET devices; ACI `aaaLogin` before class query; Meraki often has **no** extra auth call (key on every request).
3. **Synchronous REST?** Client → server → dashed return with 200 and JSON. The next arrow should not start until the return (unless the diagram shows parallel).
4. **Asynchronous?** Client POST returns 202 + task id; later the client polls, **or** the server calls the client’s webhook URL. Those are different participants.
5. **Error path** in an `alt` box: 401 then re-auth; 429 then retry.

Participant order is not arbitrary: left-to-right often follows the request path (user, script, controller, device). If a webhook appears, the **device or controller becomes the HTTP client** and your listener is the server — arrows reverse.

3. Syntax and examples

Synchronous RESTCONF GET (device-level, matches lab 06):

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    autonumber
    participant Py as Python requests
    participant XE as IOS XE RESTCONF :443
    Py->>XE: GET /restconf/data/ietf-interfaces:interfaces<br/>Accept: application/yang-data+json
    XE-->>Py: 200 yang-data+json interface list
```

Interpret: *One synchronous API call. No controller. Return is the YANG JSON.*

Catalyst Center token then inventory (sync):

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    autonumber
    participant Py as Script
    participant CC as Catalyst Center
    Py->>CC: POST /dna/system/api/v1/auth/token (Basic)
    CC->>Py: 200 { Token }
    Py->>CC: GET /dna/intent/api/v1/network-device<br/>X-Auth-Token
    CC->>Py: 200 { response: [ devices ] }
```

Interpret: *Two synchronous calls. Workflow is authenticate, then list devices. Token is not a webhook.*

Asynchronous intent + webhook (original):

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    autonumber
    participant Py as Script
    participant CC as Catalyst Center
    participant WH as Your webhook HTTPS endpoint
    Py->>CC: POST template deploy
    CC->>Py: 202 { taskId }
    Note over CC: Job runs on devices
    CC->>WH: POST /hooks/dnac-task (event JSON)
    WH->>CC: 204
    Py->>CC: GET task by taskId
    CC->>Py: 200 SUCCESS
```

Interpret: *Deploy is asynchronous (202). Controller later **calls you** (webhook — reverse HTTP). Script may also poll the task. Three participants.*

NETCONF vs REST in one picture:

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant N as ncclient
    participant D as Device :830
    N->>D: hello
    D->>N: hello + capabilities
    N->>D: get-config running
    D->>N: rpc-reply data
```

Interpret: *Capability exchange is mandatory before RPCs. Returns are XML `rpc-reply`, not HTTP status codes.*

4. Exam-style understanding

Original study items:

Item A. In a diagram, the second arrow is dashed from APIC to Python after `aaaLogin`. What is that message?

Answer: Login response (token/cookie). Next arrows should use it.

Item B. Arrow from Meraki cloud to `https://example.com/hook` with no prior dashed return wait.

Answer: Webhook (asynchronous event); Meraki is the HTTP client.

Item C. Why does a Catalyst Center diagram have more participants than a RESTCONF GET?

Answer: Controller plus devices (and maybe a webhook receiver); RESTCONF is client plus one box.

Item D. `loop` frame around GET with `offset`.

Answer: Pagination until the API returns no more rows.

Item E. Participant order: Client, Firewall, API. A SYN to port 443 dies at the firewall.

Answer: Sequence never reaches the API — useful when combined with 6.8 (blocked port).

5. Hands-on exercise

1. Draw (Mermaid or paper) the sequence for `labs/05_cisco_apis/meraki_list_devices.py`: `orgs` → `devices` → `clients`. All synchronous.
 2. Draw `catalyst_center_devices.py`: `token`, then `devices`, then optional `client-health`.
 3. Draw `netconf_get_config.py`: `hello`, `capabilities print`, `get-config`, `disconnect`.
 4. Add one webhook participant to the Catalyst Center diagram hypothetically. Label which arrows reverse direction.
-

6.0 Network Fundamentals — 15%

Network fundamentals **are on this exam**. Cisco is not asking you to re-sit the entire 200-301 CCNA curriculum. The depth here is: **purpose and usage** of addressing and VLANs, **function** of common boxes and planes, **interpret** a simple topology, **recognize** well-known ports, **diagnose** a short list of application connectivity failures, and **explain** how network constraints show up in applications.

If you already hold CCNA, use this domain as a precision review aimed at automation engineers: you must still map RESTCONF :443 vs NETCONF :830, explain why a Python script sees a load-balancer IP, and know why DNS failure looks like “the API is down.”

Lab files: `labs/10_network_troubleshooting/diagnose.py` for 6.8/6.9.

6.1 Describe the purpose and usage of MAC addresses and VLANs

1. What Cisco expects me to know

The verb is **Describe** the **purpose and usage** of **MAC addresses** and **VLANs**. You need the ideas an automation engineer uses when reading inventory JSON, Meraki client lists, or switch interface YANG — not spanning-tree timer math.

2. Detailed explanation

MAC address (Media Access Control). A 48-bit Layer 2 identifier burned into (or assigned to) a NIC, usually written `00:11:22:33:44:55` or `0011.2233.4455`. Purpose: uniquely identify an interface on a **local Ethernet segment** so switches can forward frames. Usage you will see in APIs:

- Meraki client objects include `mac` and often `ip`.
- Catalyst Center device inventory includes MAC / serial.
- Switch CAM/MAC tables map MAC → port. Automation that “finds a host” is often a MAC lookup, not a traceroute.

MACs do not route across L3. A packet that leaves a router gets a **new** Ethernet header with the next-hop MAC. If a script shows the same IP with a changing MAC, you may be looking at HSRP/VRRP virtual MACs or a VM that vMotioned.

VLAN (Virtual LAN). A VLAN is a **broadcast domain** carved out of a switch fabric. Purpose: isolate groups of endpoints (users vs cameras vs servers) without separate physical switches. Usage:

- Access port: one VLAN (untagged) for a PC or AP.
- Trunk port: multiple VLANs tagged with 802.1Q between switches, firewalls, and hypervisors.
- SVI (switched virtual interface): the Layer 3 gateway IP for that VLAN on a Layer 3 switch.

From an automation perspective, “create VLAN 20 and put Gi1/0/12 in it” is a device-level or controller-level change. ACI does not use campus VLANs the same way (EPGs), but campus Catalyst still does.

How they work together. Frames inside VLAN 20 are switched using MAC tables **scoped to that VLAN**. Hosts in VLAN 20 cannot talk to VLAN 30 at Layer 2; a router/SVI/firewall must route (and maybe filter) between them.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    PC1["PC MAC A VLAN 20"] --> SW["Switch"]
    PC2["PC MAC B VLAN 20"] --> SW
    CAM["Phone MAC C VLAN 30"] --> SW
    SW --> SVI["SVI VLAN 20 .1 and VLAN 30 .1"]
```

3. Syntax and examples

JSON you might see from a controller (original):

```
{
  "hostname": "access-01",
  "managementIpAddress": "10.10.20.11",
  "macAddress": "00:1e:14:a1:b2:c3",
  "interface": "GigabitEthernet1/0/12",
  "vlanId": 20
}
```

Interpret: *This device’s mgmt IP is L3; the endpoint or port is in VLAN 20; MAC identifies the chassis or the client depending on the API.*

IOS-XE style (conceptual, not a lab file):

```
interface GigabitEthernet1/0/12
 switchport mode access
 switchport access vlan 20
```

```
interface Vlan20
 ip address 10.20.0.1 255.255.255.0
```

Purpose: port in VLAN 20; SVI is the gateway for that subnet.

4. Exam-style understanding

Original study items:

Item A. Purpose of a MAC vs purpose of an IP.

Answer: MAC identifies an L2 interface on a local segment; IP identifies an L3 endpoint across networks.

Item B. Two PCs in VLAN 20 and VLAN 30, same switch, no SVI. Can they ARP each other?

Answer: No. Different broadcast domains.

Item C. Meraki client list has `mac` but `ip` is null.

Answer: L2-seen device without a learned DHCP/IP, or the timespan missed it.

Item D. Why does a RESTCONF interface list not contain MAC by default in `ietf-interfaces` config?

Answer: MAC is often operational hardware state (`config false`), not something you set on a physical NIC.

5. Hands-on exercise

1. On WSL, run `ip link` (or Windows `getmac`). Identify one MAC. Note it is local to that interface.
2. In the Meraki lab script output (`meraki_list_devices.py`), find MAC-like fields on devices if you have a sandbox.
3. Sketch two VLANs and one trunk. Label which links are tagged.

6.2 Describe the purpose and usage of IP addresses, routes, subnet mask / prefix, and gateways

1. What Cisco expects me to know

Describe purpose and usage of IP addresses, routes, subnet mask / prefix, and gateways. Enough to read a topology, a RESTCONF `ietf-ip` block, and a failed API call that never left the subnet. Not enough to design VLSM for a global enterprise.

2. Detailed explanation

IP address. Layer 3 identifier. IPv4 is 32-bit (`10.10.20.48`); IPv6 is 128-bit. Purpose: routing between networks, not switching inside a VLAN. Usage in automation: every device inventory field `managementIpAddress` , RESTCONF `ietf-ip:ipv4 ip` , Meraki `lanIp` .

Subnet mask / prefix. The mask (`255.255.255.0`) or prefix length (`/24`) splits the address into **network** and **host**. Purpose: decide “is this destination local (ARP/ND) or remote (send to a gateway)?” Usage: you will see either form. RESTCONF sample uses `netmask` ; many APIs use `prefix-length` . `/24` means 256 addresses, 254 usable hosts in classic IPv4 (network and broadcast reserved). `/32` is a single host route. `/128` is a single IPv6 address.

Gateway (default gateway). The IP on the **local subnet** that forwards packets toward other networks — typically the SVI, router, or firewall interface. A host with IP `10.10.20.48/24` and gateway `10.10.20.1` ARPs for `.1` when the destination is not `10.10.20.0/24` . Wrong gateway is a classic “API times out” cause: the packet never leaves the VLAN.

Routes. A route is a mapping: destination prefix → next hop (or exit interface). Devices have:

- Connected routes (the subnet on an interface).

- Static routes (operator or IaC defined).
- Dynamic routes (OSPF, BGP, EIGRP) installed by the **control plane** (6.5).

A default route `0.0.0.0/0` is “everything else.” Automation scripts on a jump host still use the host OS routing table. If you can curl Meraki cloud but not an APIC at `10.0.0.10`, think routes/VPN/firewall before thinking Python.

Longest prefix match. If `10.10.20.0/24` and `10.10.0.0/16` both exist, a packet to `10.10.20.48` uses `/24`. You do not need to compute huge tables; you need to know **more specific wins**.

3. Syntax and examples

From `sample_restconf_get.json`:

```
"ietf-ip:ipv4": {
  "address": [
    { "ip": "10.10.20.48", "netmask": "255.255.255.0" }
  ]
}
```

Purpose: this interface is in `10.10.20.0/24`. Hosts in that subnet use a gateway (not shown in this snippet) to reach other networks.

Linux host config (WSL):

```
ip addr show
ip route show
```

Typical default route line: `default via 10.10.20.1 dev eth0`. That `via` is the gateway.

Terraform lab inventory (`main.tf`) stores `mgmt = "10.10.20.48"` — usage: management IP for automation to target, not a route by itself.

Static route conceptual CLI:

```
ip route 10.30.0.0 255.255.255.0 10.10.20.1
```

Interpret: *To reach 10.30.0.0/24, send to next hop 10.10.20.1 (must be reachable on a connected subnet).*

4. Exam-style understanding

Original study items:

Item A. Mask `255.255.255.0` as prefix?

Answer: `/24`.

Item B. Host `10.10.20.48/24`, destination `10.10.20.49`. Gateway used?

Answer: No. Same subnet — ARP for the destination MAC.

Item C. Same host, destination `8.8.8.8`.

Answer: Send to default gateway.

Item D. Purpose of a route vs purpose of a VLAN.

Answer: Route forwards between L3 networks; VLAN segments L2 broadcast domains.

Item E. RESTCONF shows IP but ping to another subnet fails. First IP question?

Answer: Is there a gateway/route, and is the mask correct so the destination is actually “remote”?

5. Hands-on exercise

1. On Windows: `ipconfig / route print`. Identify your IPv4, mask, and default gateway.
 2. Convert `255.255.255.0` and `255.255.255.252` to prefix lengths (`/24` , `/30`).
 3. Read `labs/09_terraform/main.tf` device `mgmt` addresses. State the `/24` they appear to share (`10.10.20.0/24`).
 4. Optional: `python labs/01_python_basics/test_subnet.py` if it encodes subnet checks from Domain 1.
-

6.3 Describe the function of common networking components (such as switches, routers, firewalls, and load balancers)

1. What Cisco expects me to know

Describe the function of switches, routers, firewalls, and load balancers. Four boxes, four jobs. Enough to place them on a diagram (6.4) and know which one your API script is talking to.

2. Detailed explanation

Switch (Layer 2 / multilayer). Forwards Ethernet frames using MAC tables (and VLAN context). A **Layer 3 switch** also routes between SVIs. Function in a campus: connect endpoints, enforce VLANs, often PoE for APs/phones. Automation: high device count, Catalyst Center inventory, RESTCONF on IOS XE.

Router. Forwards **packets** between IP networks using the routing table. Function: WAN edge, branch internet, interconnecting VLANs when not done on an L3 switch. NAT (6.6) often lives here. Automation: fewer boxes, richer YANG, SD-WAN vManage for many edges.

Firewall. Forwards or **drops** according to policy (5-tuple, application, user). Function: security boundary — internet edge, data-center, micro-segmentation. Stateful firewalls track TCP sessions. Automation: policy APIs (FMC, FTD, ASA, cloud SG) more than “enable interface.” On a topology, a firewall in the path explains **blocked ports** (6.8).

Load balancer (ADC). Distributes connections across a pool of servers using a **virtual IP (VIP)**. Function: scale, health checks, SSL offload, sometimes URL routing. It is a **NAT-ish** device from the server’s point of view (source NAT or destination NAT). Automation: Terraform loves load-balancer objects in cloud; on-prem F5/NGINX/Cisco still show up as a hop that changes IP and port.

Related boxes you may see on diagrams: **AP** (wireless edge, still a MAC/VLAN story), **proxy** (6.8), **WAN circuit**. Stick to the four named functions unless the diagram labels more.

3. Syntax and examples

Component	Forwards based on	Typical “management” API
Switch	MAC + VLAN	Catalyst Center, RESTCONF, Meraki
Router	IP prefix / next hop	RESTCONF, SD-WAN vManage
Firewall	Policy / state	FMC, Meraki MX, cloud SG
Load balancer	VIP, health, algorithm	Cloud LB Terraform resource, REST

Original health-check idea (load balancer):

```
VIP 203.0.113.10:443 → pool 10.20.0.11:8080, 10.20.0.12:8080
```

Function: clients hit one IP; two app servers share load. The app may log the LB IP as the client (6.8 NAT).

4. Exam-style understanding

Original study items:

Item A. Which component separates VLAN 20 and VLAN 30 at Layer 3?

Answer: Router or L3 switch (SVI), possibly a firewall if that is the gateway.

Item B. Which component drops TCP/830 from the internet?

Answer: Firewall (or ACL on a router — exam “such as” list highlights firewall).

Item C. Which component owns a VIP?

Answer: Load balancer.

Item D. Meraki MS vs MX vs MR — functions?

Answer: Switch, security/appliance (firewall/NAT/VPN), wireless AP. Still the same four ideas.

5. Hands-on exercise

1. Label each icon on the 6.4 mermaid diagram before you read 6.4’s interpretation.
 2. In `diagnose.py`, note which symptoms implicate a firewall vs a load balancer vs a VPN gateway.
-

6.4 Interpret a basic network topology diagram with elements such as switches, routers, firewalls, load balancers, and port values

1. What Cisco expects me to know

The verb is **Interpret**. Read a **basic** diagram: boxes, links, **port values** (Gi0/0, port 443, VLAN tags). Say what talks to what, which device is L2 vs L3 vs security vs VIP, and which **TCP/UDP port** a client would use. Combine 6.3 + 6.7.

2. Detailed explanation

Method:

1. **Name every node** (switch, router, firewall, LB, host, controller).
2. **Follow a packet** from a labeled client to a labeled service. Each hop: switched (MAC), routed (IP), filtered (firewall), or NATed (LB/firewall).
3. **Read interface IDs** (Gi0/1 , eth0) as *which cable*, not as TCP ports.
4. **Read numbers like :443 or tcp/830** as **transport ports** (6.7). A common exam mix-up is treating GigabitEthernet1 as “port 1/tcp.”
5. **Management path vs data path**. A Python script may reach IOS XE mgmt 10.10.20.48:443 (RESTCONF) while user traffic uses a different VLAN through the LB. Diagrams sometimes draw both.

Port values on diagrams mean two different things — always disambiguate:

- **Interface names:** GigabitEthernet1, GigabitEthernet1/0/24, port-channel 1.
- **Transport ports:** 22, 443, 830, 161.

3. Syntax and examples

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    User[User PC<br/>10.20.0.50] --> SW[Access switch<br/>VLAN 20]
    SW --> FW[Firewall<br/>tcp/443 allow<br/>tcp/830 deny]
    FW --> LB[Load balancer<br/>VIP 203.0.113.10:443]
    LB --> App1[App 10.20.1.11:8080]
    LB --> App2[App 10.20.1.12:8080]
    Ops[Ops laptop] --> RTR[Edge router Gi0/0<br/>10.10.20.1]
    RTR --> XE[IOS XE Gi1<br/>10.10.20.48<br/>RESTCONF :443<br/>NETCONF :830]
```

Interpretation of this original diagram:

- User HTTP(S) path: PC → access switch (VLAN 20) → firewall (allows 443, **denies NETCONF 830** from this zone) → load balancer VIP on **tcp/443** → backend **8080**.
- Ops automation path: laptop → edge router Gi0/0 → IOS XE GigabitEthernet1 at 10.10.20.48. RESTCONF uses **443**; NETCONF uses **830**. Interface Gi1 is not port 1/tcp.

- The firewall in the *user* path does not automatically apply to the *mgmt* path unless the diagram shows it that way.

ASCII variant you might see on an exam item:

```
[PC]--Gi1/0/12--[SW]--trunk--[RTR Gi0/0]--Gi0/1--[FW]--[LB :443]--[APP :8080]
```

Interpret left to right; `Gi1/0/12` is an access interface; `:443` is HTTPS on the VIP.

4. Exam-style understanding

Original study items:

Item A. Where is HTTP terminated in the mermaid diagram?

Answer: Often at the LB (SSL offload) or the app; the VIP is `:443`. Backends are `:8080`.

Item B. Can the user PC open NETCONF to IOS XE through the firewall as drawn?

Answer: No. `tcp/830 deny` on that firewall path.

Item C. `Gi0/0` on the router — Layer 2 port number 0 or an interface name?

Answer: Interface name.

Item D. How many Layer 3 hops from User PC to App1?

Answer: At least switch (L2) then firewall/LB as L3 hops depending on whether the SW is L2-only. Count routed boundaries: PC subnet → FW/LB networks → app subnet.

5. Hands-on exercise

1. Cover the interpretation bullets, redraw the mermaid from memory, then check.
2. Add Catalyst Center as a cloud box connected only to the ops laptop and the IOS XE mgmt network. State which APIs are controller-level vs device-level (5.2).
3. Walk `diagnose.py` cases against this diagram (blocked 830, NAT at LB).

6.5 Describe the function of management, data, and control planes in a network device

1. What Cisco expects me to know

Describe the function of the **management**, **data**, and **control** planes. This is how you explain why RESTCONF can work while traffic is blackholing, or why a CPU spike (control plane) is different from an ACL drop (data plane).

2. Detailed explanation

Data plane (forwarding plane). Moves frames/packets through the device as fast as possible, usually in hardware (ASIC). Function: switching, routing lookups already programmed, ACL/QoS applied to transit traffic. If a user packet from VLAN 20 to the VIP is forwarded, that is data plane. Automation does **not** ride the data plane except when your API call is itself transit traffic through another box.

Control plane. Builds and maintains the **tables** the data plane uses: OSPF/BGP/EIGRP, ARP/ND, MAC learning, spanning tree. Function: “how should we forward?” not “forward this packet.” A BGP neighbor drop is a control-plane event; existing packets may still follow last-known hardware until the FIB updates. pyATS “learn bgp” is largely control-plane state.

Management plane. How humans and **software** operate the device: SSH, HTTPS GUI, RESTCONF, NETCONF, SNMP, gRPC telemetry config. Function: configure and observe. CPU of the management process is separate from forwarding ASICs on purpose (control-plane policing exists because mixing them is dangerous).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph mgmt [Management plane]
        SSH[SSH :22]
        RC[RESTCONF :443]
        NC[NETCONF :830]
        SNMP[SNMP :161]
    end
    subgraph ctrl [Control plane]
        BGP[BGP / OSPF]
        ARP[ARP / ND]
        RIB[RIB / LSDB]
    end
    subgraph data [Data plane]
        FIB[FIB / CAM]
        ASIC[Forward packets]
    end
    mgmt -->|writes config| ctrl
    ctrl -->|programs| data
```

Why automation cares. Your Python `requests` session is **management plane**. It can succeed (200 on RESTCONF) while the **data plane** drops user traffic (ACL, down interface at line protocol, wrong VLAN). Conversely, a device can forward packets while management is unreachable (VTY ACL, VRF isolation, HTTPS disabled). Troubleshoot the plane that matches the symptom.

NETCONF vs BGP. Both can use TCP, but BGP is control plane between routers; NETCONF is management plane from an orchestrator. Do not call NETCONF “control plane” on this exam.

3. Syntax and examples

Plane	Example protocol / feature	Exam association
Management	SSH, RESTCONF, NETCONF, SNMP, HTTPS GUI	Your Ansible/Python session
Control	OSPF hellos, BGP, ARP	Neighbor up/down, routing table
Data	Switching a frame, CEF/FIB forward	User application packets

Original symptom mapping:

- `ncclient` times out to :830, users still browse the VIP → **management** path broken (ACL, VRF, NETCONF not enabled).
- RESTCONF GET works, users fail → **data** path (NAT, blocked 443 through FW, wrong gateway).
- BGP down, some prefixes missing → **control** plane; data plane forwards remaining routes.

4. Exam-style understanding

Original study items:

Item A. Which plane does Ansible SSH use?

Answer: Management.

Item B. Which plane computes OSPF shortest path?

Answer: Control.

Item C. Which plane applies an interface ACL to transit HTTP?

Answer: Data.

Item D. CoPP (control-plane policing) protects which plane from being flooded?

Answer: Control (and often management sharing the CPU) — so forwarding ASICs keep working.

5. Hands-on exercise

1. Three-column table: list every protocol in 6.6 and 6.7 into a plane. NTP/SNMP/SSH are management (NTP is often classed management; SNMP too). BGP is control. User HTTPS through a switch is data.
2. Using the 6.4 diagram, color mgmt path vs user data path.

6.6 Describe the functionality of these IP Services: DHCP, DNS, NAT, SNMP, NTP

1. What Cisco expects me to know

Describe the functionality of **DHCP, DNS, NAT, SNMP, NTP**. What problem each solves, who talks to whom, and how a failure looks to an application or an automation script. Ports appear again in 6.7.

2. Detailed explanation

DHCP (Dynamic Host Configuration Protocol). Function: automatically assign IP address, mask, gateway, and DNS servers to clients. Server (or relay) hears a broadcast; client configures the lease. Without DHCP, a PC may have a 169.254.x.x APIPA address and cannot reach the API. Automation devices often use **static** mgmt IPs instead, so DHCP matters more for clients (Meraki client lists) than for the IOS XE sandbox.

DNS (Domain Name System). Function: resolve names (`api.meraki.com` , `devnetsandboxiosxe.cisco.com`) to IP addresses. Applications and `requests` call DNS **before** TCP. DNS failure looks like “the API is down” but `curl https://8.8.8.8` might still work. Split-horizon DNS and corporate suffixes matter on VPN (6.8).

NAT (Network Address Translation). Function: rewrite IP (and often port) so private hosts share a public address or so overlapping networks can communicate. PAT/overload maps many insides to one outside. Destination NAT / VIP on a load balancer rewrites the destination. **Impact:** servers see the firewall/LB IP, not the client (see `diagnose.py`). Automation ACLs that allow “the client IP” may be wrong.

SNMP (Simple Network Management Protocol). Function: poll devices for operational data (`GET` on UDP **161**) and receive **traps/informs** (UDP **162**). Still widely used for monitoring; model-driven telemetry is the newer path. SNMPv2c uses community strings (weak); SNMPv3 adds crypto. Ansible/Python may still `snmpget` ; it is not RESTCONF.

NTP (Network Time Protocol). Function: synchronize clocks (UDP **123**). Certificates, log correlation, Kerberos, and `expiring` API tokens all assume reasonable time. A device years off will fail TLS to Catalyst Center. Automation pipelines that log across hops need NTP.

3. Syntax and examples

Service	Function in one line	Typical port
DHCP	Assign IP/mask/gateway/DNS	UDP 67 server, 68 client
DNS	Name → IP	UDP/TCP 53
NAT	Rewrite addresses/ports	N/A (feature, not a listener)
SNMP	Monitor/query/trap	UDP 161 / 162
NTP	Time sync	UDP 123

Original NAT illustration:

```
Client 10.20.0.50:51000 → (PAT) → 203.0.113.5:40001 → Internet API
App sees 203.0.113.5, not 10.20.0.50
```

`/etc/resolv.conf` or Windows DNS server list is the DNS **usage** on the automation host.

4. Exam-style understanding

Original study items:

Item A. Script fails with `NameResolutionError` but ping to a known IP works. Which service?

Answer: DNS.

Item B. App logs show all users as `10.0.0.1`. Which service/feature?

Answer: NAT (or LB SNAT).

Item C. Device TLS to controller fails; clock is 2019. Which service?

Answer: NTP (time).

Item D. Monitoring platform has no interface counters; RESTCONF works. Which service might be filtered?

Answer: SNMP 161.

Item E. New laptop has no IP on a DHCP VLAN.

Answer: DHCP (or relay, or port in wrong VLAN — 6.1).

5. Hands-on exercise

1. `nslookup api.meraki.com` (or `Resolve-DnsName`). That is DNS functionality.
 2. Read cases in `labs/10_network_troubleshooting/diagnose.py` that mention NAT.
 3. On a home router GUI, find DHCP pool, DNS, NAT/PAT, and NTP — map each to this table.
-

6.7 Recognize common protocol port values (such as, SSH, Telnet, HTTP, HTTPS, and NETCONF)

1. What Cisco expects me to know

The verb is **Recognize**. Memorize the well-known **TCP/UDP port numbers** the blueprint calls out and the related automation/ops ports listed below. The official “such as” list is **SSH, Telnet, HTTP, HTTPS, and NETCONF**. You should also recognize RESTCONF, SNMP, NTP, DNS, and DHCP because 6.6 and the rest of the exam use them constantly.

2. Detailed explanation

A **port** is a 16-bit transport demux value. TCP 443 and UDP 443 are different sockets; we still say “HTTPS is 443/tcp.” Firewalls (6.3, 6.8) allow or deny **these** numbers. Your Python `requests` library uses 443 by default for `https://`; `ncclient` defaults to **830**.

Telnet vs SSH. Telnet **23** is cleartext and should be disabled; SSH **22** is the management-plane remote CLI. Ansible’s default is SSH 22.

HTTP vs HTTPS vs RESTCONF. HTTP **80** is cleartext web. HTTPS **443** is TLS web. **RESTCONF typically runs on 443** with `yang-data+json`. Do not confuse RESTCONF with NETCONF: different protocol, different port.

NETCONF is 830/tcp over SSH. If a topology allows 443 but not 830, RESTCONF might work while `nccClient` fails.

Well-known numbers are IANA defaults. Operators can rebind (`ip http secure-port 8443`). Exam items almost always use defaults unless the diagram labels otherwise.

3. Syntax and examples

Protocol	Port	Transport	Plane / use
SSH	22	TCP	Management CLI, Ansible, Git over SSH
Telnet	23	TCP	Legacy cleartext CLI — avoid
HTTP	80	TCP	Unencrypted web / some APIs (lab only)
HTTPS	443	TCP	Web, most REST APIs, RESTCONF default
NETCONF	830	TCP	YANG XML RPCs over SSH
RESTCONF	443 (typical)	TCP	YANG over HTTPS
DNS	53	UDP (TCP for large)	Name resolution
DHCP	67 / 68	UDP	Server / client
NTP	123	UDP	Time
SNMP	161 / 162	UDP	Get / trap

`labs/.env.example` already encodes the IOS XE defaults: `IOSXE_SSH_PORT=22` , `IOSXE_NETCONF_PORT=830` , `IOSXE_RESTCONF_PORT=443` .

Python recognition:

```
manager.connect(host=host, port=830, ...)      # NETCONF
requests.get("https://host/restconf/data/ ... ") # HTTPS 443 implied
```

4. Exam-style understanding

Original study items:

Item A. Match: 22, 23, 80, 443, 830.

Answer: SSH, Telnet, HTTP, HTTPS, NETCONF.

Item B. RESTCONF blocked but SSH to the same IP works. Which port to check besides 22?

Answer: 443 (HTTPS/RESTCONF), not 830 unless they asked NETCONF.

Item C. `ncclient` vs `requests` default ports.

Answer: 830 vs 443.

Item D. SNMP poll vs trap ports.

Answer: 161 vs 162.

Item E. Is GigabitEthernet0/0 “port 0”?

Answer: No — interface name (6.4).

5. Hands-on exercise

1. Recite the table until you can fill it blank.
2. In `netconf_get_config.py` and `restconf_get_interfaces.py`, highlight the port numbers.
3. Optional: from WSL `nc -vz devnetsandboxiosxe.cisco.com 830` and `443` when the always-on sandbox is up (do not scan random internet hosts).

6.8 Diagnose application connectivity issues (NAT problem, Transport Port blocked, proxy, and VPN)

1. What Cisco expects me to know

The verb is **Diagnose**. Given symptoms, pick among the blueprint’s four causes: **NAT problem**, **transport port blocked**, **proxy**, and **VPN**. Use a method, not guesswork. The lab

`labs/10_network_troubleshooting/diagnose.py` is the cheat-sheet in code form.

You are not doing full packet-tracer expert troubleshooting. You are mapping **application** failures (browser, `requests`, API SDK) to those four network causes.

2. Detailed explanation

Method. Ask, in order:

1. **DNS** — does the name resolve? (If not, it may still *look* like VPN split-tunnel or proxy PAC, but start here.)
2. **TCP handshake** — SYN-ACK on the destination **port**? If SYN with no SYN-ACK, think **transport port blocked** (firewall, SG, ip http not running, wrong port 830 vs 443).
3. **TLS** — handshake stall or cert error: often **proxy** SSL inspection or clock/NTP.
4. **Application data** — TCP works but app sees wrong client IP or cannot callback: **NAT**. Works off-net but not on corporate Wi-Fi: **proxy**. Works on LAN but not over tunnel, or specific subnets fail: **VPN** overlap or missing interesting traffic.

NAT problem. Source NAT hides the client; destination NAT/VIP hides the server’s real IP. Symptoms from the lab: browser reaches the VIP but the app logs the load-balancer IP; return traffic fails because NAT state expired; overlapping RFC1918 on both sides of a firewall. First check: compare app logs vs real client; `X-Forwarded-For`; packet capture source IPs.

Transport port blocked. Firewall/SG/ACL drops a port. Symptom: SYN, no SYN-ACK, capture shows no return. First check: confirm the **actual** port (80/443/830/22) is allowed **in the path drawn on the topology**. RESTCONF vs NETCONF mix-ups belong here.

Proxy. Corporate HTTP/HTTPS proxy intercepts or requires `HTTP_PROXY / HTTPS_PROXY`. Symptom: works on home network, fails on corp Wi-Fi; TLS stall to `api.example.com`; Python `requests` works if you set `verify` to the corp CA bundle. First check: proxy env vars, PAC file, trust store. Explicitly **not** a YANG error.

VPN. IPsec/SSL VPN. Two classic failures:

- **Overlap:** both sites use `10.10.20.0/24`. Interesting traffic never encrypts correctly or return routes to the local LAN.
- **Split tunnel / ACL:** only some prefixes go over the VPN; the API's prefix does not.

First check: proxy IDs / encryption domains / local vs remote subnets.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
flowchart TD
    Fail[App cannot connect] --> DNS{Name resolves?}
    DNS -->|no| DnsFix[DNS / VPN DNS / proxy]
    DNS -->|yes| TCP{SYN-ACK on port?}
    TCP -->|no| Blocked[Transport port blocked]
    TCP -->|yes| TLS{TLS OK?}
    TLS -->|no| Proxy[Proxy / inspection / NTP]
    TLS -->|yes| App{Wrong IP or only on-VPN?}
    App -->|wrong src IP| NAT[NAT / SNAT / VIP]
    App -->|on-net vs off-net| Proxy
    App -->|site-to-site subnet| VPN[VPN overlap / ACL]
```

3. Syntax and examples

From `labs/10_network_troubleshooting/diagnose.py`:

```

Symptom(
    "Browser reaches the VIP but the app sees the load-balancer IP as client IP",
    "NAT / SNAT on the load balancer or firewall",
    "Compare source IP in app logs with the real client; check X-Forwarded-For",
)
Symptom(
    "TCP SYN is sent, no SYN-ACK. Packet capture shows no return traffic",
    "Transport port blocked by firewall or security group",
    "Confirm destination port (80/443/830) is allowed in both directions",
)
Symptom(
    "Works off-network, fails on corporate Wi-Fi. TLS handshake to api.example.com stalls",
    "HTTP/HTTPS proxy intercept or missing proxy config",
    "Check HTTPS_PROXY/HTTP_PROXY and corporate SSL inspection trust store",
)
Symptom(
    "Site-to-site users cannot reach 10.10.20.0/24. Local LAN uses the same prefix",
    "VPN overlap / interesting-traffic ACL",
    "Inspect local and remote proxy IDs / encryption domains for overlapping subnets",
)

```

Python `requests` proxy usage (original):

```

proxies = {"https": "http://proxy.corp.example:8080"}
requests.get("https://api.meraki.com/api/v1/organizations",
             headers={"X-Cisco-Meraki-API-Key": key},
             proxies=proxies, timeout=30)

```

If this works on corp Wi-Fi and the no-proxy version works at home, you diagnosed **proxy**.

4. Exam-style understanding

Original study items — practice picking **exactly one** of the four named causes:

Item A. `ncclient` to 10.10.20.48:830 from the internet times out; SSH :22 from the jump host on the mgmt VRF works.

Answer: Transport port blocked (830 not allowed on the internet path) — not NAT unless the stem shows rewritten IPs.

Item B. API client IP allow-list has 10.20.0.50 but the SaaS vendor logs 203.0.113.5.

Answer: NAT.

Item C. Same Python works at home; at work `SSLError: certificate verify failed`.

Answer: Proxy (SSL inspection) until proven otherwise.

Item D. Two companies merge; both use 10.10.20.0/24 ; tunnel is up; those prefixes fail.

Answer: VPN overlap.

Item E. HTTP 200 from RESTCONF but users cannot reach the VIP :443.

Answer: Not a Python bug. Port blocked or NAT on the **data** path (6.5). Diagnose with the four causes on the user path.

5. Hands-on exercise

1. Run `python labs/10_network_troubleshooting/diagnose.py` and recap each case without reading the `likely_cause` line first (cover it).
 2. Using the 6.4 diagram, invent one extra symptom per cause (NAT, blocked port, proxy, VPN) and write the first check.
 3. Intentionally set `HTTPS_PROXY` to a closed port in a scratch `requests` call and read the error — that is a proxy failure mode.
-

6.9 Explain the impacts of network constraints on applications

1. What Cisco expects me to know

The verb is **Explain the impacts of network constraints on applications**. Constraints to have ready: **latency, jitter, loss, bandwidth, MTU**, and **DNS failure**. Tie each to how a REST API, a voice stream, or a bulk NETCONF `<get>` behaves. This is not a QoS design course.

2. Detailed explanation

Latency (delay). Round-trip time. Impact: every synchronous API call waits. A script that GETs 500 Meraki devices **sequentially** feels $500 \times \text{RTT}$. Chatty NETCONF (hello + many small rpcs) suffers more than one filtered `get-config`. Humans notice >100–200 ms on interactive apps; storage replication and clustered DBs notice much less. Timeout values in `requests.get(..., timeout=30)` must exceed worst-case latency plus processing.

Jitter (variation in delay). Impact: real-time media (voice/video) glitches even when *average* latency is fine. Bulk file transfer and REST mostly care about mean latency and loss, not jitter. If the exam mentions Webex media vs Webex REST, jitter hits media.

Loss (dropped packets). TCP retransmits → goodput collapses, API calls hang then succeed slowly, or `timeout`. UDP (voice, some DNS) does not retransmit — loss is audible or causes resolution failure. A firewall that silently drops (6.8) is 100% loss on that 5-tuple.

Bandwidth (capacity). Impact: large YANG operational GETs, IOS image transfers, Docker pulls, Terraform provider downloads. Low bandwidth with large payloads → long jobs, CI timeouts. A 200 OK REST call with a tiny JSON body is rarely bandwidth-bound.

MTU (maximum transmission unit). Typical Ethernet 1500; IP/TCP headers eat some; VPN encapsulation eats more. Impact: packets that do not fit are fragmented or, with DF bit, **dropped**. Symptom: small pings work, large HTTPS POST or NETCONF RPC fails, or TCP stalls after the handshake (“PMTUD blackhole”). Automation over IPsec VPN (6.8) is a classic MTU victim.

DNS failure. Impact: the application never starts a TCP session to the right IP. `requests` raises a resolution error. Cached IPs may work until TTL expires. Split DNS on VPN can resolve an **internal** IP that is unreachable from the current tunnel — looks like a mysterious timeout (combine with 6.8).

Other constraints worth one sentence: congestion (queues add latency/loss), NAT timeouts (long-lived idle API websockets die), TLS inspection adding latency.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    App[Application] --> DNS[DNS must work]
    DNS --> RTT[Each API RTT adds latency]
    RTT --> Loss[Loss causes retries/timeouts]
    Loss --> BW[Large payloads need bandwidth]
    BW --> MTU[Path MTU must fit VPN]
```

3. Syntax and examples

Original mapping table:

Constraint	Typical application impact	Automation example
High latency	Slow pages, sequential API loops crawl	200 RESTCONF GETs in a <code>for</code> loop
Jitter	Voice/video artifacts	Webex media, not Webex room JSON
Loss	Timeouts, retransmits, truncated downloads	<code>requests</code> timeout; flaky CI
Low bandwidth	Long transfers, failed image upgrades	IOS image, <code>terraform init</code> providers
MTU too small / PMTUD fail	Large packets fail; small work	NETCONF RPC over VPN
DNS down	Immediate name errors	Cannot reach <code>api.meraki.com</code>

`diagnose.py` does not list MTU; add it mentally as a fifth constraint when “small works, large fails.”

Timeout example:

```
requests.get(url, timeout=(3.05, 27)) # connect timeout, read timeout
```

If RTT is 200 ms, connect usually succeeds; if the YANG payload is huge on a thin link, **read** timeout fires — bandwidth/latency impact, not a 404.

4. Exam-style understanding

Original study items:

Item A. Voice is choppy; REST monitoring is fine. Which constraint?

Answer: Jitter (or loss on UDP), not bandwidth for tiny JSON.

Item B. `get-config` of the entire running config over a 256 kbps VPN times out; a filtered interface GET works.

Answer: Bandwidth / payload size (and maybe MTU), not “NETCONF is down.”

Item C. Ping 32-byte succeeds; ping 1500 fails over the tunnel; API POST fails.

Answer: MTU / encapsulation.

Item D. Failed to resolve 'dnac.lab.local' from GitHub Actions but works on the laptop.

Answer: DNS (public CI cannot see internal DNS) — impact: pipeline cannot call the controller.

Item E. Explain why Ansible `serial: 1` against 50 high-latency devices is slow.

Answer: Each SSH round trip waits full RTT; constraint is latency amplified by serialization.

5. Hands-on exercise

1. Run `diagnose.py` again and add a spoken sentence for each case: which 6.9 constraint is involved (loss for blocked port, latency for proxy inspection, etc.).
2. Time a RESTCONF GET against the sandbox (`Measure-Command` or `time python restconf_get_interfaces.py`). That number is mostly latency + server time.
3. Write five flashcards: constraint on the front, application impact on the back.

Domain 5–6 study checklist

Use this against the official 200-901 CCNAAUTO v1.1 blueprint. You are ready to move on when you can teach each line without notes.

Objective	Verb	Artifact to practice
5.1	Describe	YANG vs NETCONF vs RESTCONF value
5.2	Compare	Controller URL vs <code>/restconf/data</code>
5.3	Describe	CML simulate; pyATS parse/test/diff
5.4	Describe	Git → lint → test → plan → deploy
5.5	Describe	Desired state, versioned, reviewable, repeatable
5.6	Describe	Ansible YAML/SSH; Terraform HCL/state; NSO services
5.7	Identify	<code>labs/05_cisco_apis/*</code> + RESTCONF script
5.8	Interpret	<code>labs/08_ansible/playbook.yml</code>
5.9	Interpret	Bash: <code>cd</code> , <code>apt</code> , <code>useradd</code> , <code>mkdir</code> / <code>cp</code>
5.10	Interpret	<code>sample_restconf_get.json</code> , <code>sample_netconf_get-config.xml</code>
5.11	Interpret	<code>labs/06_yang_netconf_restconf/sample.yang</code>
5.12	Interpret	<code>labs/04_git/example.diff</code>
5.13	Describe	PR + diff + CI gates
5.14	Interpret	Mermaid sequences in 5.14
6.1	Describe	MAC L2 identity; VLAN broadcast domain
6.2	Describe	IP, mask/prefix, gateway, routes
6.3	Describe	Switch, router, firewall, LB functions
6.4	Interpret	Topology mermaid in 6.4
6.5	Describe	Mgmt vs control vs data
6.6	Describe	DHCP DNS NAT SNMP NTP
6.7	Recognize	22 23 80 443 830 (+ 53 67/68 123 161/162)
6.8	Diagnose	<code>labs/10_network_troubleshooting/diagnose.py</code>
6.9	Explain	Latency jitter loss bandwidth MTU DNS

Authoritative references used in this chapter

- Cisco Modeling Labs: <https://developer.cisco.com/modeling-labs/>
- pyATS: <https://developer.cisco.com/pyats/>
- Ansible docs: <https://docs.ansible.com/>
- Terraform docs: <https://developer.hashicorp.com/terraform/docs>

- Cisco NSO: <https://developer.cisco.com/docs/nso/>
- NETCONF RFC 6241: <https://datatracker.ietf.org/doc/html/rfc6241>
- RESTCONF RFC 8040: <https://datatracker.ietf.org/doc/html/rfc8040>
- YANG RFC 7950: <https://datatracker.ietf.org/doc/html/rfc7950>
- Cisco IOS XE RESTCONF: https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/prog/configuration/1717/b_1717_programmability_cg/restconf-protocol.html
- DevNet Sandbox: <https://devnetsandbox.cisco.com>

This chapter is original instructional material aligned to the published blueprint. It does not reproduce Cisco exam questions.